SAMSUNG

Samsung Innovation Campus

Internet of Things Course

Together for Tomorrow!
Enabling People
Education for Future Generations

Chapter 9.

Phát triển thiết bị IoT SmartThings thông qua tái sử dụng Galaxy Upcycling

Vạn vật kết nối – IoT

Mô tả chương

Mục tiêu các chương

- Tìm hiểu về Galaxy Upcycling và cách tái sử dụng các thiết bị cũ để triển khai các dịch vụ IoT thiết yếu.

Nội dung các chương

- Bài 1. Galaxy Upcycling: Tổng quan
- Bài 2. Galaxy Upcycling: Hướng dẫn thực hành (Cơ bản)
- Bài 3. Sử dụng SmartThings Device SDK để điều khiển MCU (Nâng cao)
- Bài 4. Cấu hình và kiểm thử cuối: Bóng đèn thông minh

Bài 1.

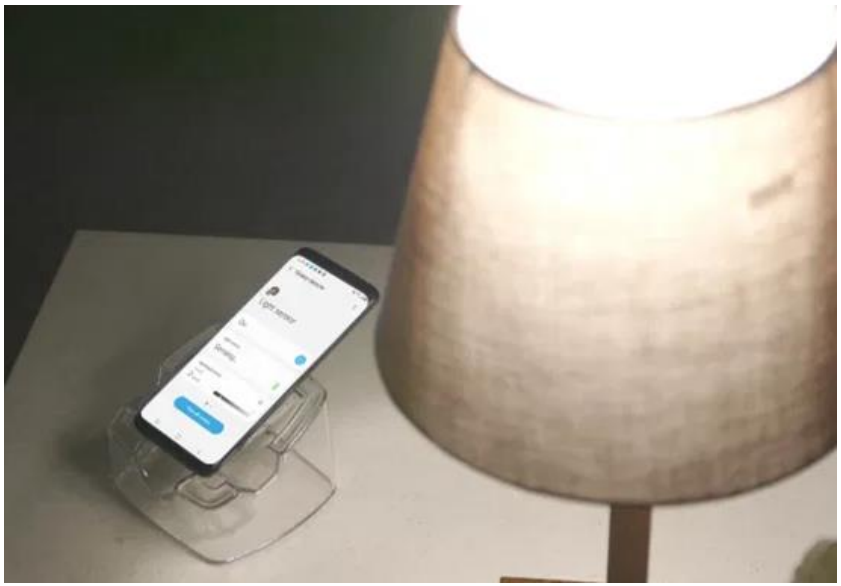
Galaxy Upcycling: Tổng quan

- 1.1. Giới thiệu về Galaxy Upcycling
- 1.2. Mục tiêu và cấu trúc tổng thể của kiến trúc phần mềm (S/W) và phần cứng (H/W)

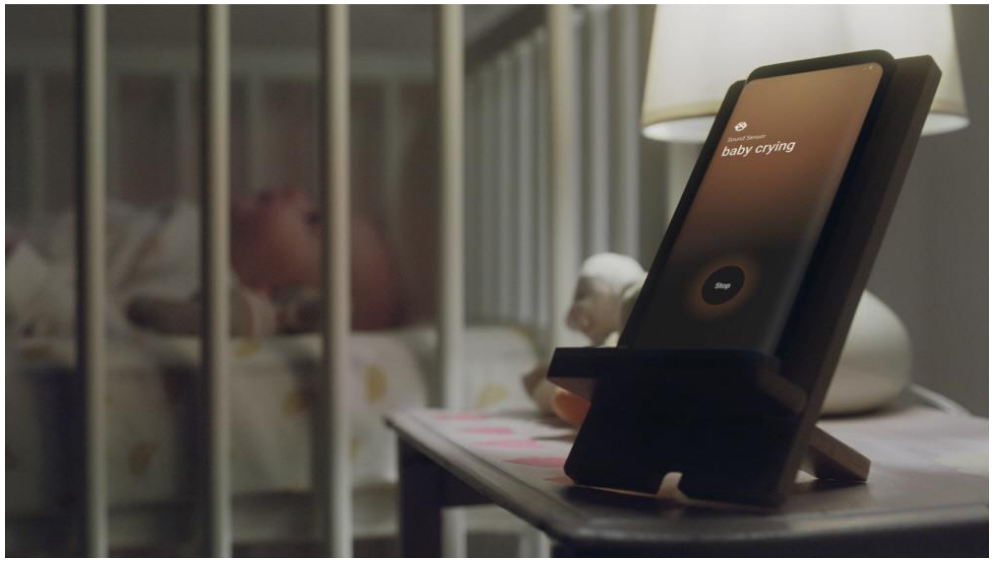
Sáng kiến này thúc đẩy việc tái sử dụng các thiết bị điện tử cũ như một giải pháp sáng tạo nhằm giảm thiểu rác thải điện tử, đồng thời mang lại sự tiện lợi cho người dùng trong đời sống hằng ngày.

Với Galaxy Upcycling at Home, người dùng có thể dễ dàng chuyển đổi các thiết bị Galaxy cũ thành thiết bị nhà thông minh như hệ thống giám sát trẻ em, giải pháp chăm sóc thú cưng và các công cụ khác phù hợp với nhu cầu cá nhân.

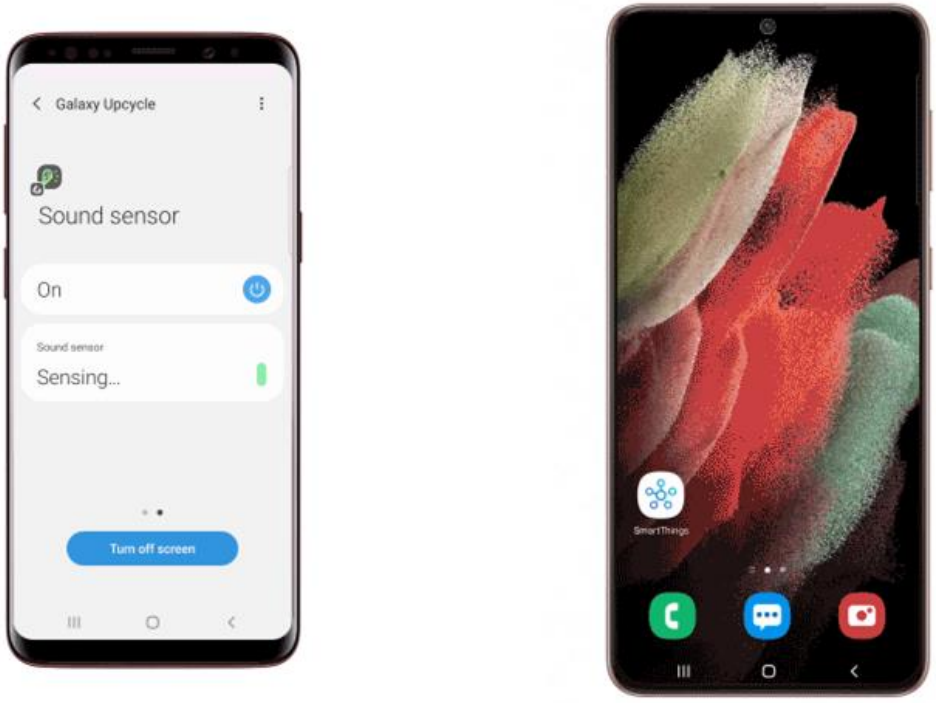
Người dùng có thể chuyển đổi các thiết bị cũ thông qua SmartThings Labs, một tính năng trong ứng dụng SmartThings.



Giải pháp đổi mới này là một phần trong cam kết liên tục của Samsung nhằm hỗ trợ Chương trình Nghị sự 2030 vì Phát triển Bền vững, một định hướng toàn cầu hướng tới việc xây dựng tương lai bền vững và thịnh vượng hơn cho tất cả mọi người thông qua 17 Mục tiêu Phát triển Bền vững (SDGs). Galaxy Upcycling at Home tập trung vào Mục tiêu số 12 – Tiêu dùng và Sản xuất có trách nhiệm – thông qua việc giảm thiểu rác thải và thúc đẩy các phương thức sản xuất cũng như hành vi tiêu dùng bền vững hơn.

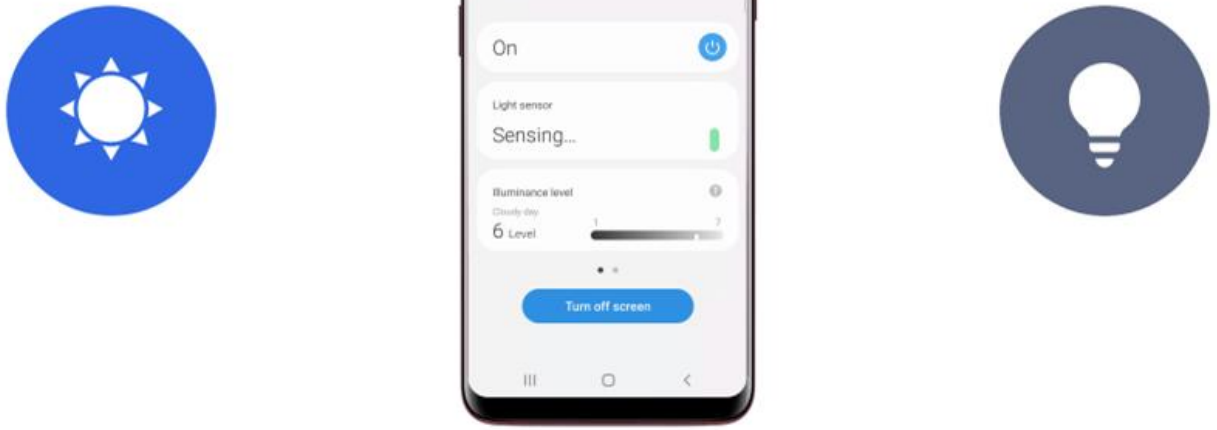


Sound Sensor



Light Sensor

Your older device detects when it gets dark outside. It then signals the lights in your home to turn on, so your pets and your home aren't stuck in the dark.

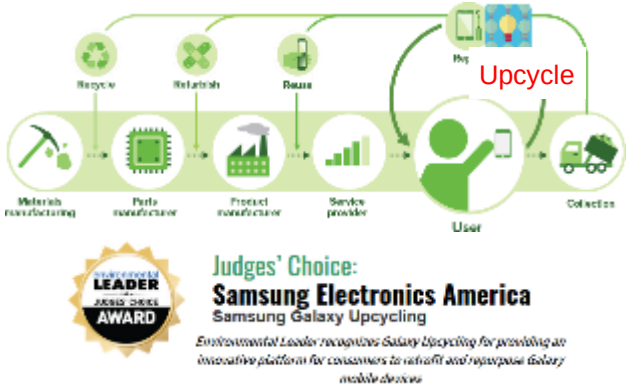


- Galaxy Upcycling là một chương trình tuần hoàn sinh thái nhằm tái sử dụng các nguồn tài nguyên hiện có vào những mục đích mới một cách hiệu quả.
  - Đây là một phương pháp mang lại giá trị gia tăng cao hơn so với tái chế, do có thể kéo dài vòng đời sản phẩm và giảm chi phí cơ hội trong việc sản xuất sản phẩm mới.



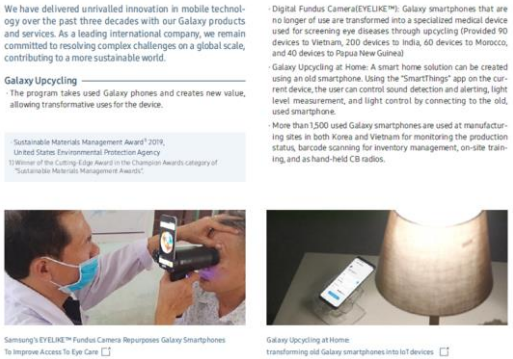
Năm 2018, 2020 (Hoa Kỳ, EPA)

Giải thưởng SMM của EPA, Hạng mục Quán quân – Công nghệ tiên phong



Tháng 5/2018 (Hoa Kỳ, Giải do Ban giám khảo lựa chọn)

Environmental Leader – Dự án của năm



2018~ 2020

Báo cáo Phát triển Bền vững của Samsung

- Galaxy Upcycling at Home được vinh danh tại Reuters Responsible Business Awards

  - Ban giám khảo nhận định rằng giải pháp upcycling của Samsung là: “Một sáng kiến đơn giản với ý tưởng cơ bản, nhưng góp phần thay đổi nhận thức về giá trị của điện thoại cũ. Đây là một mô hình vòng tuần hoàn thực sự, khác biệt so với tái chế rác thải điện tử, với cách tiếp cận sáng tạo.”

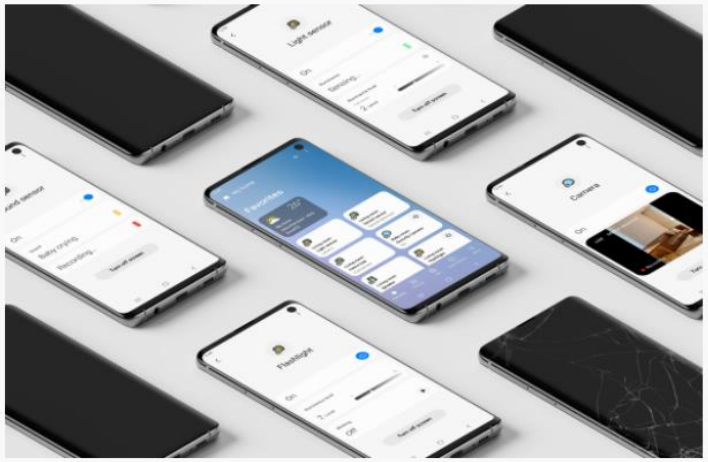


Tháng 10/2021 (Hoa Kỳ, Reuters Events: Responsible Business 2021)

Giải Khuyến nghị cao (Highly Commended) – Hạng mục Chuyển đổi Kinh tế Tuần hoàn

- Galaxy Upcycling at Home được vinh danh tại Giải thưởng Thiết kế iF 2022





DESIGN AWARD 2022

Share:

User Experience (UX) | Communication UX

Galaxy Upcycling at Home

Home IoT Application UX

Samsung Electronics Co., Ltd.

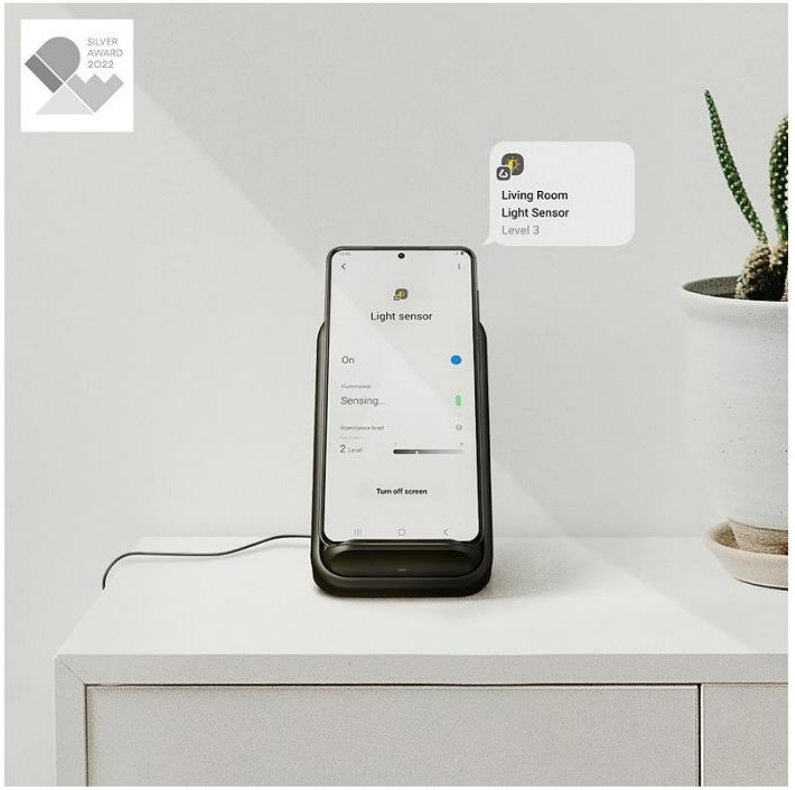
Galaxy Upcycling at Home is a sustainable design concept that allows the user to transform their decommissioned Galaxy smartphone into a home IoT device. Download the Galaxy Upcycle App onto the Galaxy phone and sync with SmartThings to not only monitor the home in real-time but also gain access to automated functions in sync with other IoT devices. This contributes to more people recycling resources for a more sustainable future.

Client / Manufacturer

SAMSUNG

Samsung Electronics Co., Ltd.
Seoul, KR

- Galaxy Upcycling at Home được vinh danh với Giải Bạc IDEA 2022



Another Silver winner was the Galaxy Upcycling at Home, a program that gives new life to older Galaxy smartphones by repurposing them into IoT devices with a simple software update. Through the SmartThings app, users can repurpose unused smartphones into convenient smart home devices such as a childcare monitor or a pet care.

- Galaxy Upcycling hướng tới xây dựng nền kinh tế tuần hoàn thông qua việc tái sử dụng.
- Trong chương này, bạn sẽ xây dựng các ứng dụng IoT thực tiễn bằng ứng dụng Galaxy Upcycle và nền tảng SmartThings.

- Các thiết bị/vật tư cần thiết cho Galaxy Upcycling



Hình ảnh minh họa
Bộ kit Galaxy
Upcycling



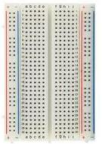
Old Galaxy (S9 or Note9) 1ea



USB Type-C Adapter & Cable (Charging for old Galaxy) 1ea



ESP32 Dev board & Micro 5p Cable 1ea



Breadboard (400points) 1ea



Micro 5p USB Adapter (Power supply to ESP32) 1ea



Jumper Cable M-M (10P 20cm) 1set



Resistors (220ohm, 10kohm) 2ea



LED (5pi, Milky white) 1ea



Tactile Switch 1ea



Organizer Box 1ea

Nếu học viên muốn tiếp tục phát triển chương trình bằng Galaxy Upcycling Kit sau khi hoàn thành Chương 9, bộ kit này vẫn được cung cấp cho đến khi kết thúc khóa học IoT. Học viên có thể triển khai dự án của mình bằng Galaxy Upcycling Kit trong quá trình thực hiện Capstone Project. Việc công khai dự án trên các nền tảng maker toàn cầu như hackster.io và instructables sẽ giúp xây dựng portfolio chất lượng. Vui lòng tham khảo đường link. Nếu có trường hợp tương tự, vui lòng chia sẻ với CCO.

References

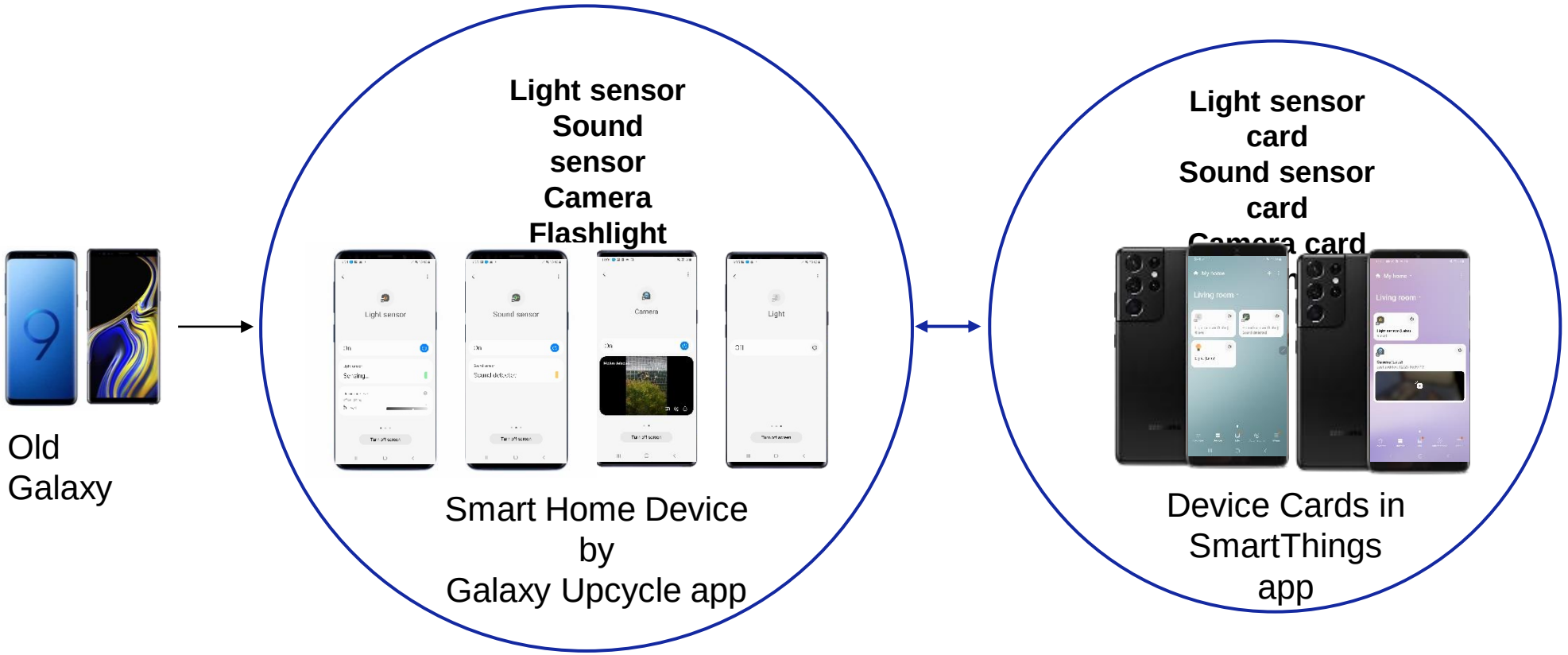
instructables

[Instructables: Xmas Decor] https://me2.kr/zgyhb

hackster.io
AN AVNET COMMUNITY

[Hackster.io: Fish tank lighting upcycle] https://me2.kr/95vej

- Tái sử dụng thiết bị Galaxy cũ thông qua các cảm biến tích hợp (cảm biến ánh sáng, cảm biến âm thanh, camera, v.v.) và điều khiển thông qua ứng dụng SmartThings





MCU (ESP32)

Internet

Smart Things

MQTT

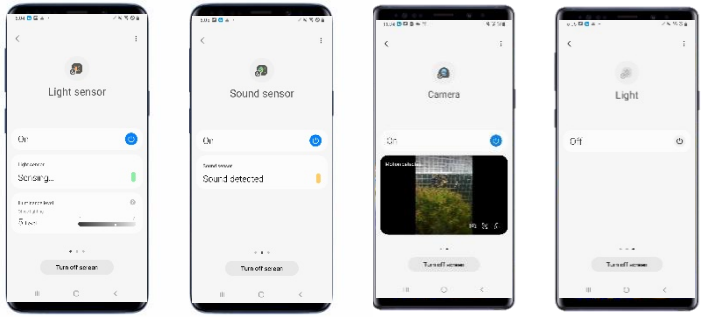
Internet

Internet

Protocol



Using Galaxy with SmartThings app

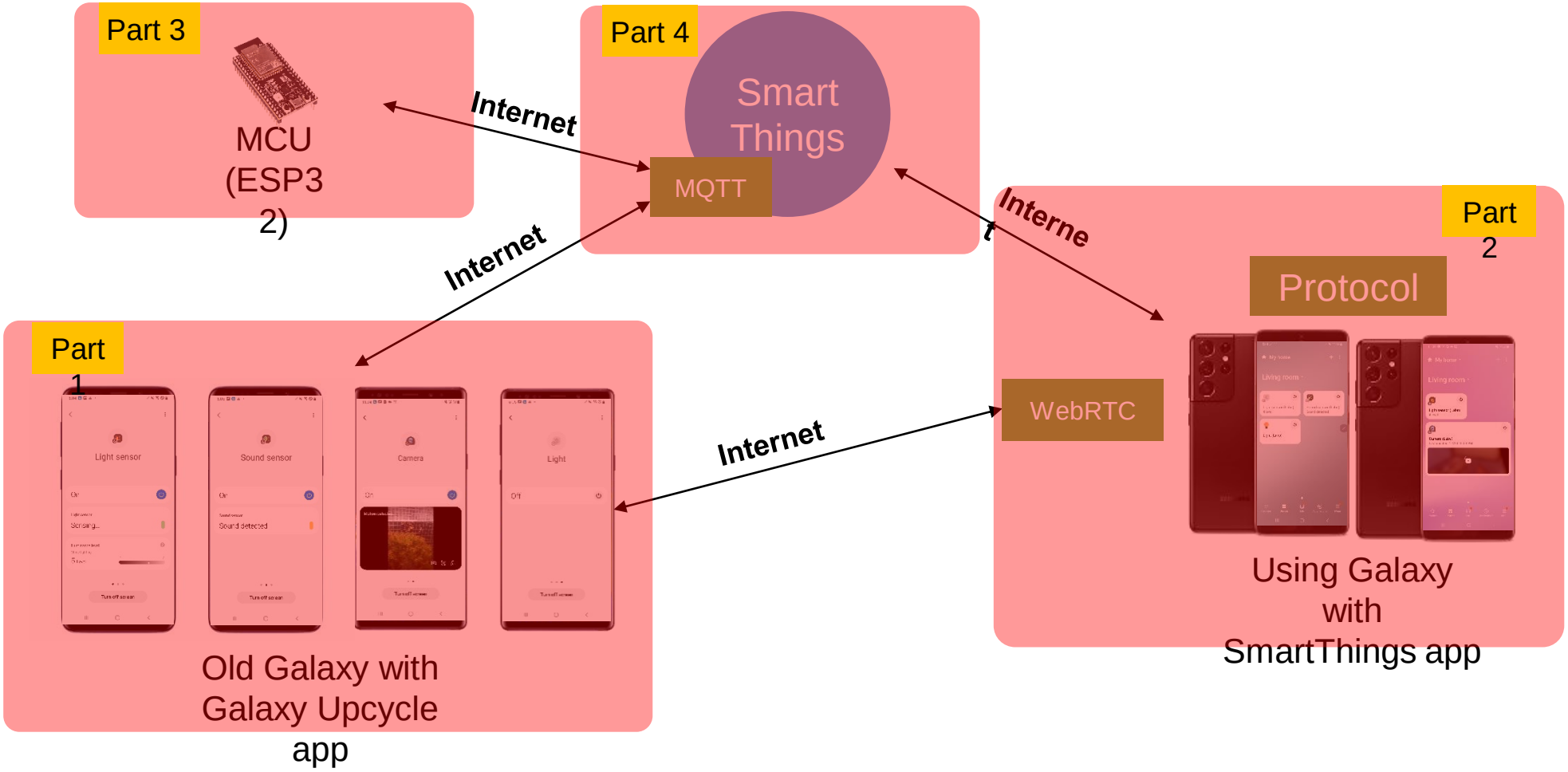


Old Galaxy with Galaxy Upcycle app

Internet

WebRTC

- Hoàn thành từng phần để tạo thành một thiết bị upcycling hoàn chỉnh



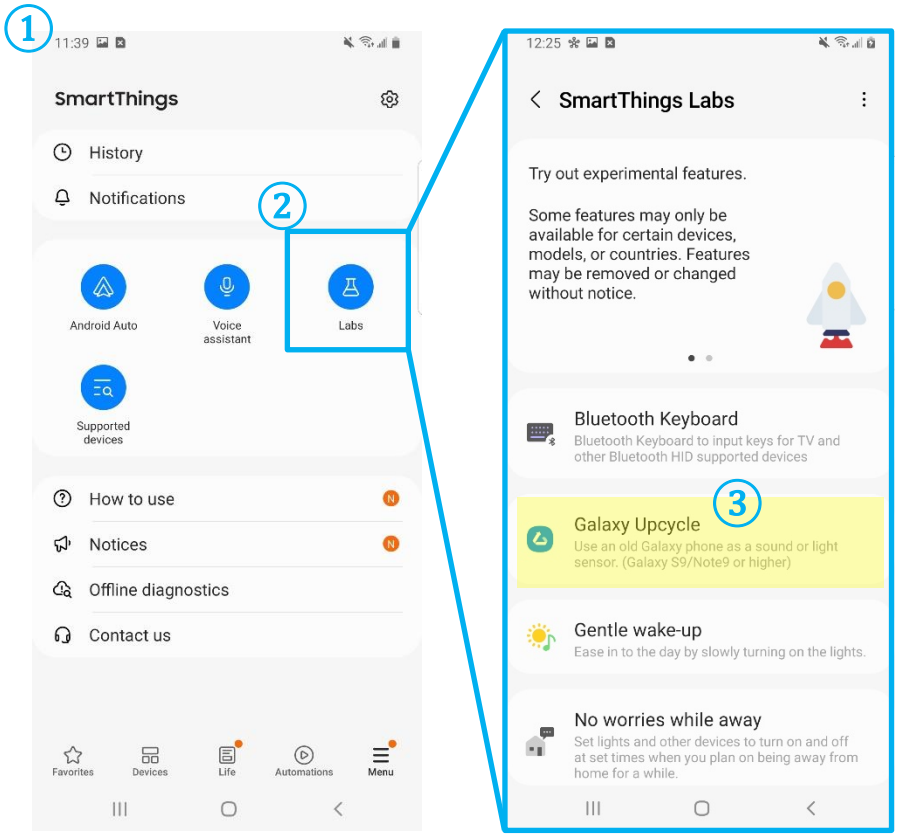
BÀI 2.

Galaxy Upcycling: Hướng dẫn thực hành (Cơ bản)

- 2.1. Cài đặt ứng dụng Galaxy Upcycle
- 2.2. Thêm thiết bị IoT vào SmartThings
- 2.3. Điều khiển thiết bị IoT

2.1. Cài đặt ứng dụng Galaxy Upcycle

- Cài đặt ứng dụng Galaxy Upcycle trên thiết bị Galaxy cũ (chọn mục Labs trong ứng dụng SmartThings)



Quy trình

① SmartThings app

② Labs

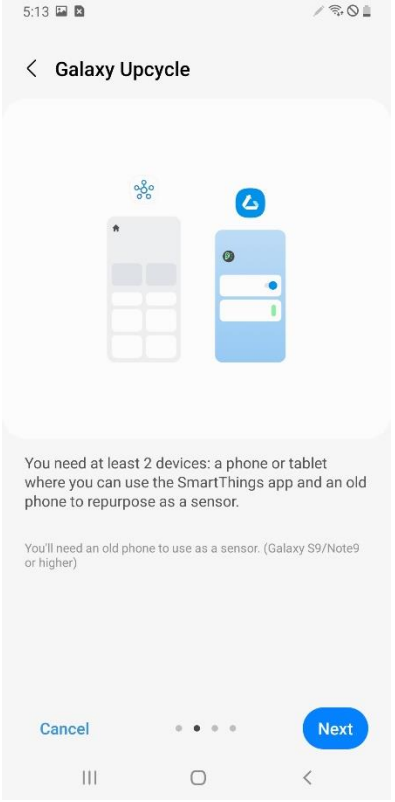
③ Galaxy Upcycle

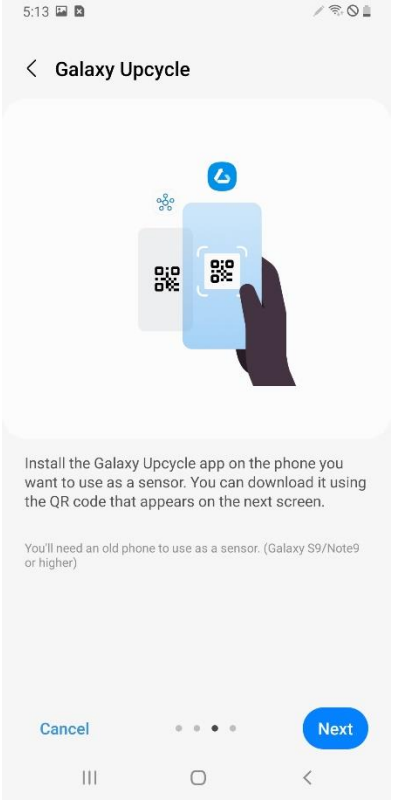
Tính năng Labs của Galaxy Upcycle hiện khả dụng tại Hàn Quốc, Hoa Kỳ và Vương quốc Anh. Tại các quốc gia khác, việc cài đặt chỉ được hỗ trợ cho mục đích đào tạo như chương trình SIC.

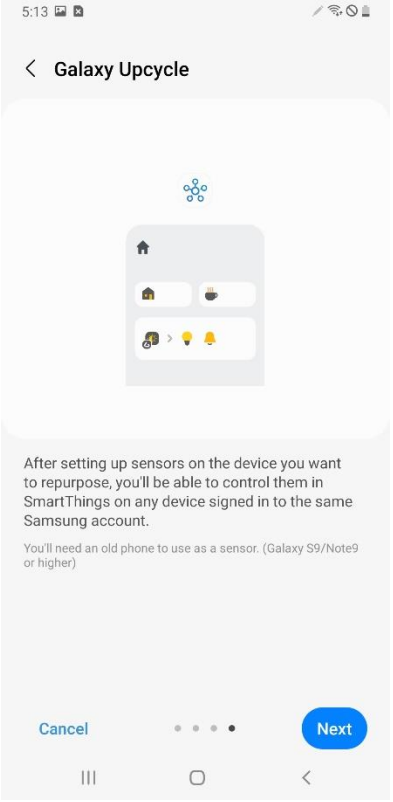
2.1. Cài đặt ứng dụng Galaxy Upcycle

- Cài đặt ứng dụng Galaxy Upcycle trên thiết bị Galaxy cũ (chọn mục Labs trong ứng dụng SmartThings)



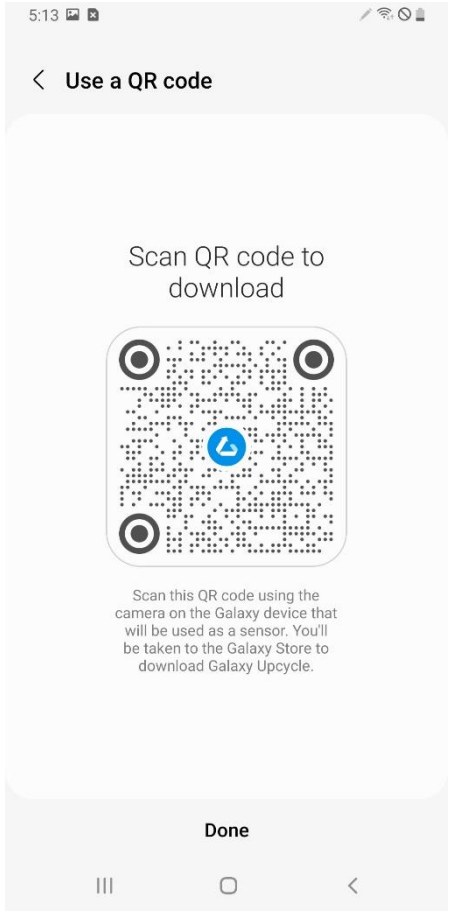


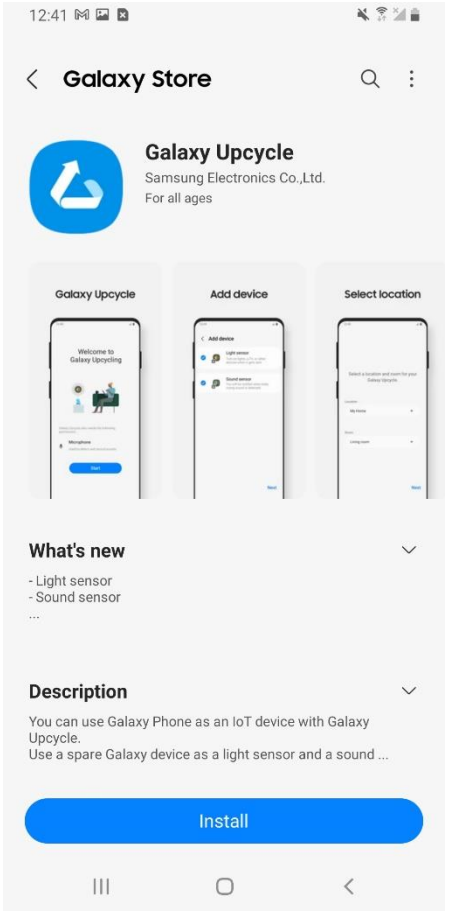


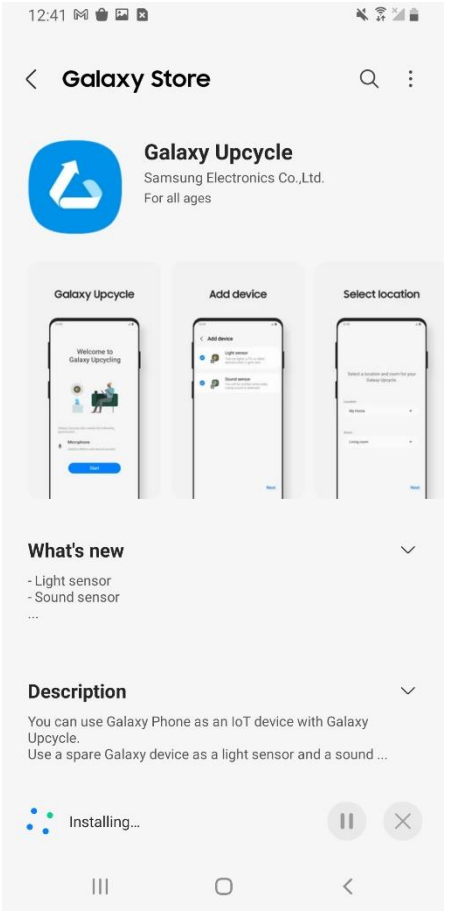


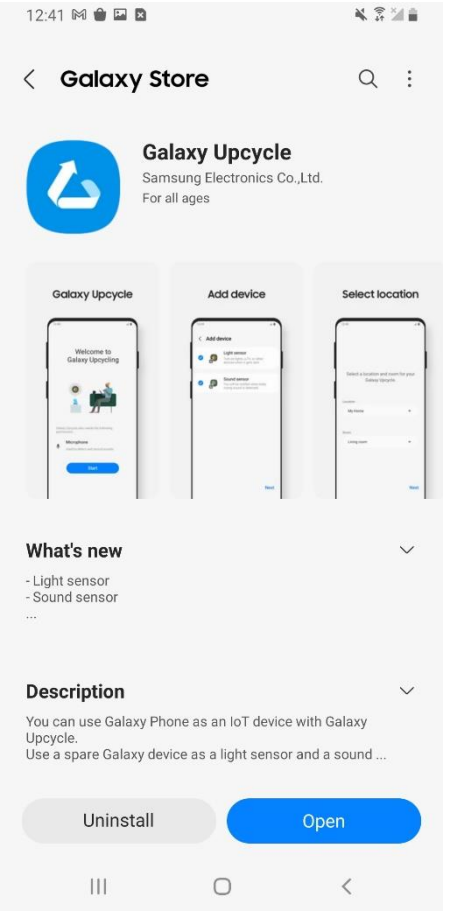
2.1. Cài đặt ứng dụng Galaxy Upcycle

- Cài đặt ứng dụng Galaxy Upcycle trên thiết bị Galaxy cũ (chọn mục Labs trong ứng dụng SmartThings)

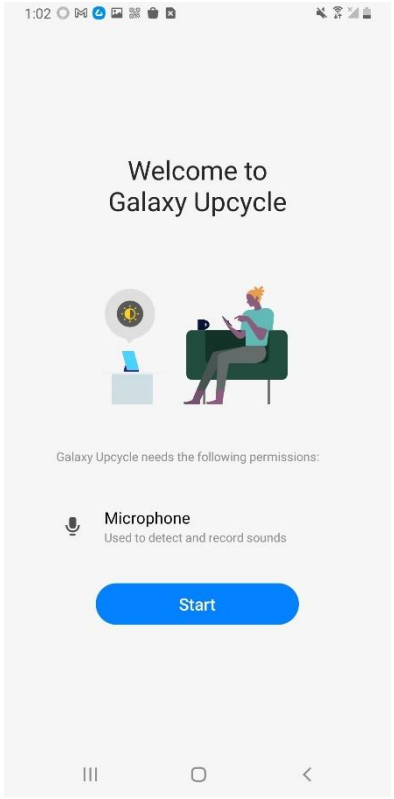


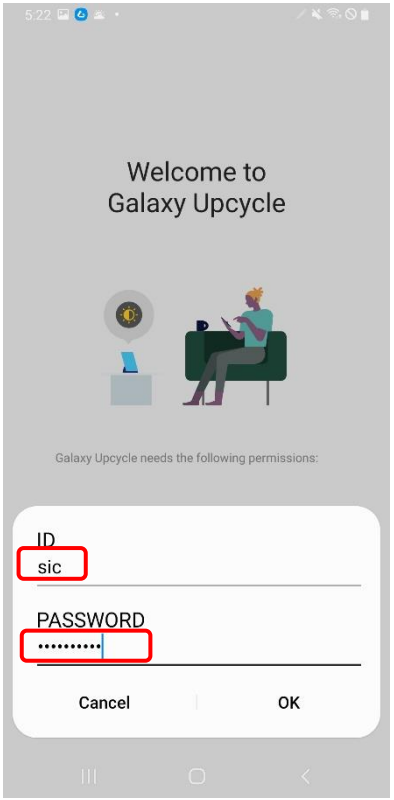


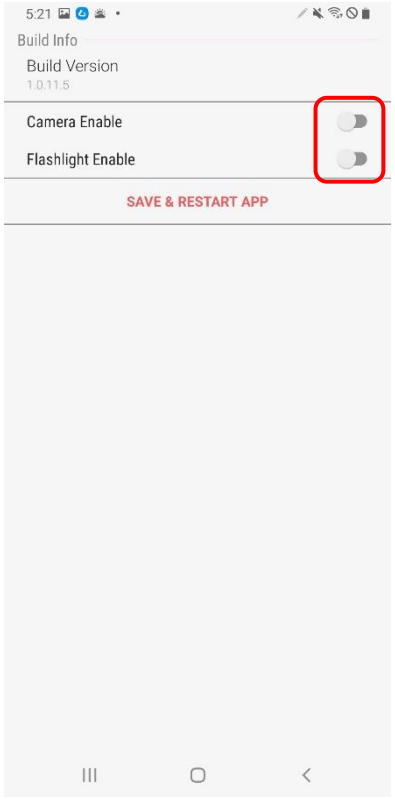


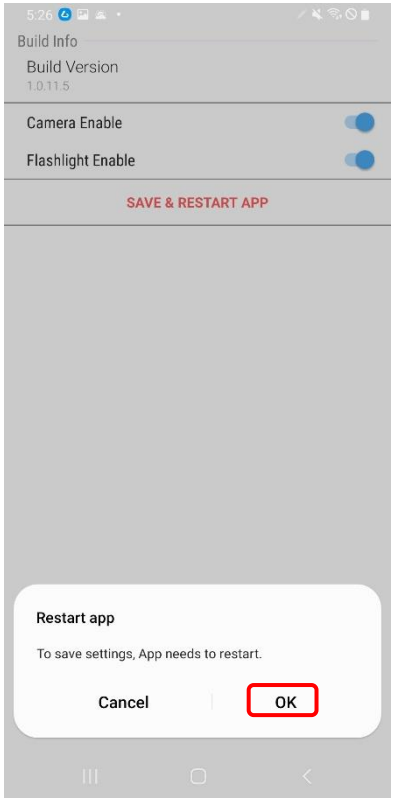


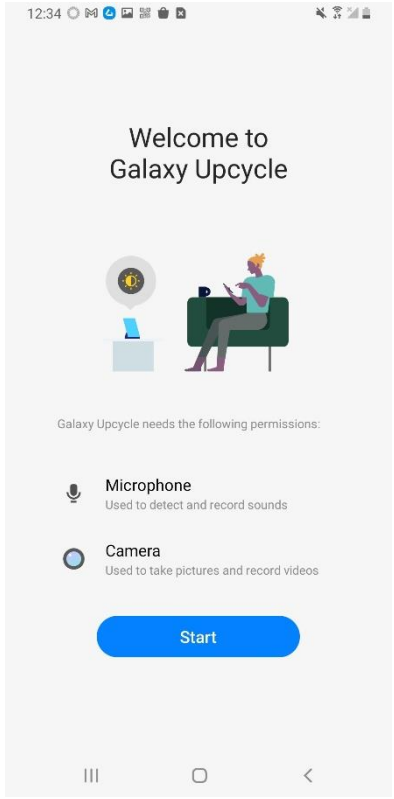
- Kích hoạt trước các tính năng ẩn
  - Để kích hoạt các tính năng ẩn, nhấn vào hình ảnh 10 lần tại trang chào mừng, nhập ID/mật khẩu SIC, sau đó lưu và khởi động lại.



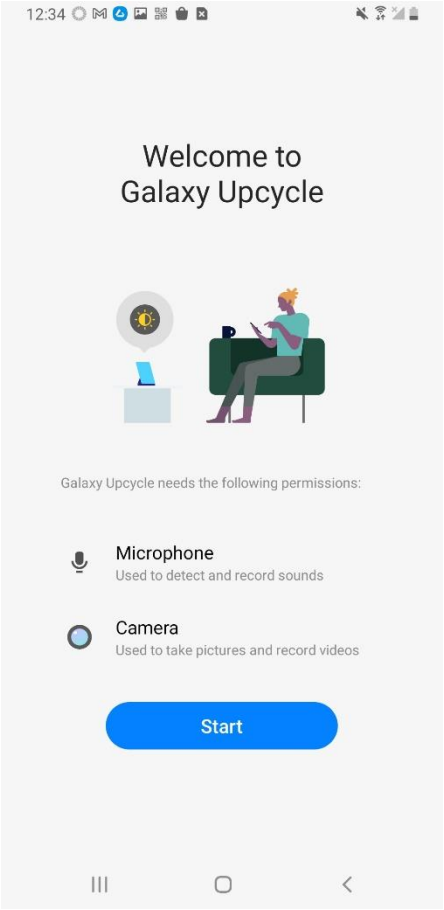


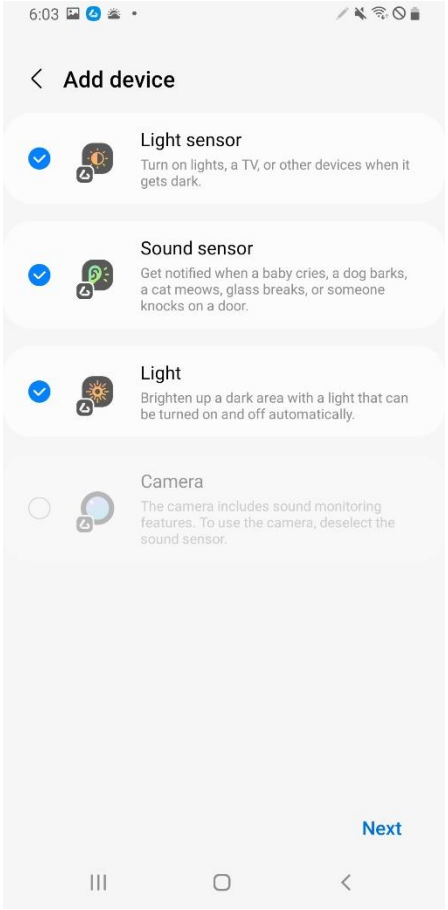


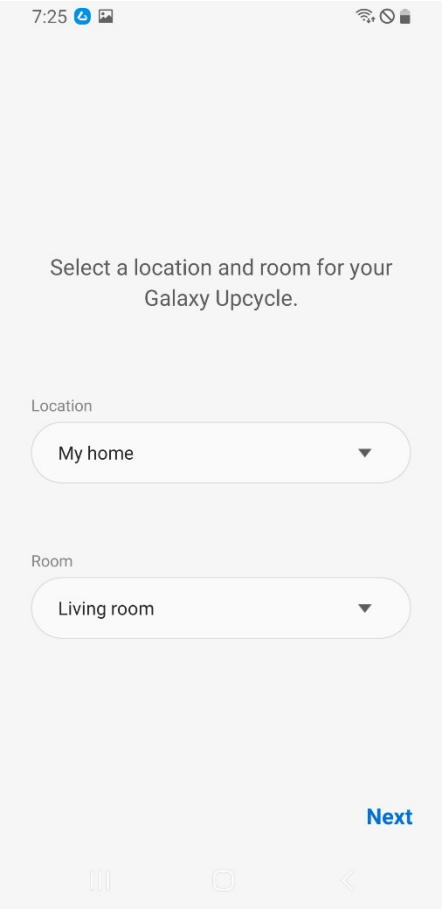


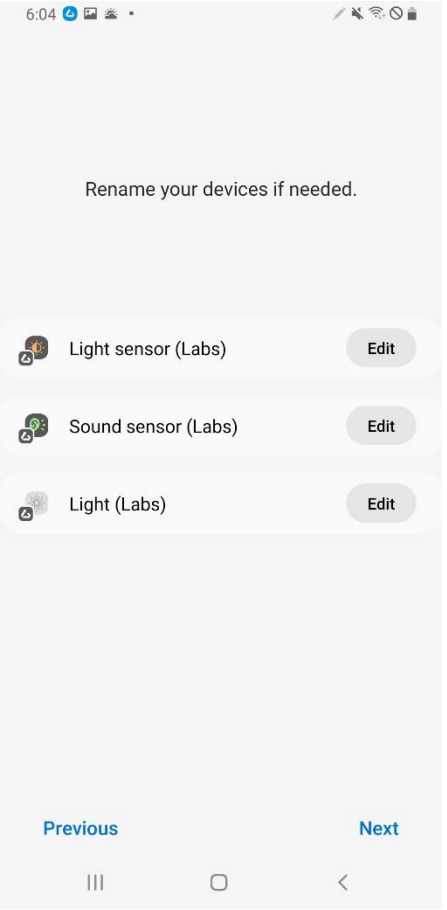


- Thêm cảm biến ánh sáng, cảm biến âm thanh và đèn pin (Tùy chọn 1) (1/2)

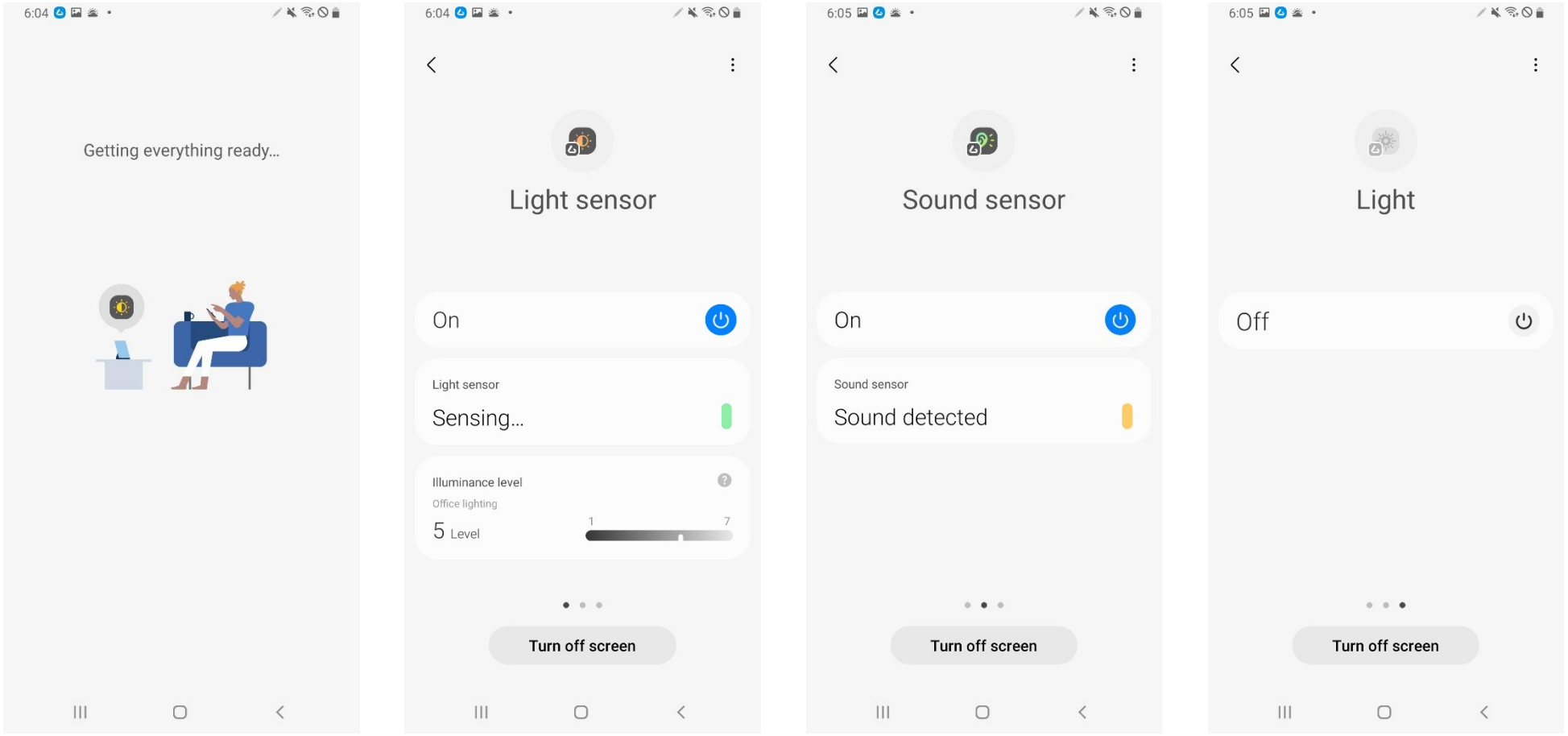






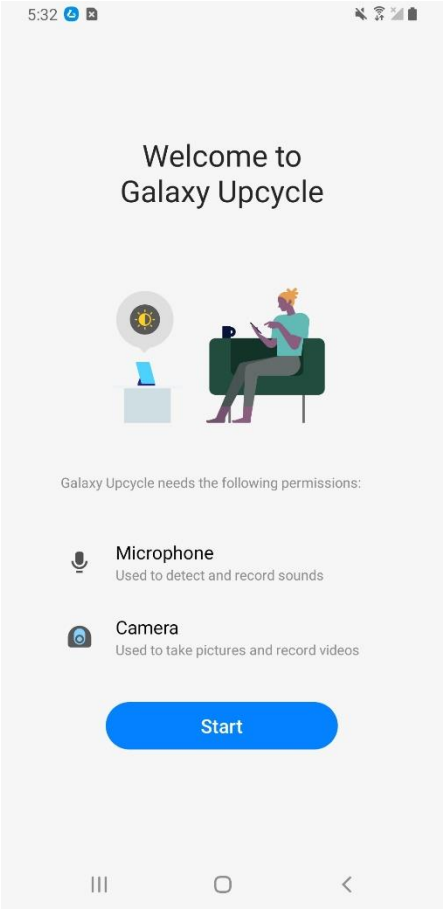


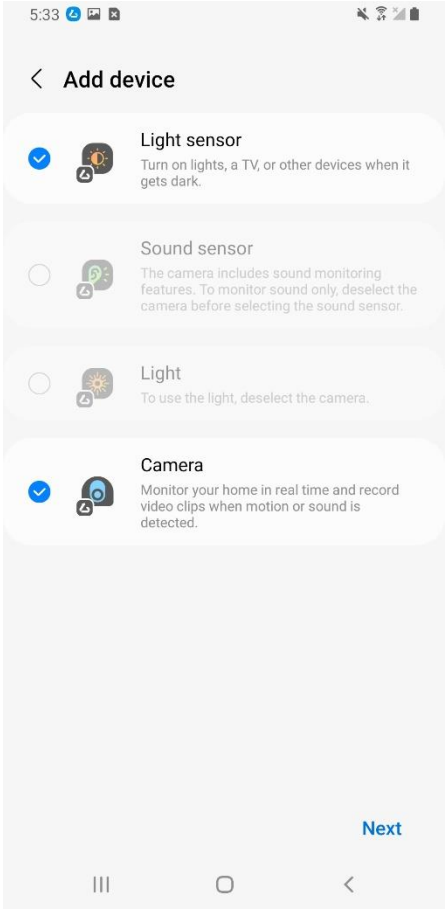
- Thêm cảm biến ánh sáng, cảm biến âm thanh và đèn pin (Tùy chọn 1) (2/2)

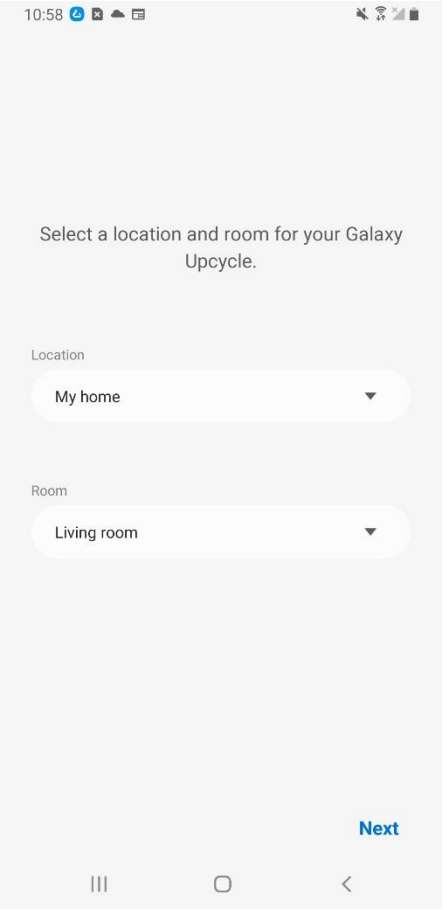


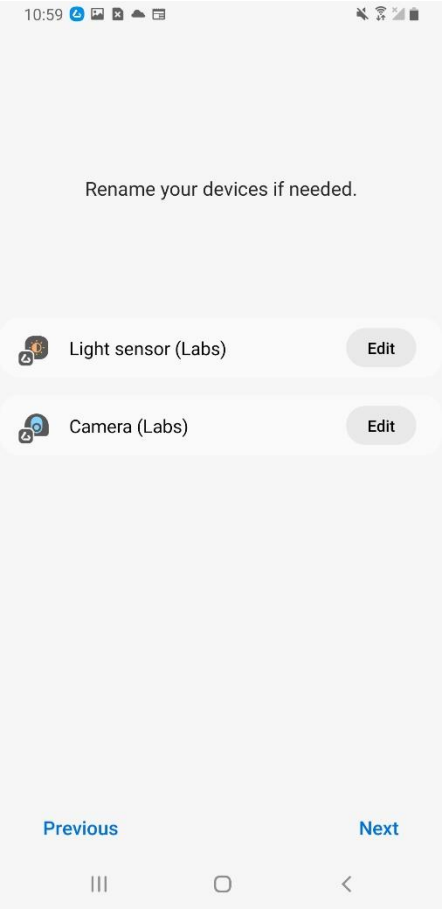
2.2. Thêm thiết bị IoT vào SmartThings

- Thêm cảm biến ánh sáng và camera (Tùy chọn 2) (1/2)

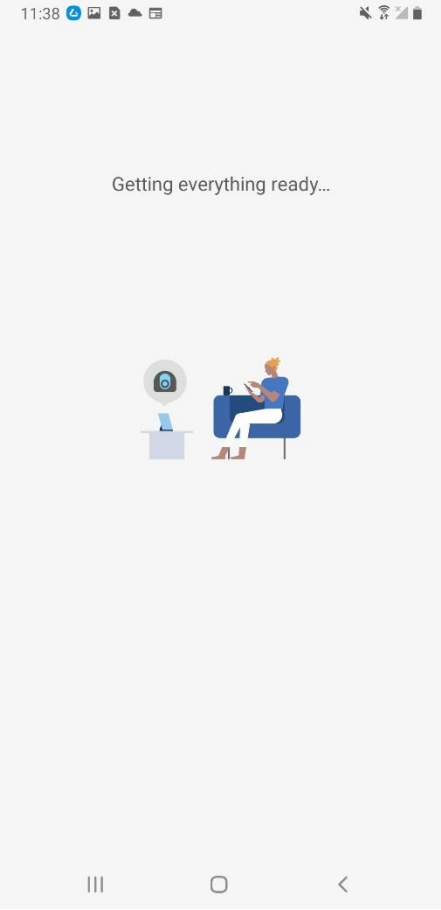


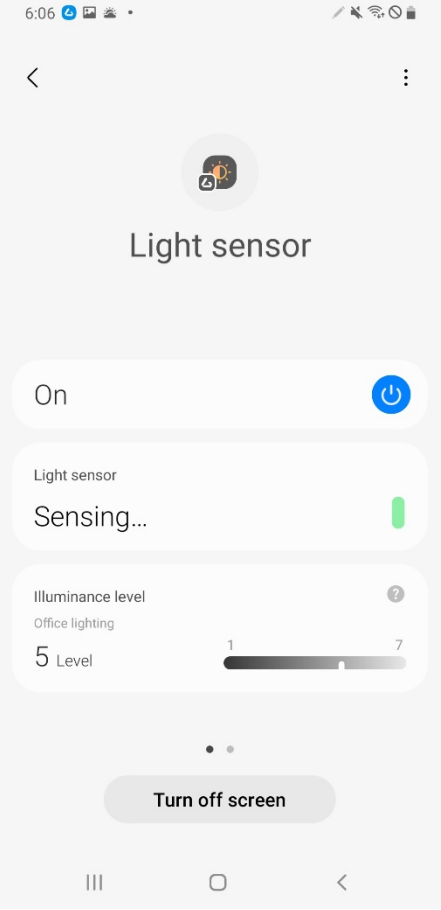


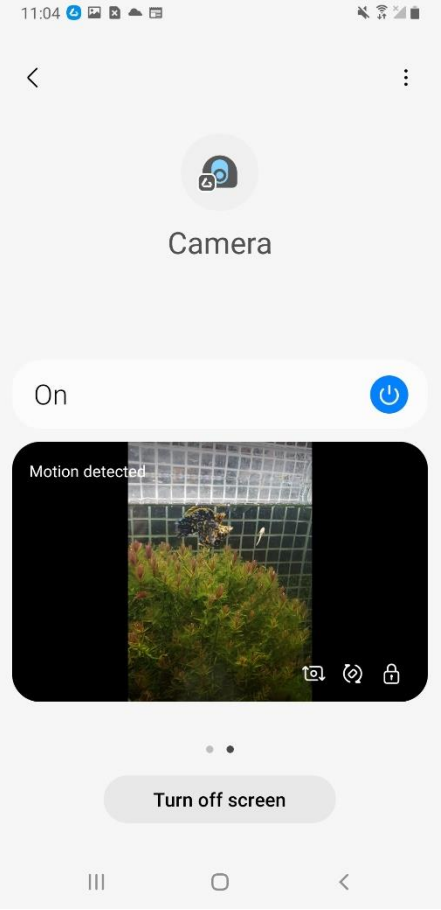


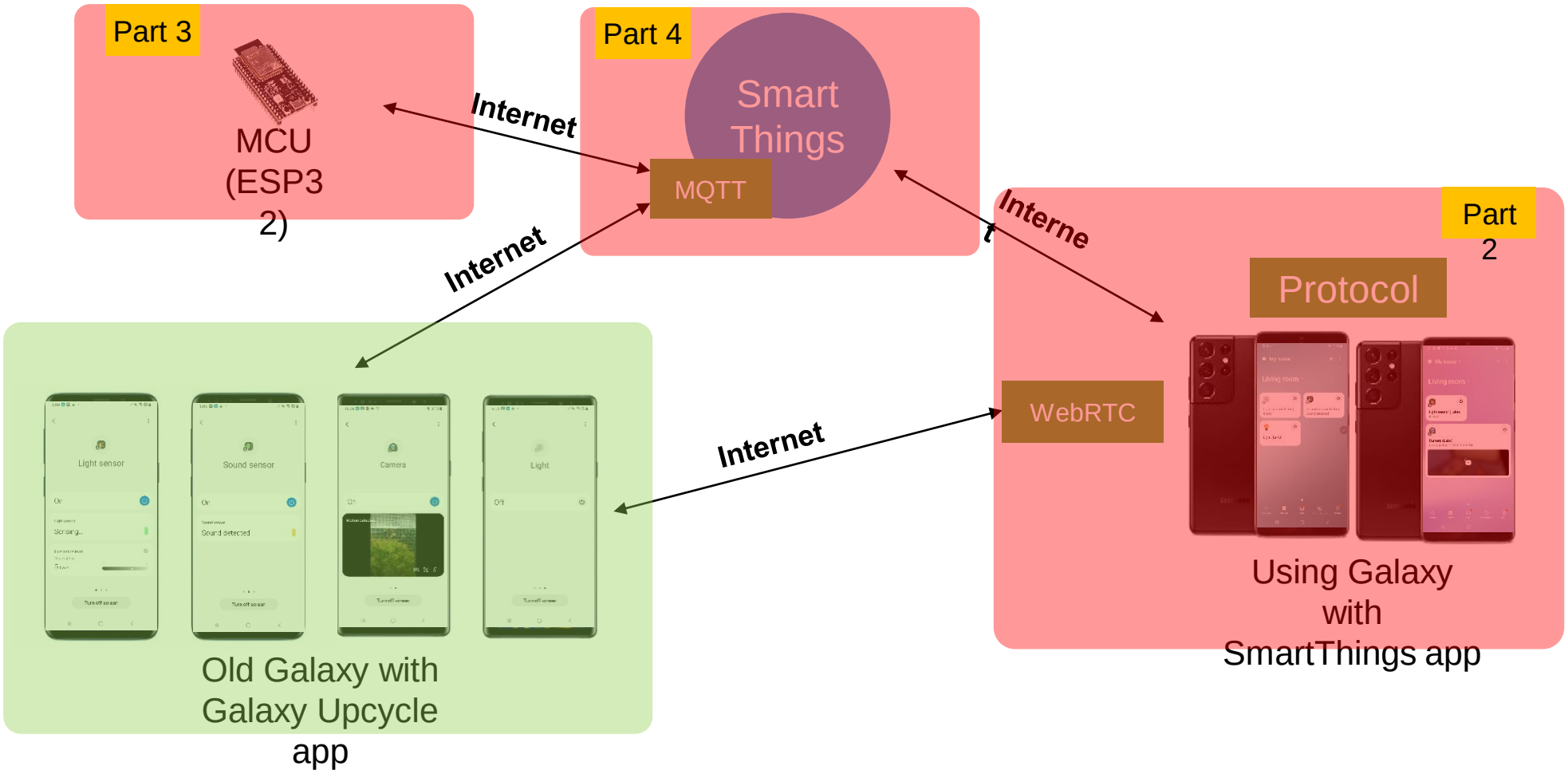


- Thêm cảm biến ánh sáng và camera (Tùy chọn 2) (2/2)





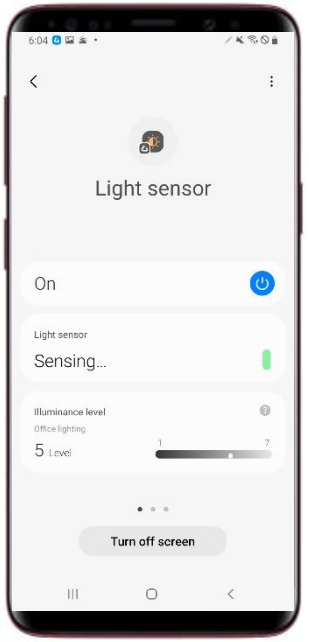



Part 3
MCU (ESP32)
Internet
Part 4
Smart Things
MQTT
Internet
Internet
Part 2
Protocol
WebRTC
Using Galaxy with SmartThings app
Internet
Light sensor
Sound sensor
Camera
Light
Old Galaxy with Galaxy Upcycle app

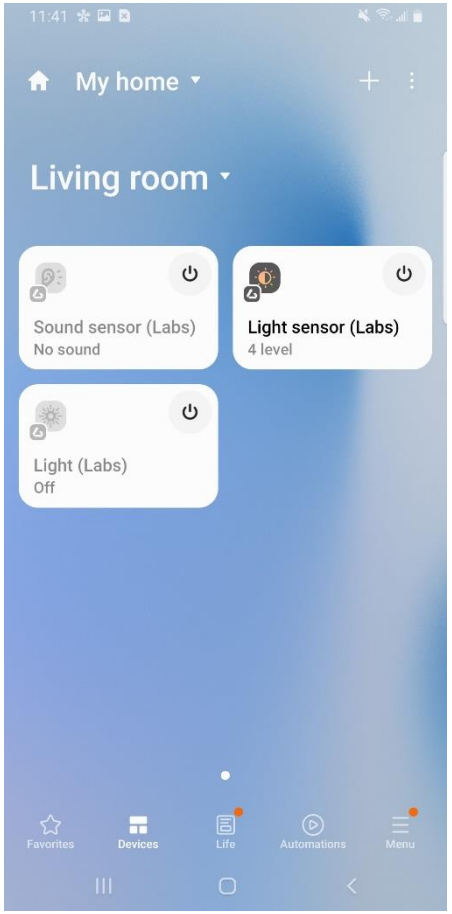
Cảm biến ánh sáng

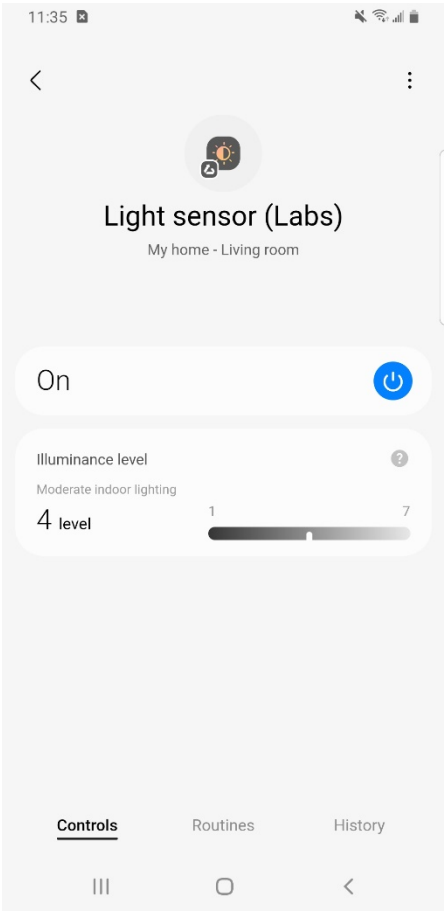
- Thiết bị IoT sử dụng cảm biến ánh sáng nằm ở mặt trước của thiết bị Galaxy.
- Cảm biến ánh sáng sẽ phát hiện khi môi trường trở nên tối và gửi tín hiệu để bật đèn trong nhà.
- Tham khảo đường link: https://www.samsung.com/levant/support/mobile-devices/galaxy-s9-what-is-the-device-layout/ , để xem ví dụ và các kịch bản (routine)

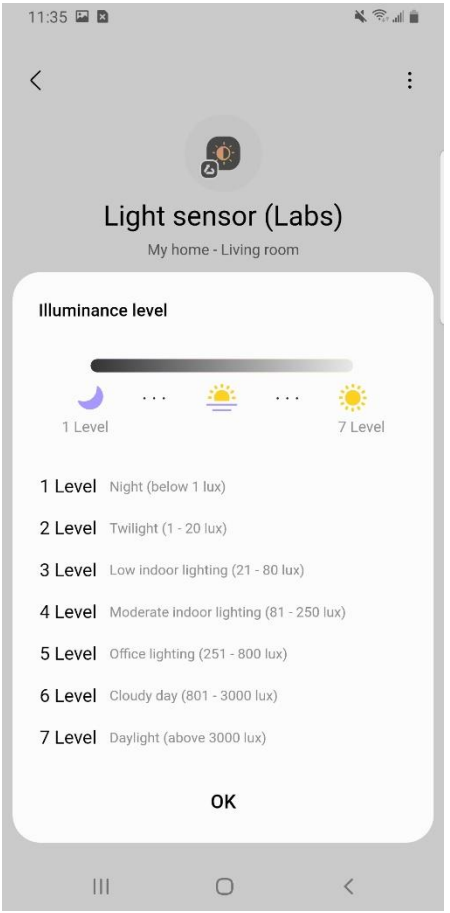




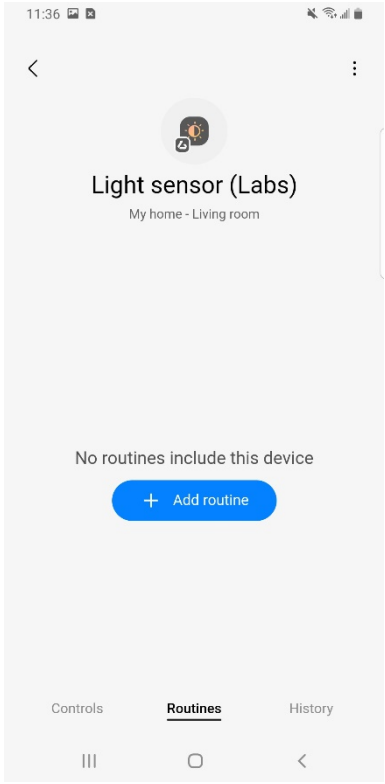
- Bước 1: Ứng dụng SmartThings → Devices → Thẻ cảm biến ánh sáng → Kiểm tra trạng thái ánh sáng (7 mức độ chiếu sáng)

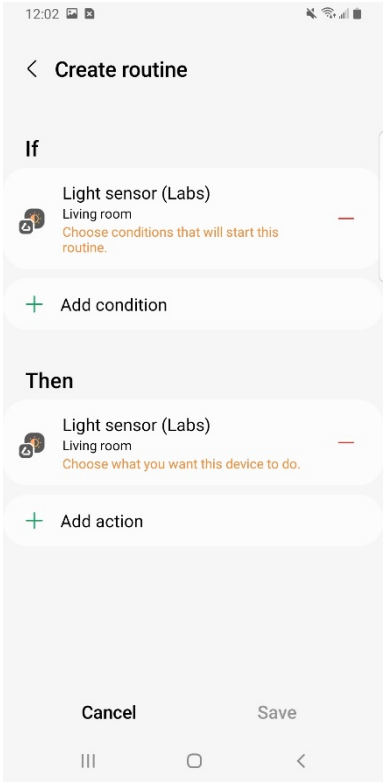


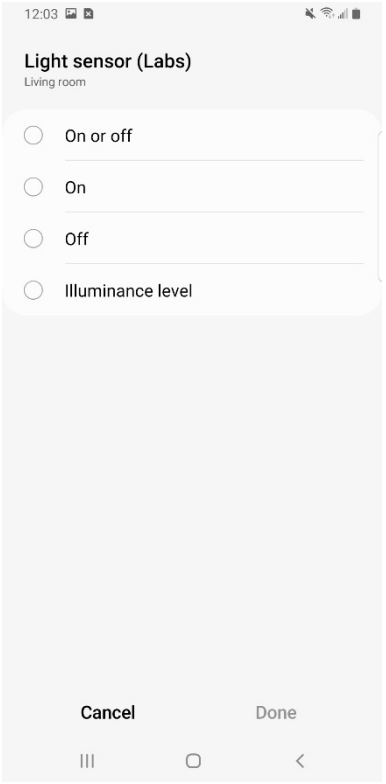


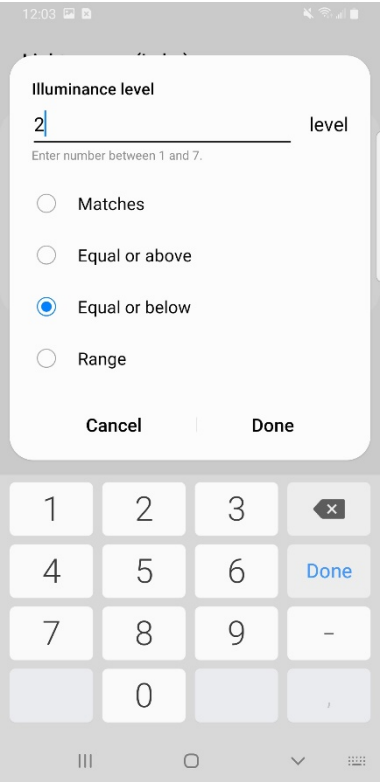


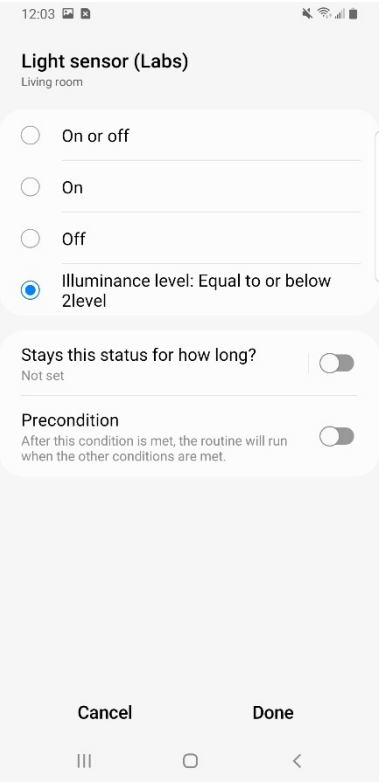
- Bước 2: Tab Routines → Thêm Routine → Thiết lập điều kiện tối dựa trên mức độ ánh sáng của cảm biến (1/3)



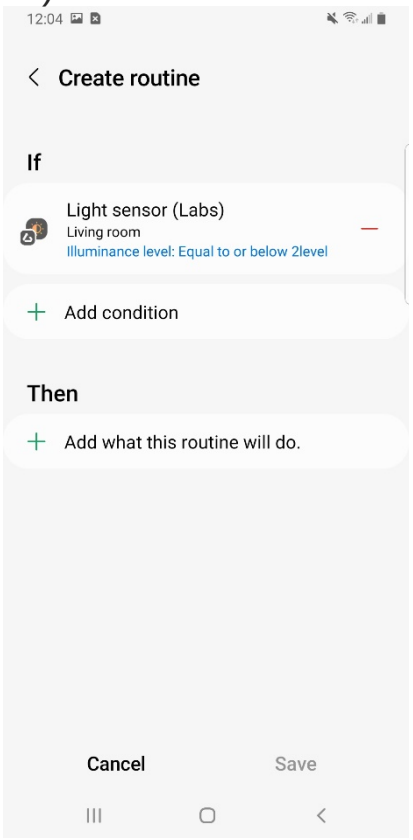


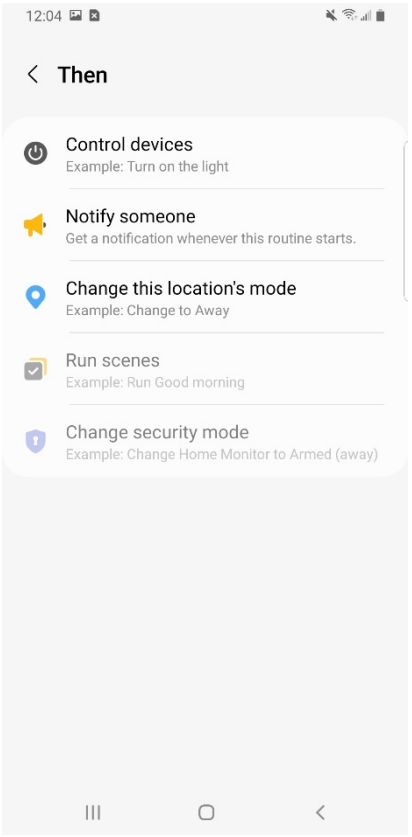


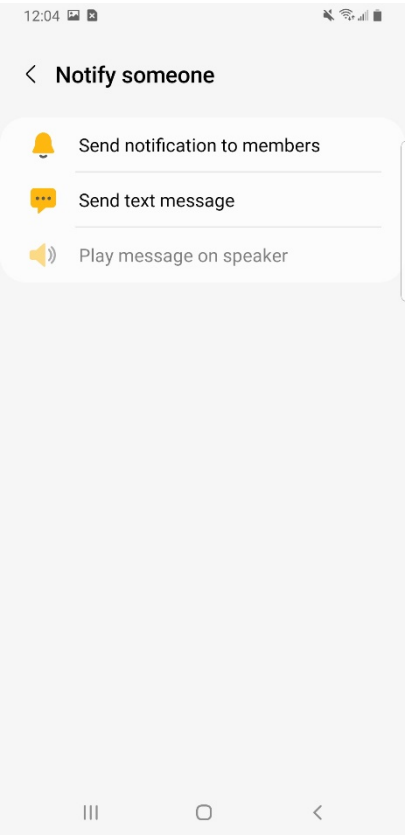


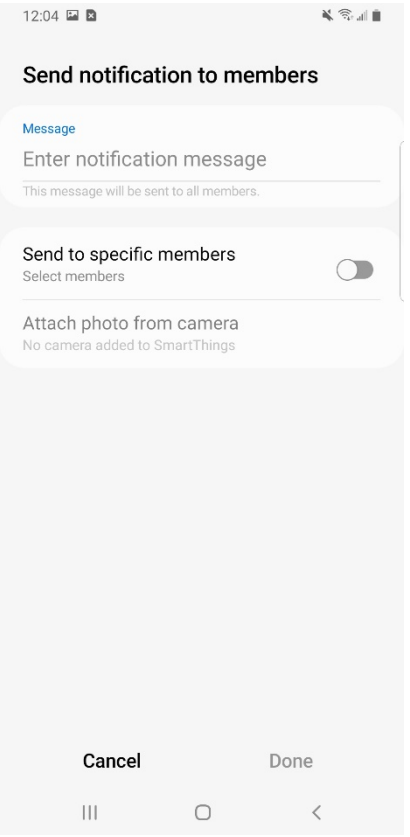


- Bước 2: Tab Routines → Thêm Routine → Thiết lập điều kiện tối dựa trên mức độ ánh sáng của cảm biến (2/3)

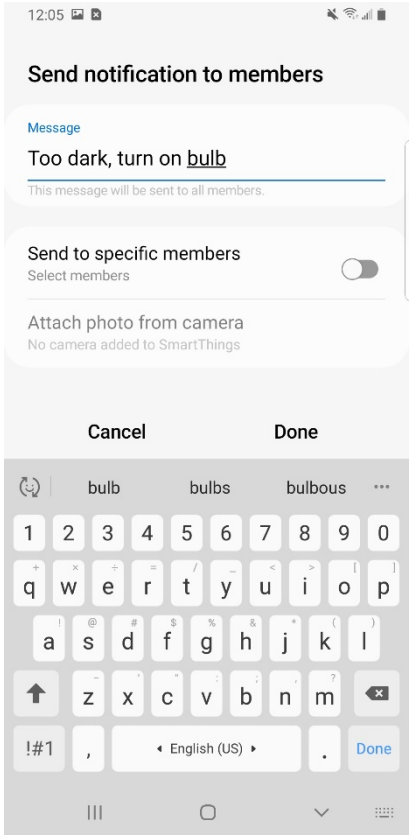


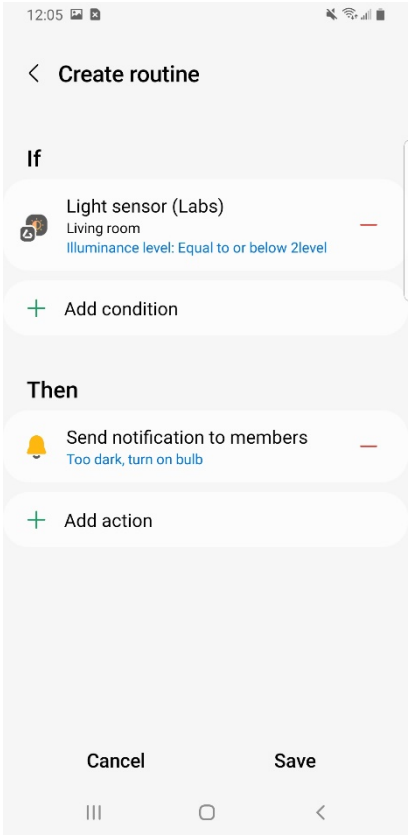


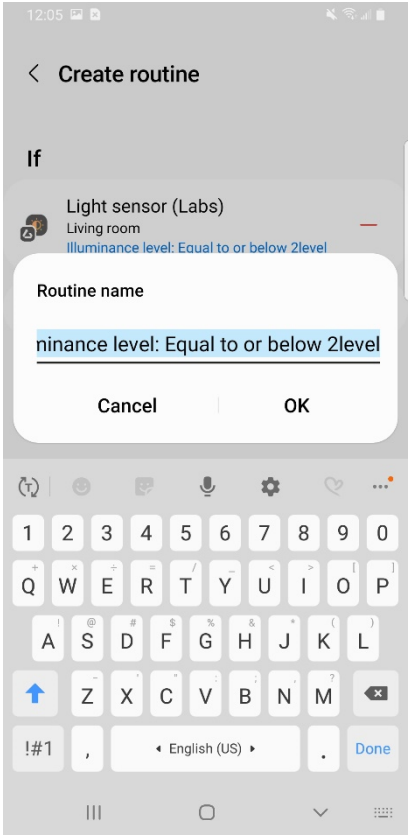


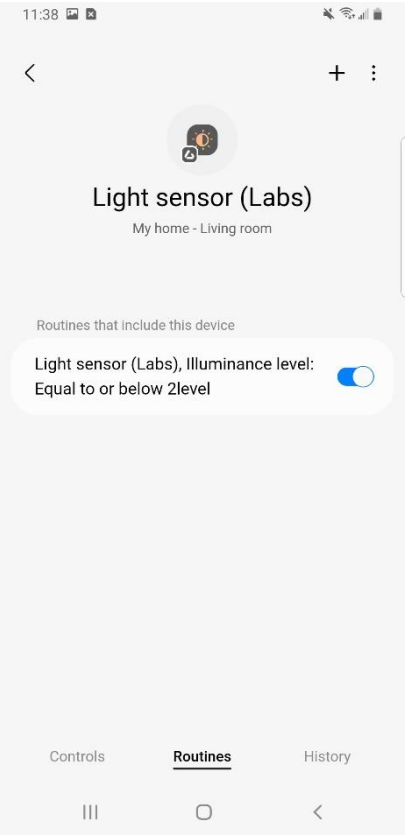


- Bước 2: Tab Routines → Thêm Routine → Thiết lập điều kiện tối dựa trên mức độ ánh sáng của cảm biến (3/3)

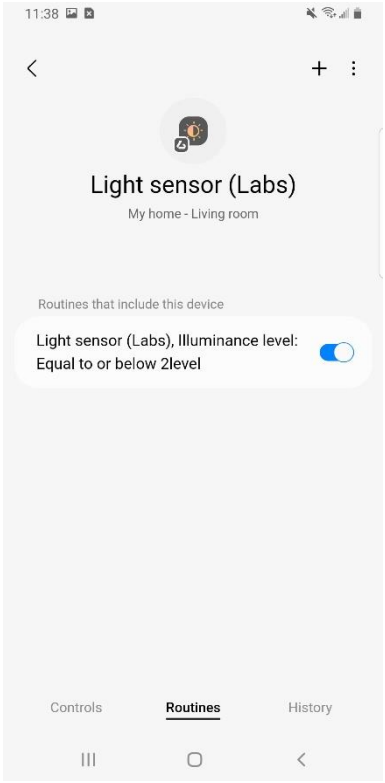


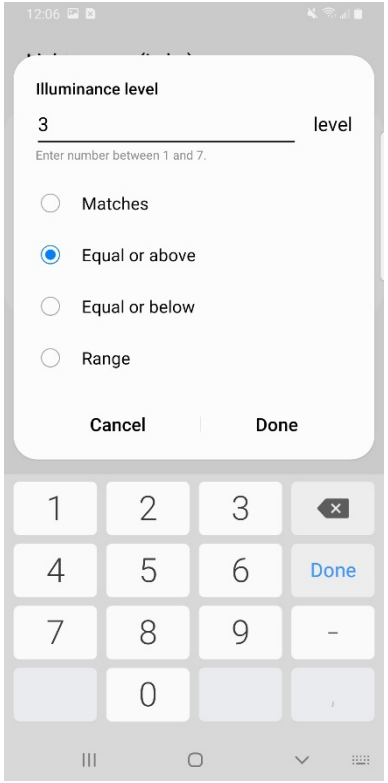


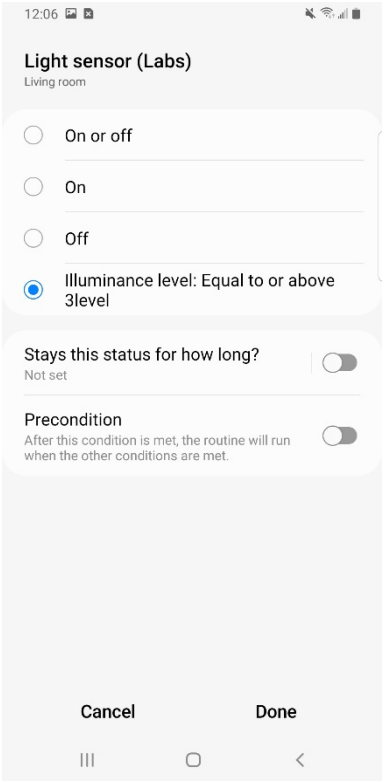


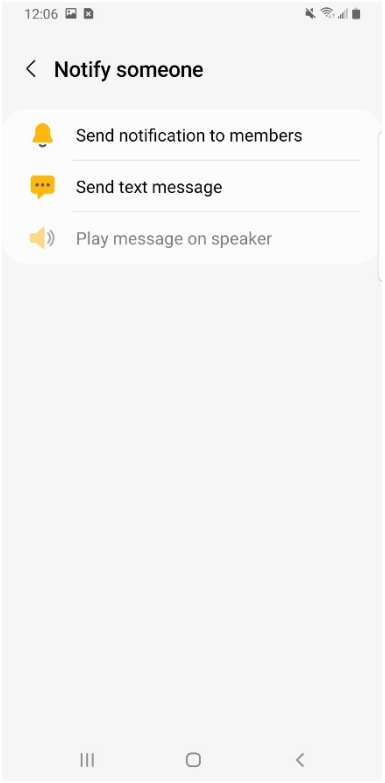


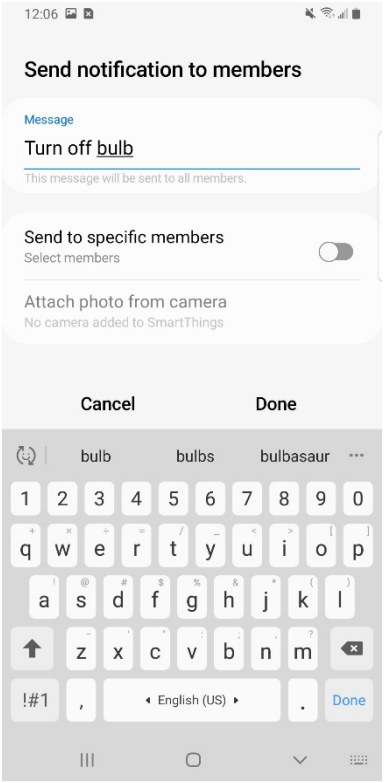
- Bước 3: Tab Routines → Thêm Routine → Thiết lập điều kiện sáng dựa trên mức độ ánh sáng của cảm biến (1/2)



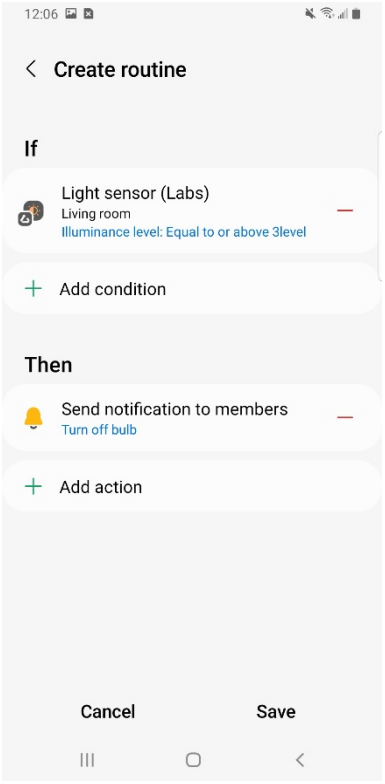


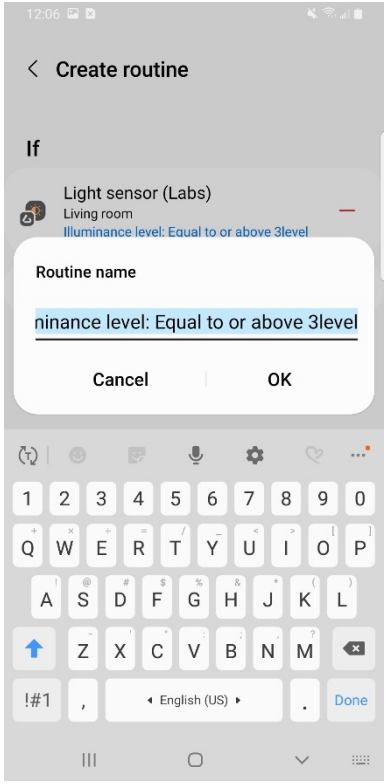


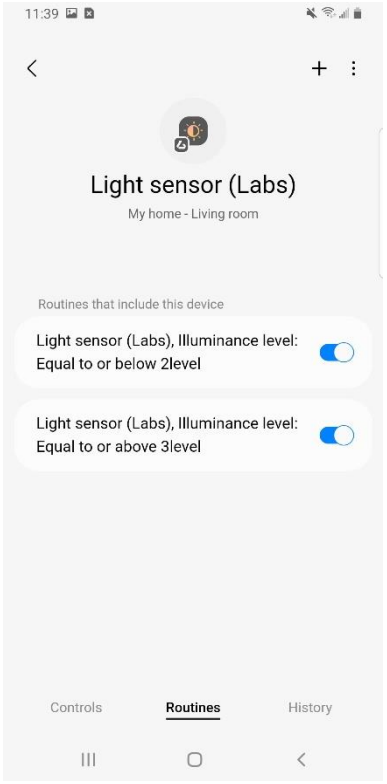


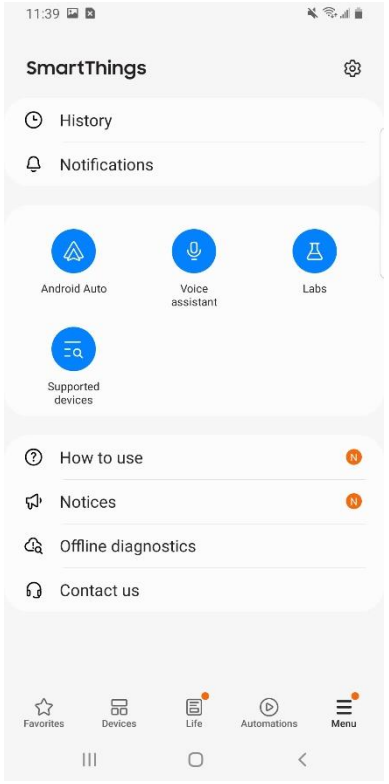


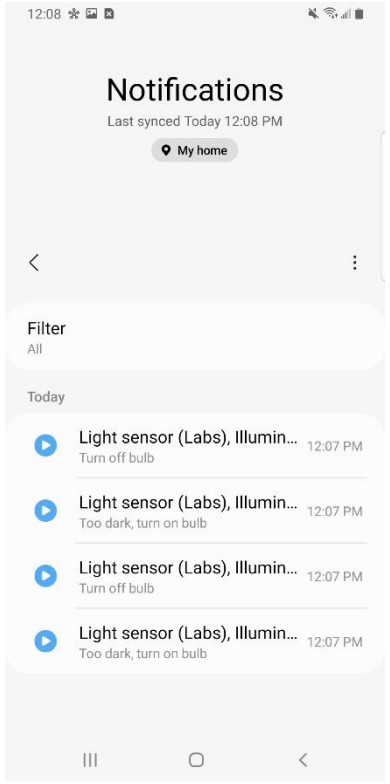
- Bước 3: Tab Routines → Thêm Routine → Thiết lập điều kiện sáng dựa trên mức độ ánh sáng của cảm biến (2/2)
- Bước 4: Tạo điều kiện tối/sáng bằng tay (che/mở cảm biến) → Kiểm tra trạng thái thông báo





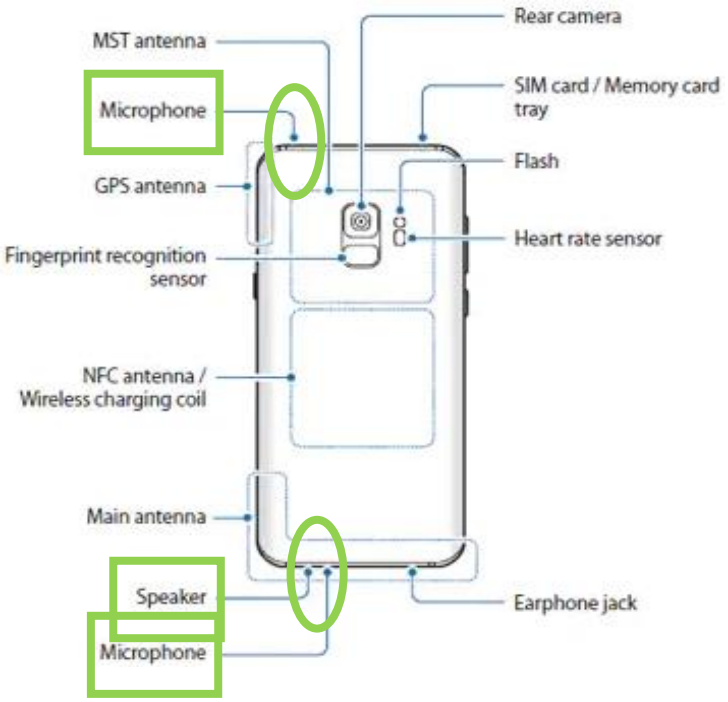


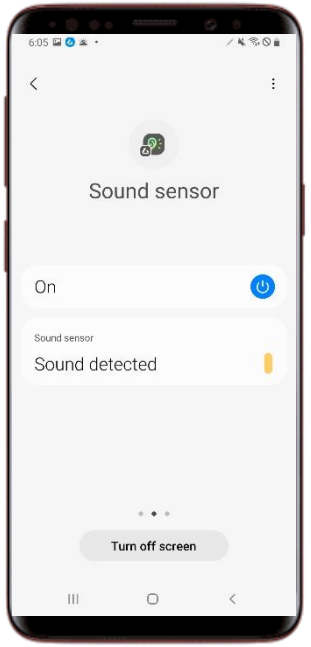




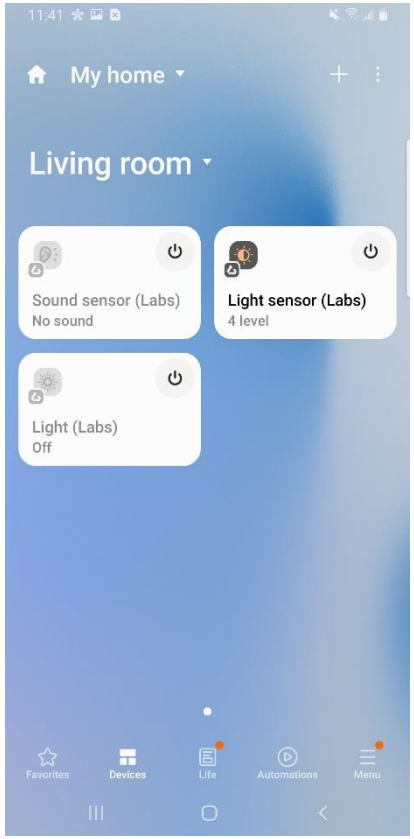
Cảm biến âm thanh

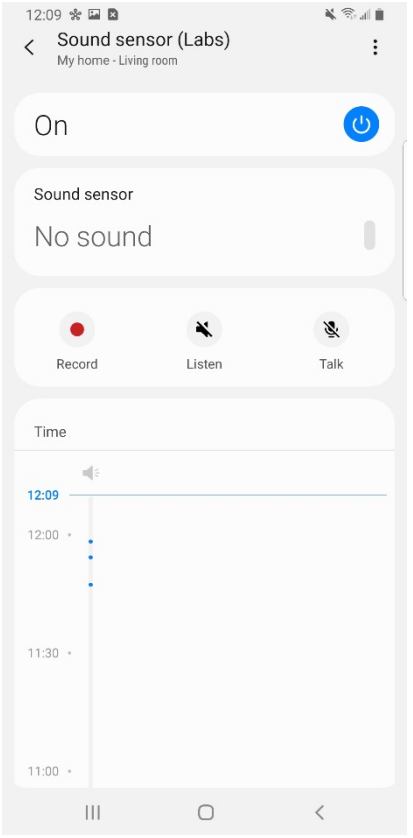
- Thiết bị IoT sử dụng micro và loa nằm ở mặt sau của thiết bị Galaxy.
- Khi cảm biến âm thanh phát hiện âm thanh, hệ thống có thể gửi cảnh báo hoặc thông báo đến thiết bị của bạn.
- Tham khảo đường link https://www.samsung.com/levant/support/mobile-devices/galaxy-s9-what-is-the-device-layout/ , để xem ví dụ và các kịch bản.

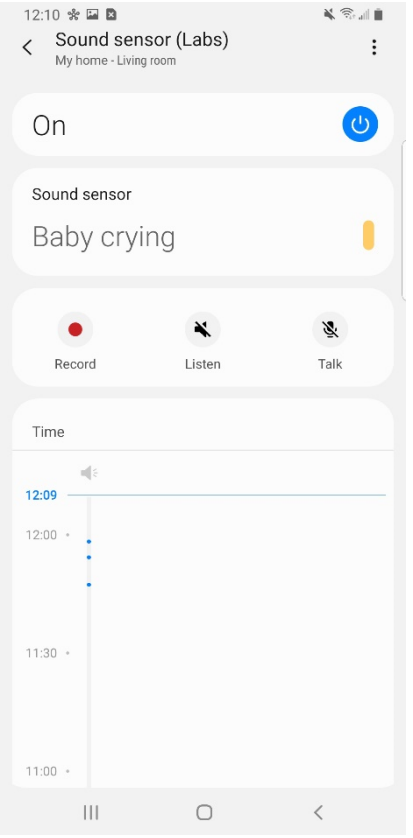




- Bước 1: Ứng dụng SmartThings → Devices → Thẻ cảm biến âm thanh → Kiểm tra trạng thái nhận diện âm thanh (YAMNet)

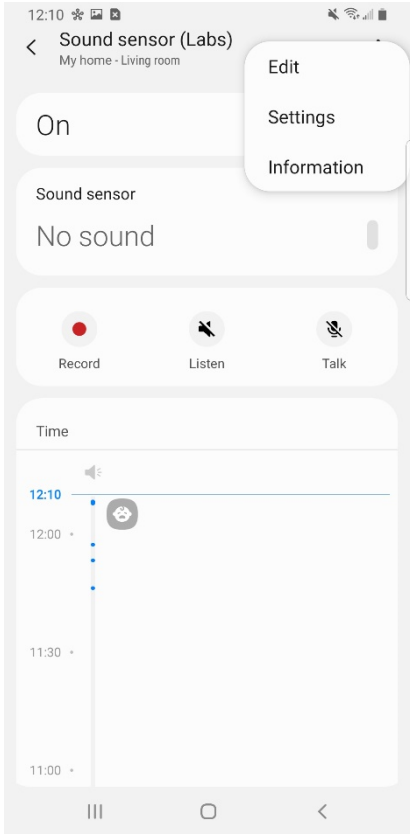


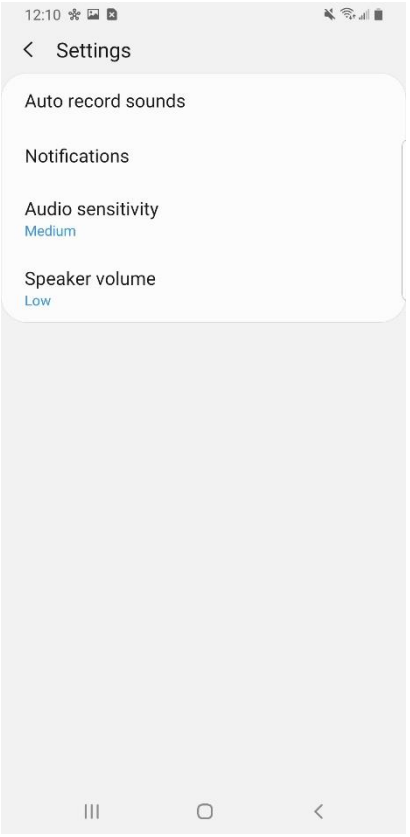


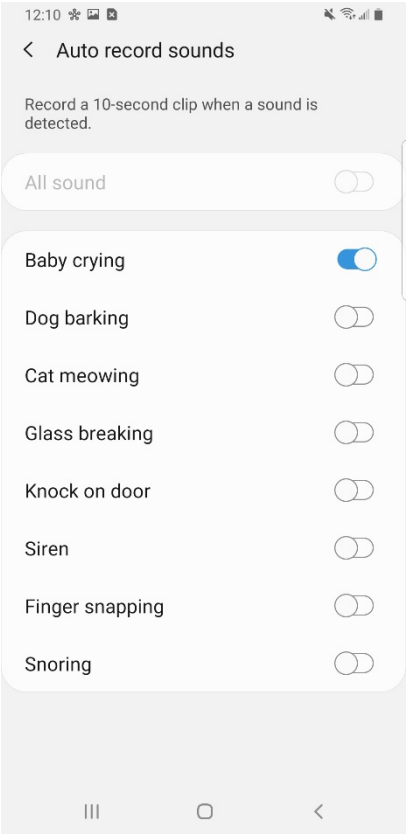


2.3. Điều khiển thiết bị IoT: Cảm biến âm thanh

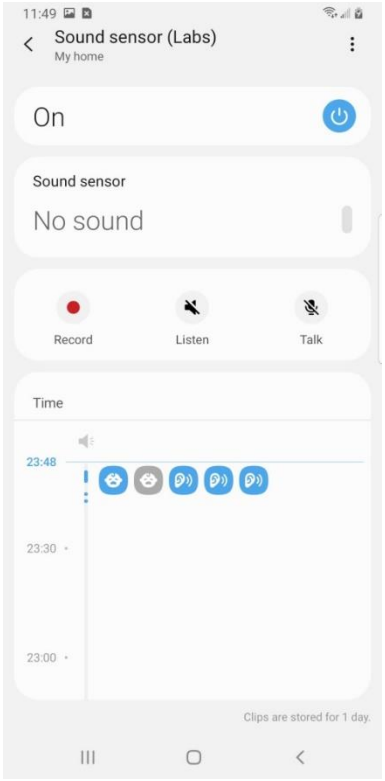
- Bước 2: Thẻ cảm biến âm thanh → More → Settings → Tự động ghi âm → Bật chế độ nhận diện tiếng trẻ khóc

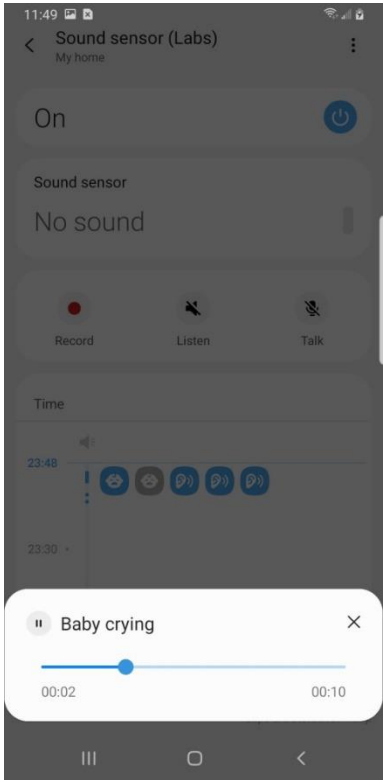






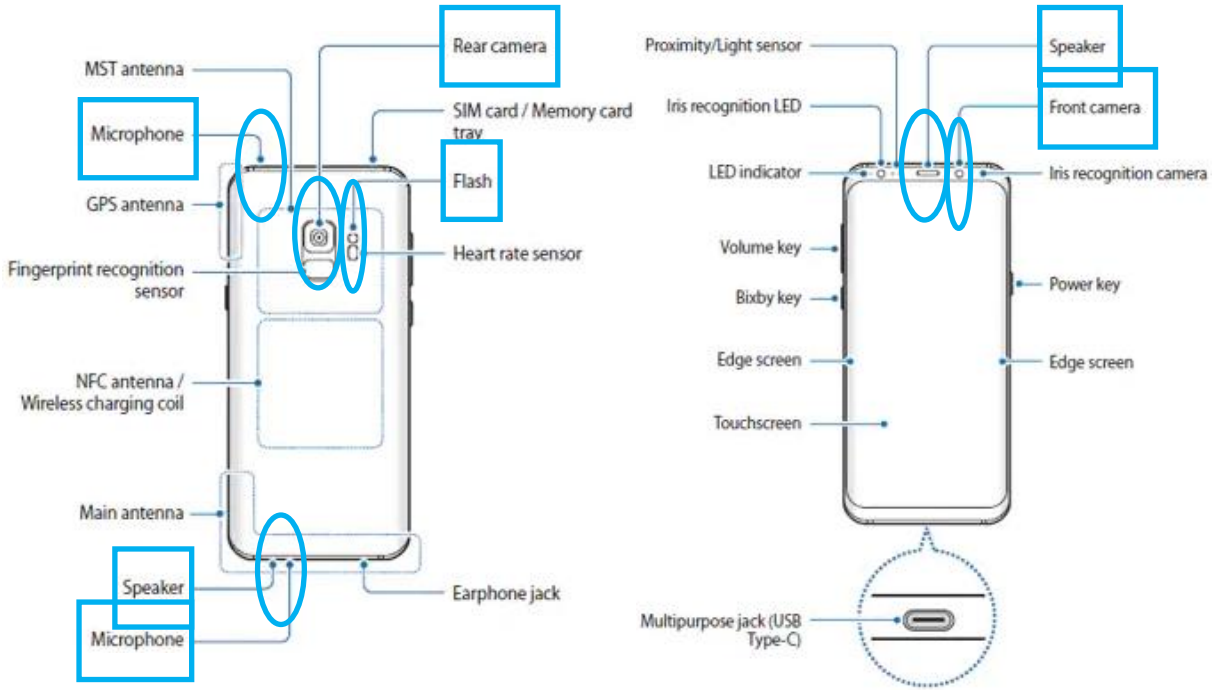
Bước 3: Thẻ cảm biến âm thanh → Timeline → Chọn biểu tượng em bé → Kiểm tra đoạn âm thanh đã được phát hiện



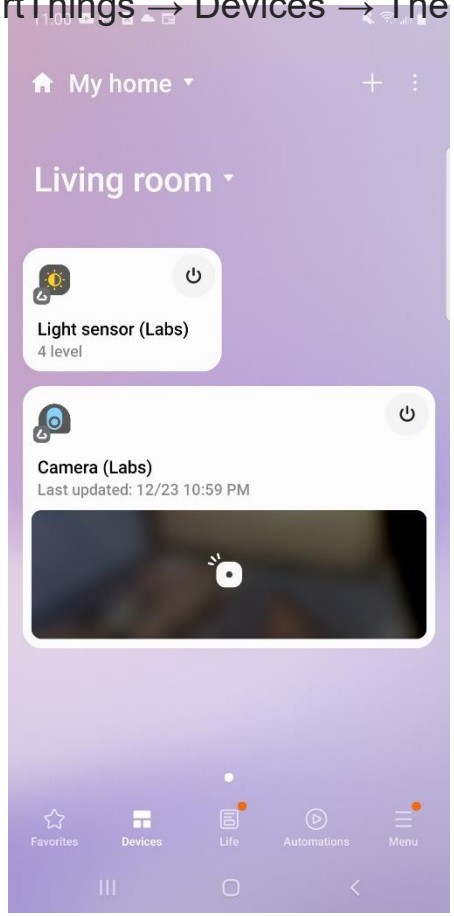


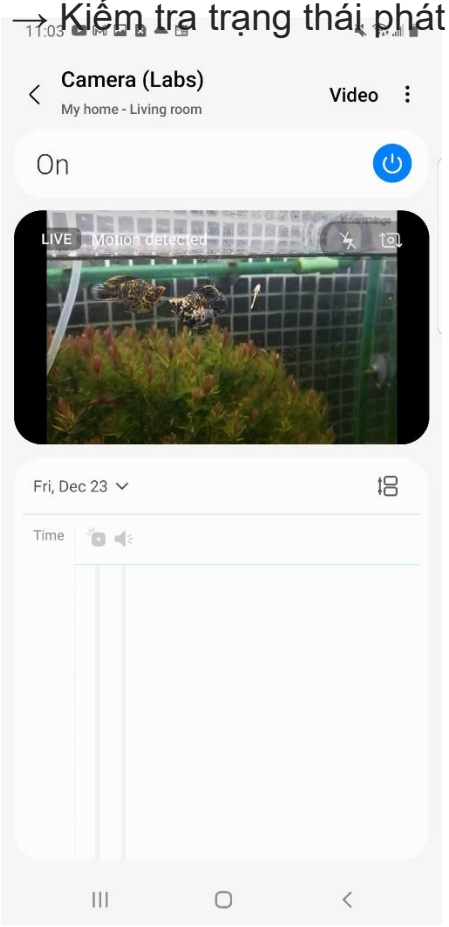
Camera

- Thiết bị IoT sử dụng micro, loa, đèn flash và camera trước/sau được bố trí ở cả hai mặt của thiết bị Galaxy.
- Khi camera phát hiện chuyển động hoặc âm thanh, hệ thống có thể chụp ảnh hoặc ghi hình.
- Tham khảo đường link https://www.samsung.com/levant/support/mobile-devices/galaxy-s9-what-is-the-device-layout/, để xem ví dụ và các kịch bản.



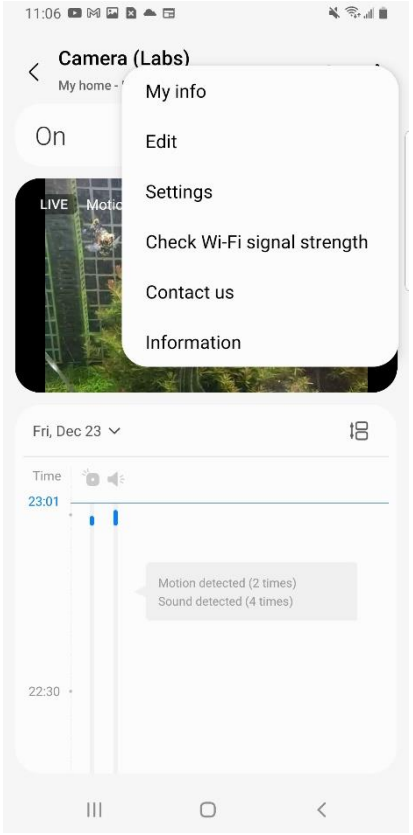
Bước 1: Ứng dụng SmartThings → Devices → Thẻ camera → Kiểm tra trạng thái phát trực tiếp (WebRTC)

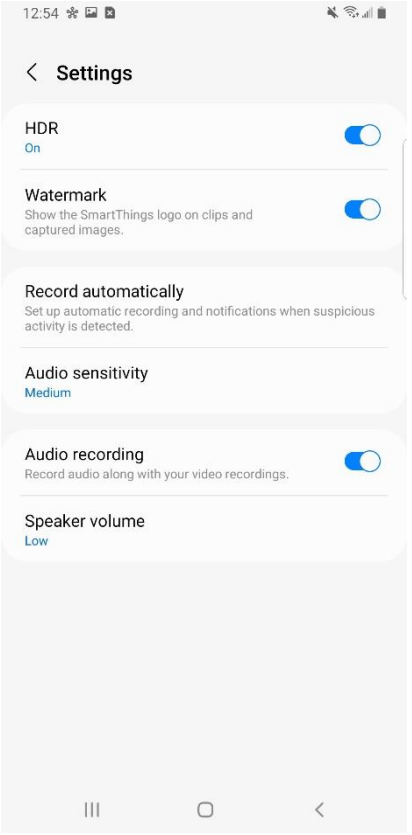


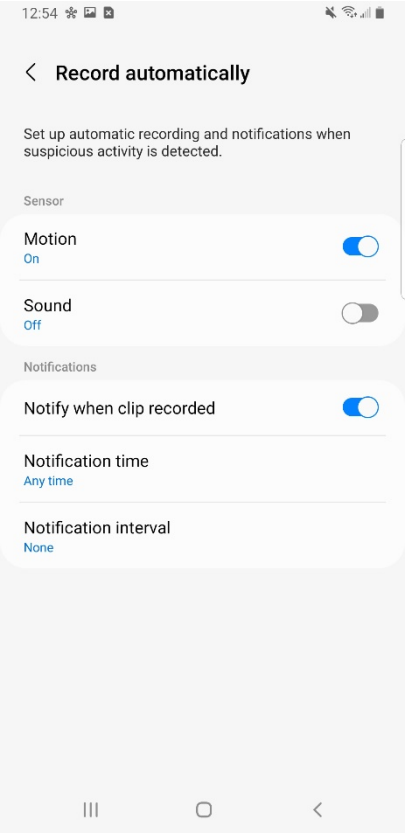


Tỷ lệ khung hình video chỉ được điều chỉnh thông qua nút xoay màn hình trên thiết bị Galaxy cũ.
Vì vậy, cần điều chỉnh chế độ dọc/ngang trước khi bật phát trực tiếp.

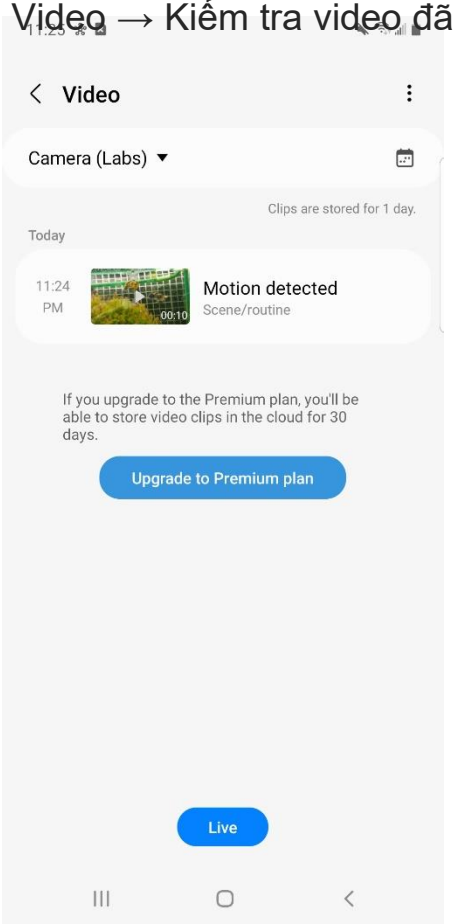
Bước 2: Thẻ camera → More → Settings → Ghi tự động → Bật Motion/Notify

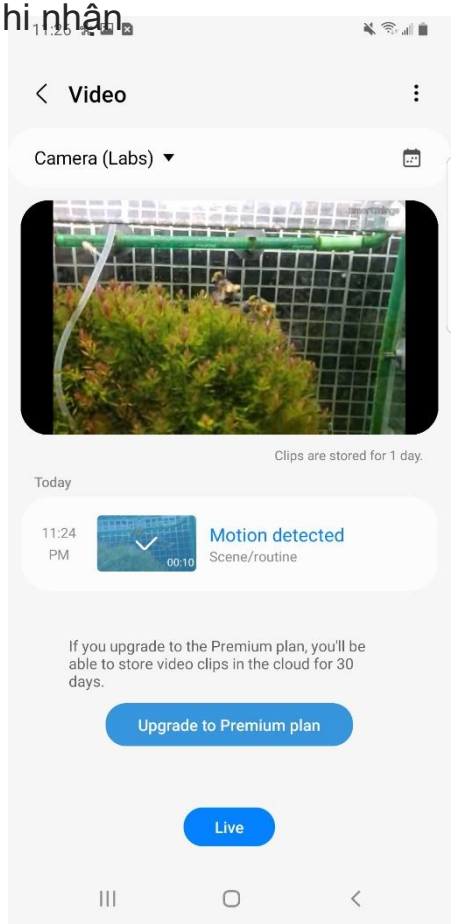






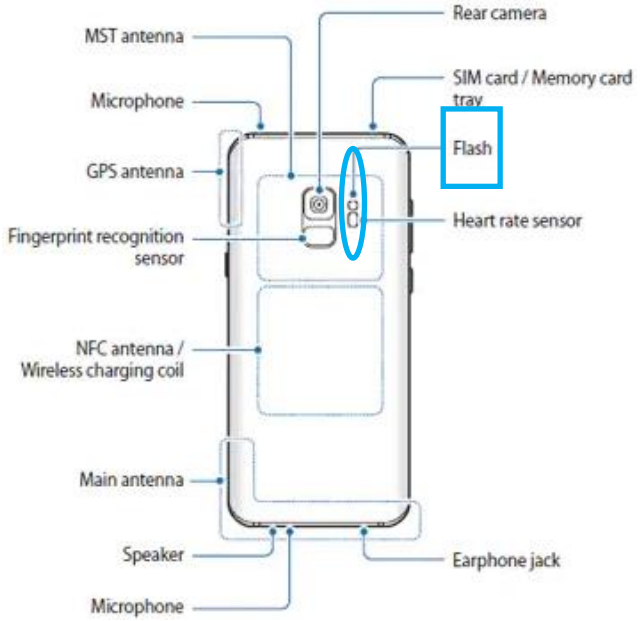
Bước 3: Thẻ camera → Video → Kiểm tra video đã được ghi nhận.





Đèn pin

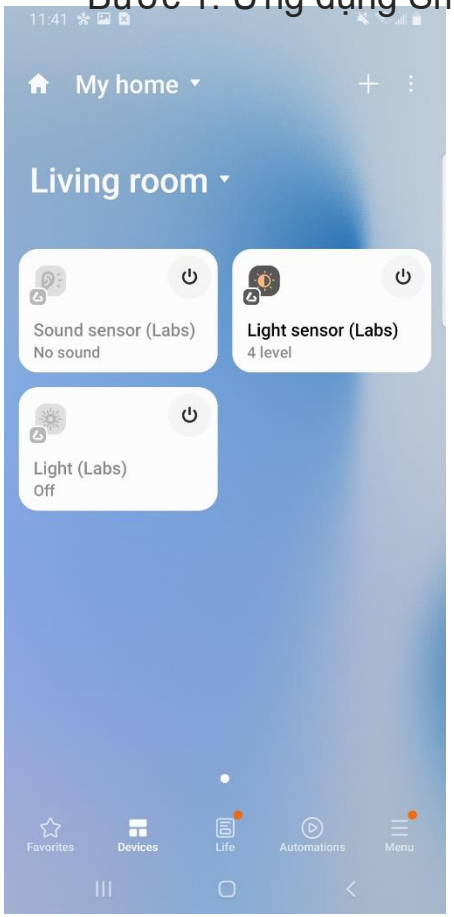
- Thiết bị IoT sử dụng đèn flash nằm ở mặt sau của thiết bị Galaxy.
- Có thể điều khiển bật/tắt đèn pin thông qua thiết bị của bạn.
- Tham khảo đường link https://www.samsung.com/levant/support/mobile-devices/galaxy-s9-what-is-the-device-layout/, để xem ví dụ và các kịch bản

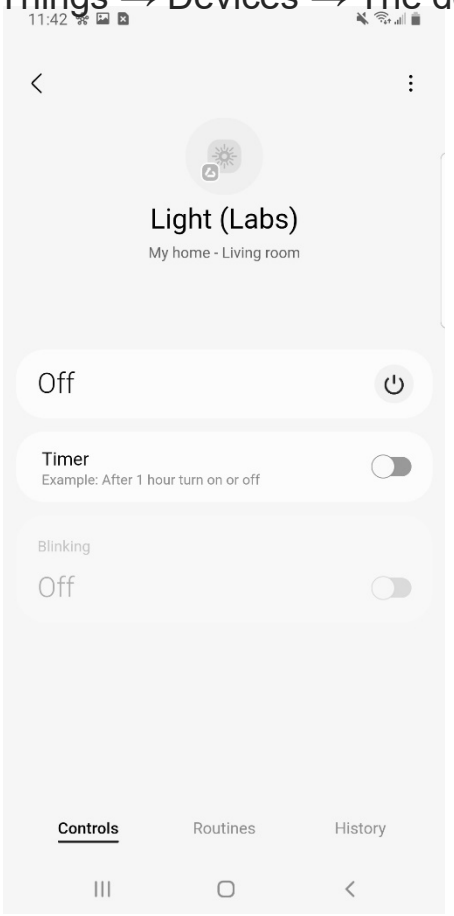


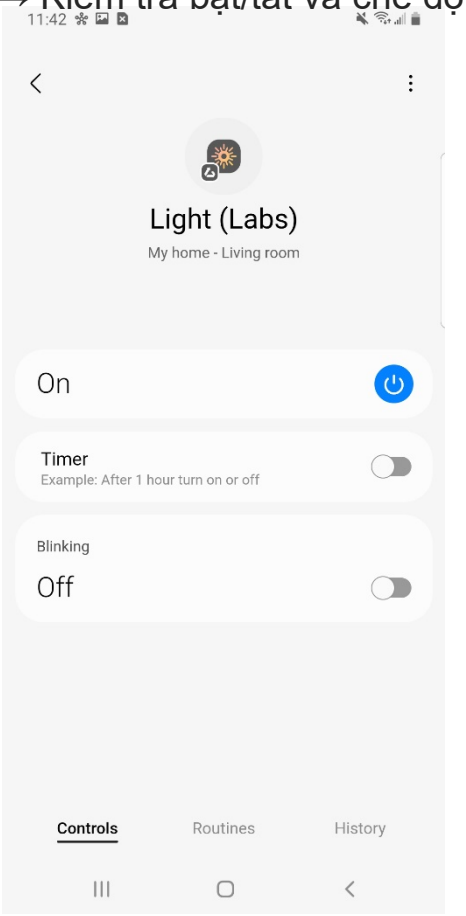


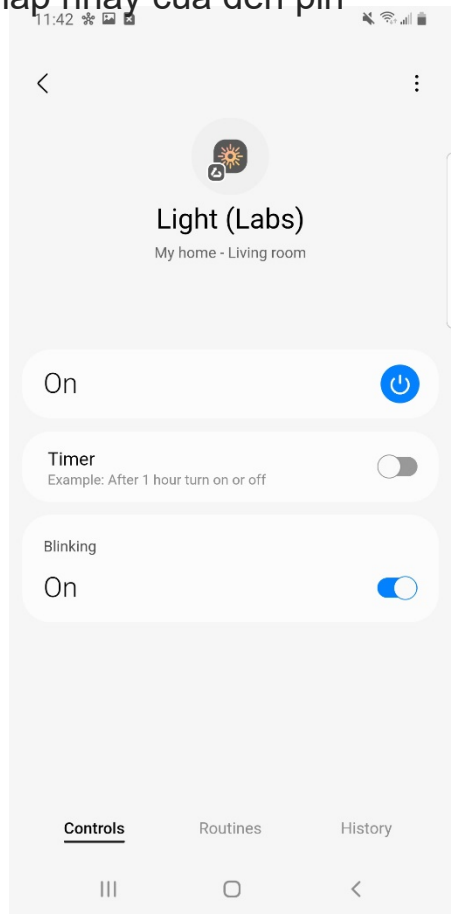
2.3. Điều khiển thiết bị IoT: Đèn pin

Bước 1: Ứng dụng SmartThings → Devices → Thẻ đèn → Kiểm tra bật/tắt và chế độ nhấp nháy của đèn pin



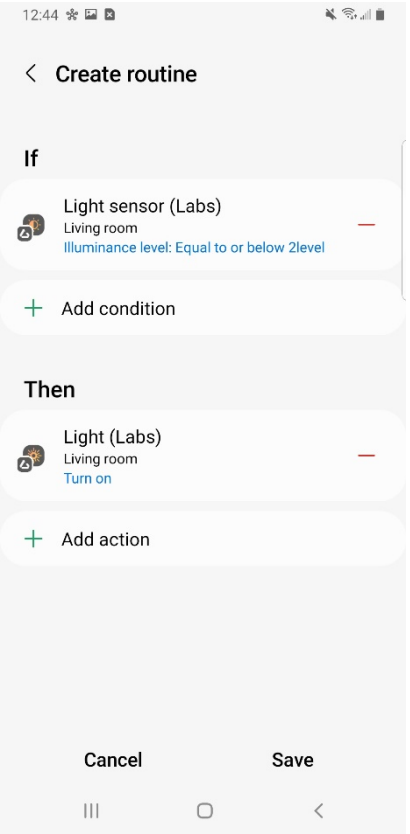


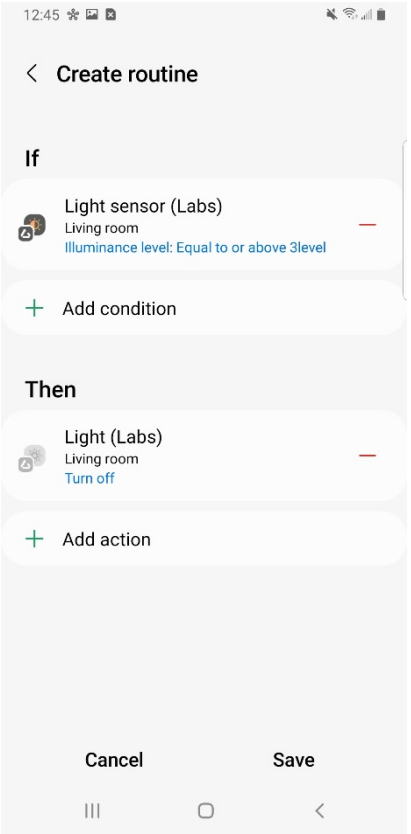


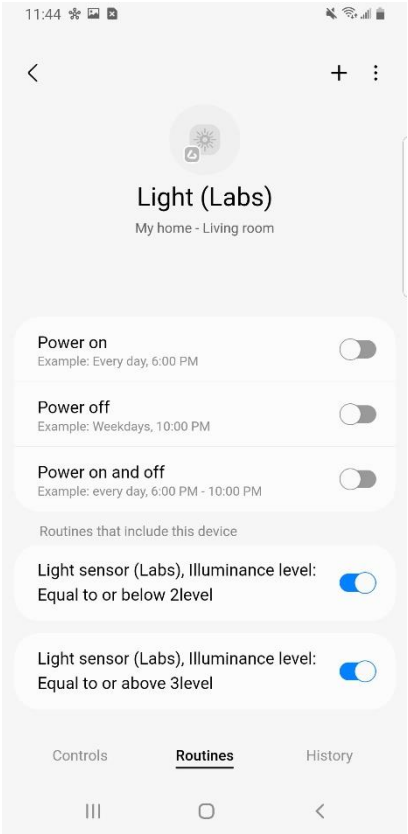


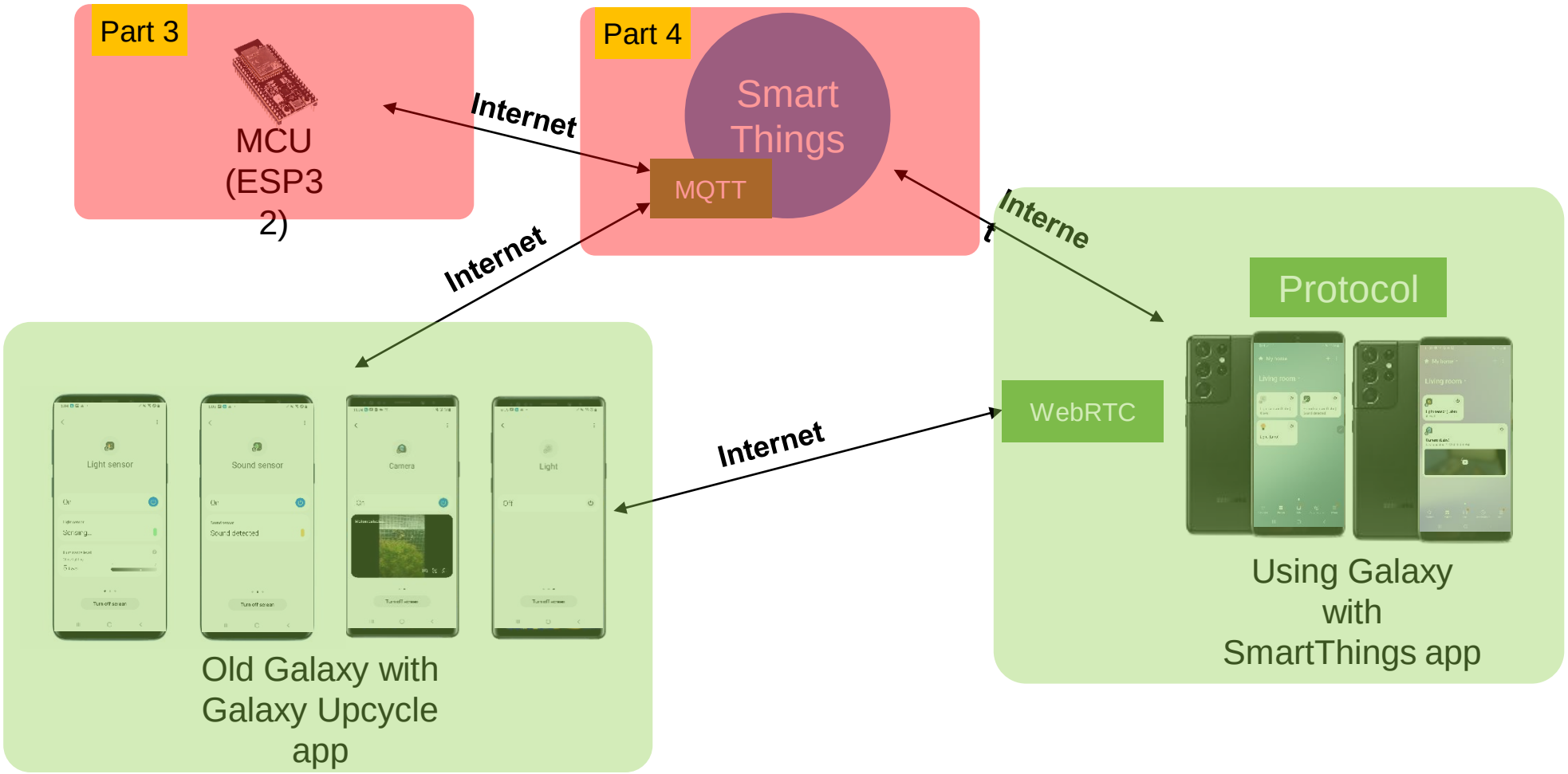
2.3. Điều khiển thiết bị IoT: Đèn pin

- Bước 2: Thẻ đèn → Tab Routines → Thêm routine (nút +)
- Bước 3: Tạo điều kiện sáng/tối bằng tay (che/mở cảm biến) → Kiểm tra trạng thái bật/tắt đèn pin








Part 3
MCU (ESP32)
Internet
Part 4
Smart Things
MQTT
Internet
Internet
Protocol
WebRTC
Internet
Using Galaxy with SmartThings app
Old Galaxy with Galaxy Upcycle app

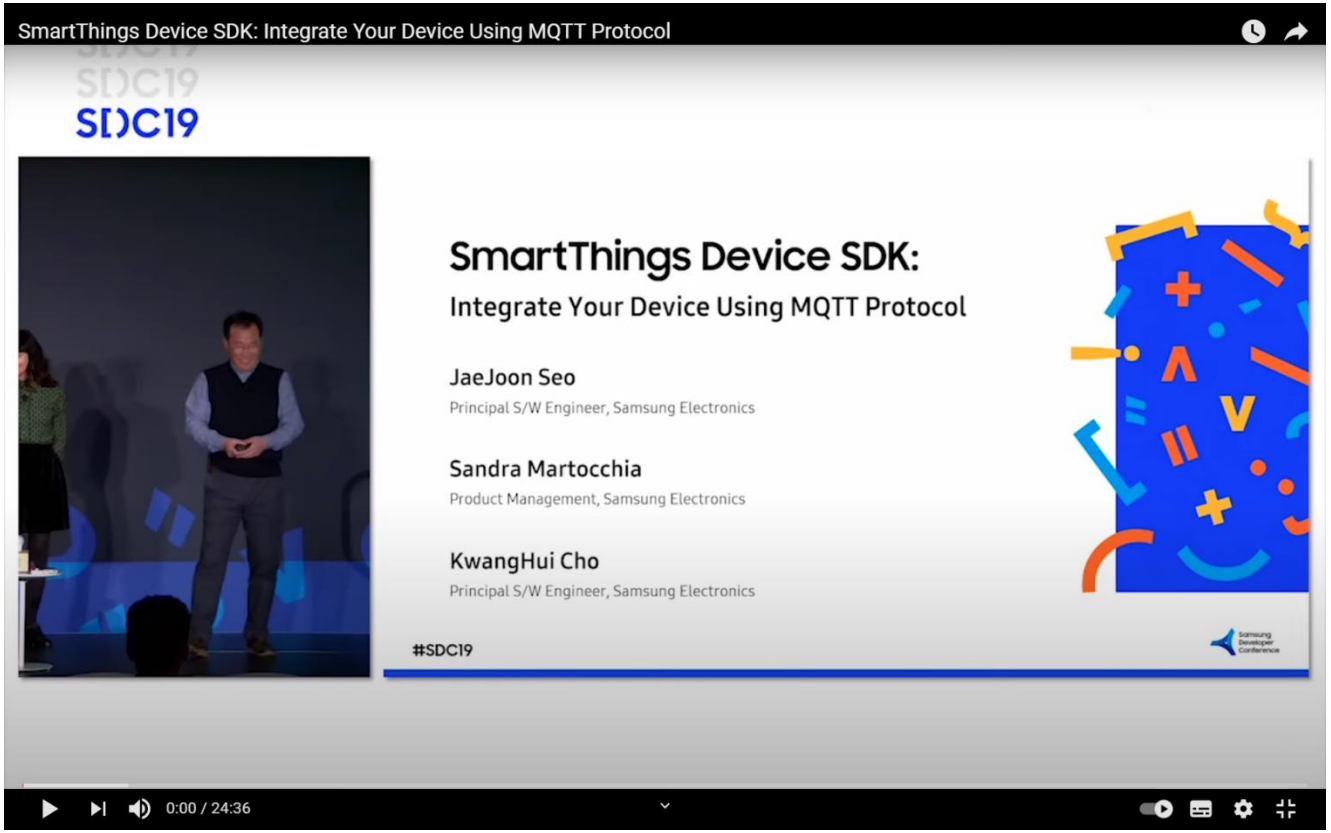
BÀI 3.

Sử dụng SmartThings Device SDK để điều khiển MCU (Nâng cao)

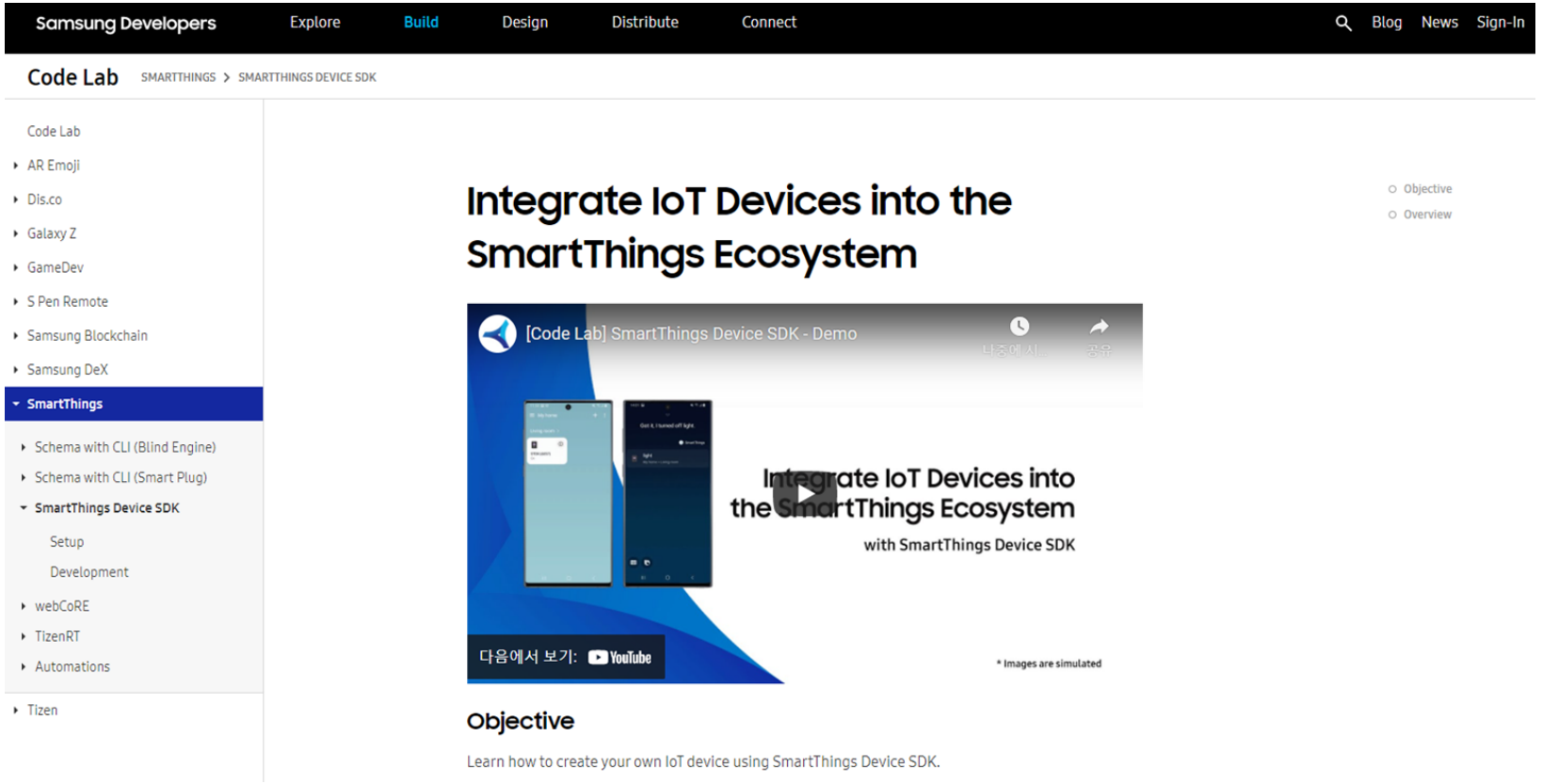
- 3.1. Giới thiệu về STDK (SmartThings Device SDK)
- 3.2. Thiết lập môi trường phát triển
- 3.3. Xây dựng và kiểm thử

- STDK là bộ công cụ phục vụ phát triển thiết bị IoT trên nền tảng SmartThings.
- Trong khóa học này, bạn sẽ trải nghiệm phát triển IoT thực tế và nắm được kỹ năng điều khiển MCU để trở thành lập trình viên IoT.

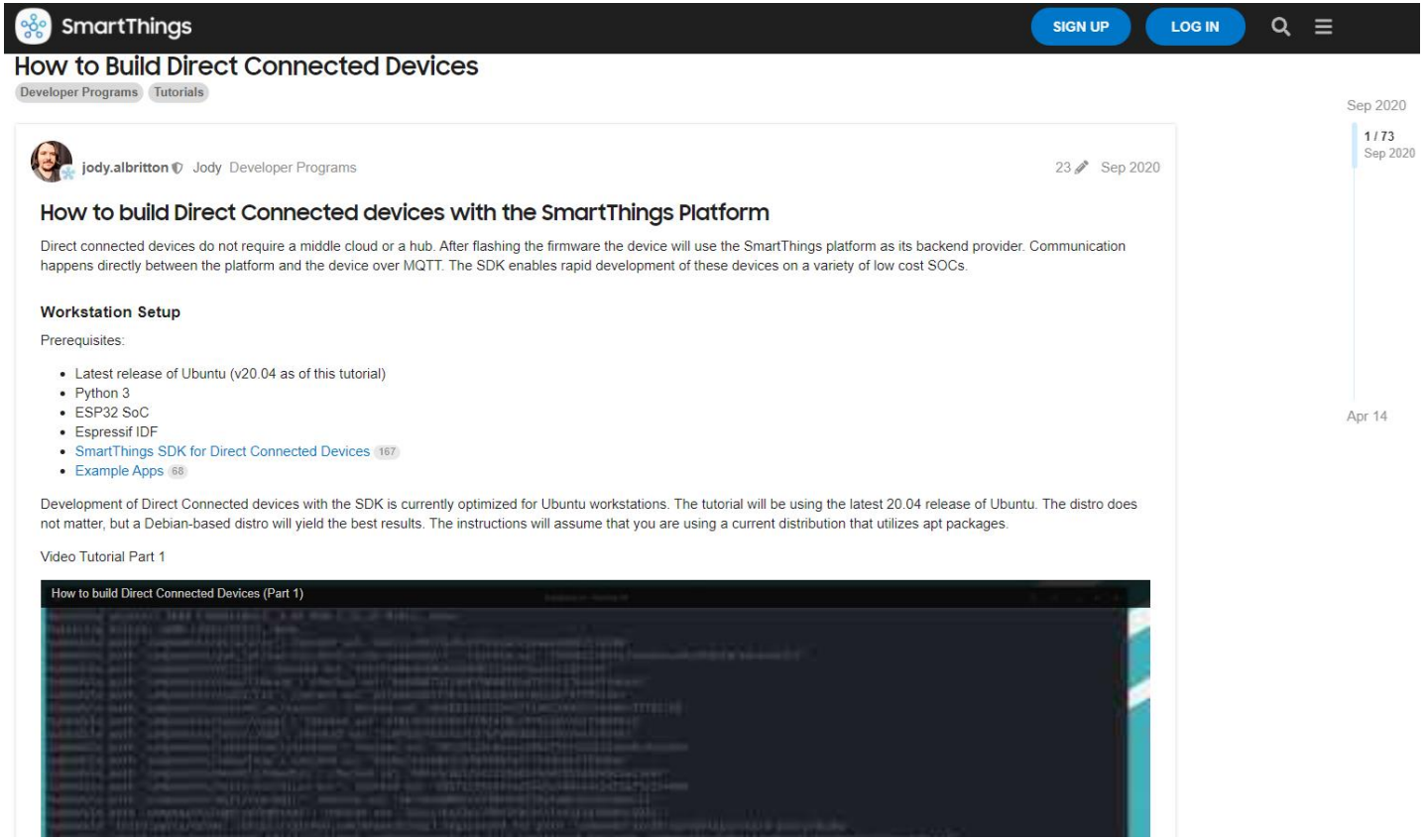
- Video giới thiệu đầy đủ tại SDC (Samsung Developer Conference) 2019
  - https://www.youtube.com/watch?v=Rjz0wept1fE



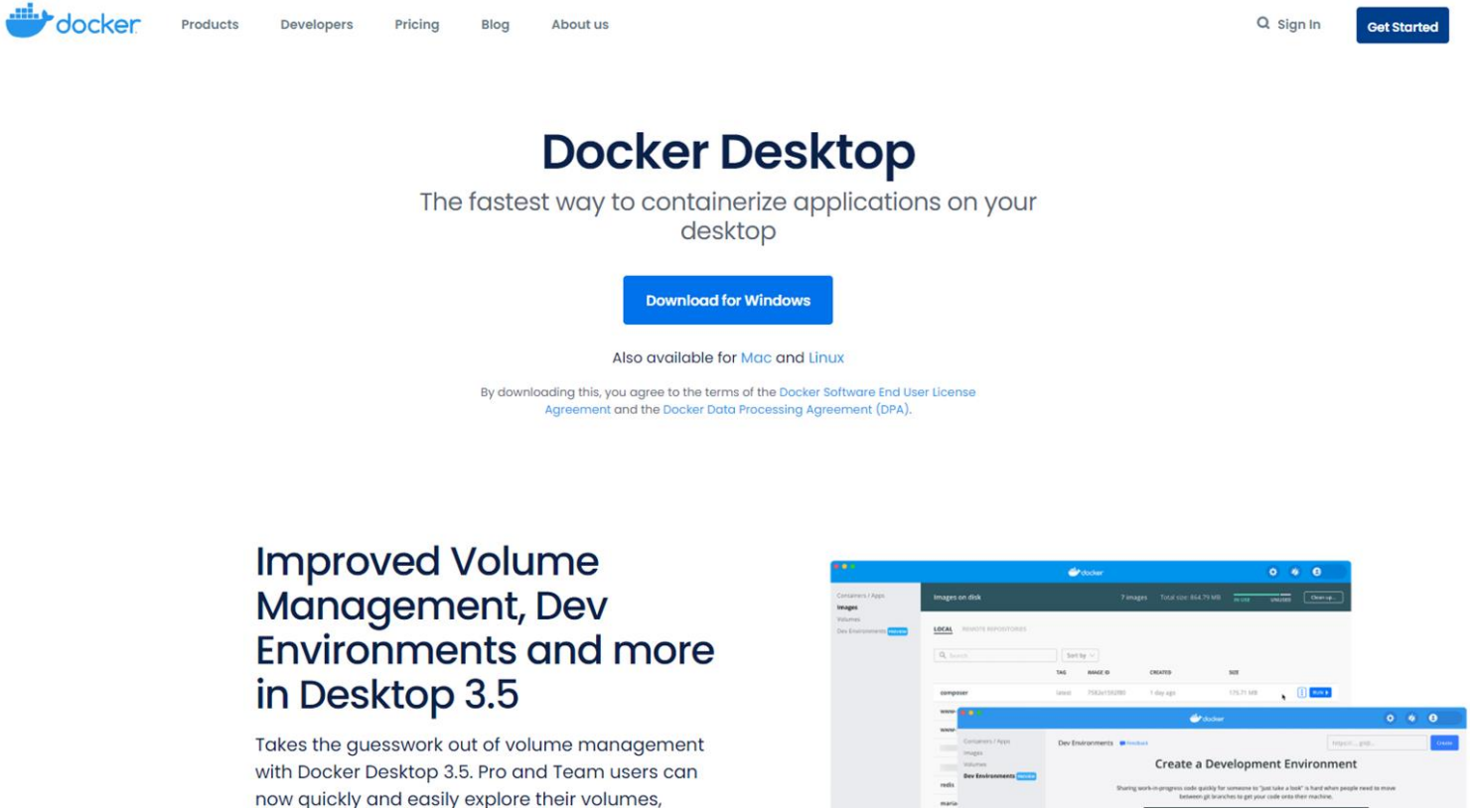
- Tài liệu tham khảo trong Code Lab
  - https://developer.samsung.com/codelab/smartthings/smartthings-device-sdk/overview.html



- Tài liệu tham khảo trong cộng đồng SmartThings
  - https://community.smartthings.com/t/how-to-build-direct-connected-devices/204055



- Cài đặt Docker Desktop để thực hiện build đơn giản
  - https://www.docker.com/products/docker-desktop



- Tải project từ GitHub để sử dụng file Docker build

```
$ git clone https://github.com/nanishin/stdk-docker-build.git
Cloning into 'stdk-docker-build'...
remote: Enumerating objects: 9, done.
remote: Counting objects: 100% (9/9), done.
remote: Compressing objects: 100% (9/9), done.
remote: Total 9 (delta 1), reused 5 (delta 0), pack-reused 0
Unpacking objects: 100% (9/9), done.
Checking connectivity... done.
```

- Chạy file lệnh để chuẩn bị môi trường phát triển

```
$ cd stdk-docker-build/
$ ./build_stdk_esp32.sh
[+] Building 3.2s (10/10) FINISHED
 => [internal] load build definition from stdk-common-env.dockerfile                                    0.0s
 => => transferring dockerfile: 933B                                                                    0.0s
 => [internal] load .dockerignore                                                                       0.0s
 => => transferring context: 2B                                                                         0.0s
 => [internal] load metadata for docker.io/library/ubuntu:20.04                                         3.0s
 => [auth] library/ubuntu:pull token for registry-1.docker.io                                           0.0s
 => [1/5] FROM docker.io/library/ubuntu:20.04@sha256:626ffe58f6e7566e00254b638eb7e0f3b11d4da9675088f4781a50ae288f3322
0.0s
 => [2/5] RUN dpkg --add-architecture i386                                                              0.0s
 => [3/5] RUN ln -snf /usr/share/zoneinfo/Asia/Seoul /etc/localtime && echo Asia/Seoul > /etc/timezone
0.0s
 => [4/5] RUN apt-get update                                                                            0.0s
 => [5/5] RUN apt-get install -y --no-install-recommends gcc git wget make libncurses-dev flex bison gperf python3
python3-seri  0.0s
 => exporting to image                                                                                  0.0s
 => => exporting layers                                                                                 0.0s
 => => writing image sha256:a29bf1360ed35d8f61ab6dc513a466cecc678348de01390986244f04d8b2dfee
0.0s
 => => naming to docker.io/library/stdk_common_env                                                      0.0s
```

- Chạy tệp lệnh để thiết lập môi trường phát triển

```
[+] Building 266.9s (15/15) FINISHED
 => [internal] load build definition from stdk-esp32-dev.dockerfile                                                
```

- Kiểm tra các môi trường đã tạo và chạy môi trường làm việc để thực hiện biên dịch

```
$ docker images
REPOSITORY        TAG              IMAGE ID          CREATED           SIZE
stdk_esp32_dev    latest           7df3eb2fe815      3 minutes ago     1.83GB
stdk_common_env   latest           16097de5e34e      8 minutes ago     463MB
$ docker run -it –v /YOUR_ABSOLUTE_PATH/output:/st-device-sdk-c-ref/output stdk_esp32_dev bash
root@e1ae8b10fa68:/st-device-sdk-c-ref# ls -al
total 96
drwxr-xr-x  1 root root  4096 Jul  5 00:27 .
drwxr-xr-x  1 root root  4096 Jul  5 00:36 ..
drwxr-xr-x  1 root root  4096 Jul  5 00:27 .git
drwxr-xr-x  2 root root  4096 Jul  5 00:27 .github
-rw-r--r--  1 root root     8 Jul  5 00:27 .gitignore
-rw-r--r--  1 root root   981 Jul  5 00:27 .gitmodules
-rwxr-xr-x  1 root root 11357 Jul  5 00:27 LICENSE
-rwxr-xr-x  1 root root  9359 Jul  5 00:27 README.md
drwxr-xr-x 11 root root  4096 Jul  5 00:27 apps
drwxr-xr-x  1 root root  4096 Jul  5 00:27 bsp
-rwxr-xr-x  1 root root  2781 Jul  5 00:27 build.py
-rwxr-xr-x  1 root root  1785 Jul  5 00:27 build.sh
drwxr-xr-x  3 root root  4096 Jul  5 00:27 doc
drwxr-xr-x  1 root root  4096 Jul  5 00:27 iot-core
drwxr-xr-x 11 root root  4096 Jul  5 00:27 patches
-rwxr-xr-x  1 root root  2362 Jul  5 00:27 setup.py
-rwxr-xr-x  1 root root  1551 Jul  5 00:27 setup.sh
drwxr-xr-x 12 root root  4096 Jul  5 00:27 tools
root@e1ae8b10fa68:/st-device-sdk-c-ref#
```

- Kiểm tra quá trình biên dịch bằng dữ liệu cấu hình thiết bị giả lập

```
root@ad7b99273e5e:/st-device-sdk-c-ref# . /root/.profile
root@ad7b99273e5e:/st-device-sdk-c-ref# ./build.py esp32 switch_example
Adding ESP-IDF tools to PATH...
Checking if Python packages are up to date...
Python requirements from /st-device-sdk-c-ref/bsp/esp32/requirements.txt are satisfied.
Updated PATH variable:
  /st-device-sdk-c-ref/bsp/esp32/components/esptool_py/esptool:/st-device-sdk-c-ref/bsp/esp32/components/espcoredump:/st-device-sdk-c-
ref/bsp/esp32/components/partition_table/:/root/.espressif/tools/xtensa-esp32-elf/esp-2019r2-8.2.0/xtensa-esp32-elf/bin:/root/.espressif/tools/esp32ulp-
elf/2.28.51.20170517/esp32ulp-elf-binutils/bin:/root/.espressif/tools/openocd-esp32/v0.10.0-esp32-20190313/openocd-
esp32/bin:/root/.espressif/python_env/idf4.0_py3.8_env/bin:/st-device-sdk-c-
ref/bsp/esp32/tools:/usr/local/sbin:/usr/local/bin:/usr/sbin:/usr/bin:/sbin:/bin:/root/esp/xtensa-esp32-elf/bin
Done! You can now compile ESP-IDF projects.
Go to the project directory and run:

  idf.py build

Checking Python dependencies...
Python requirements from /st-device-sdk-c-ref/bsp/esp32/requirements.txt are satisfied.
Executing action: all (aliases: build)
Running cmake in directory /st-device-sdk-c-ref/apps/esp32/switch_example/build
Executing "cmake -G 'Unix Makefiles' -DPYTHON_DEPS_CHECKED=1 -DESP_PLATFORM=1 —warn-uninitialized -DWARN_UNINITIALIZED=1 -DCCACHE_ENABLE=0 /st-
device-sdk-c-ref/apps/esp32/switch_example"...

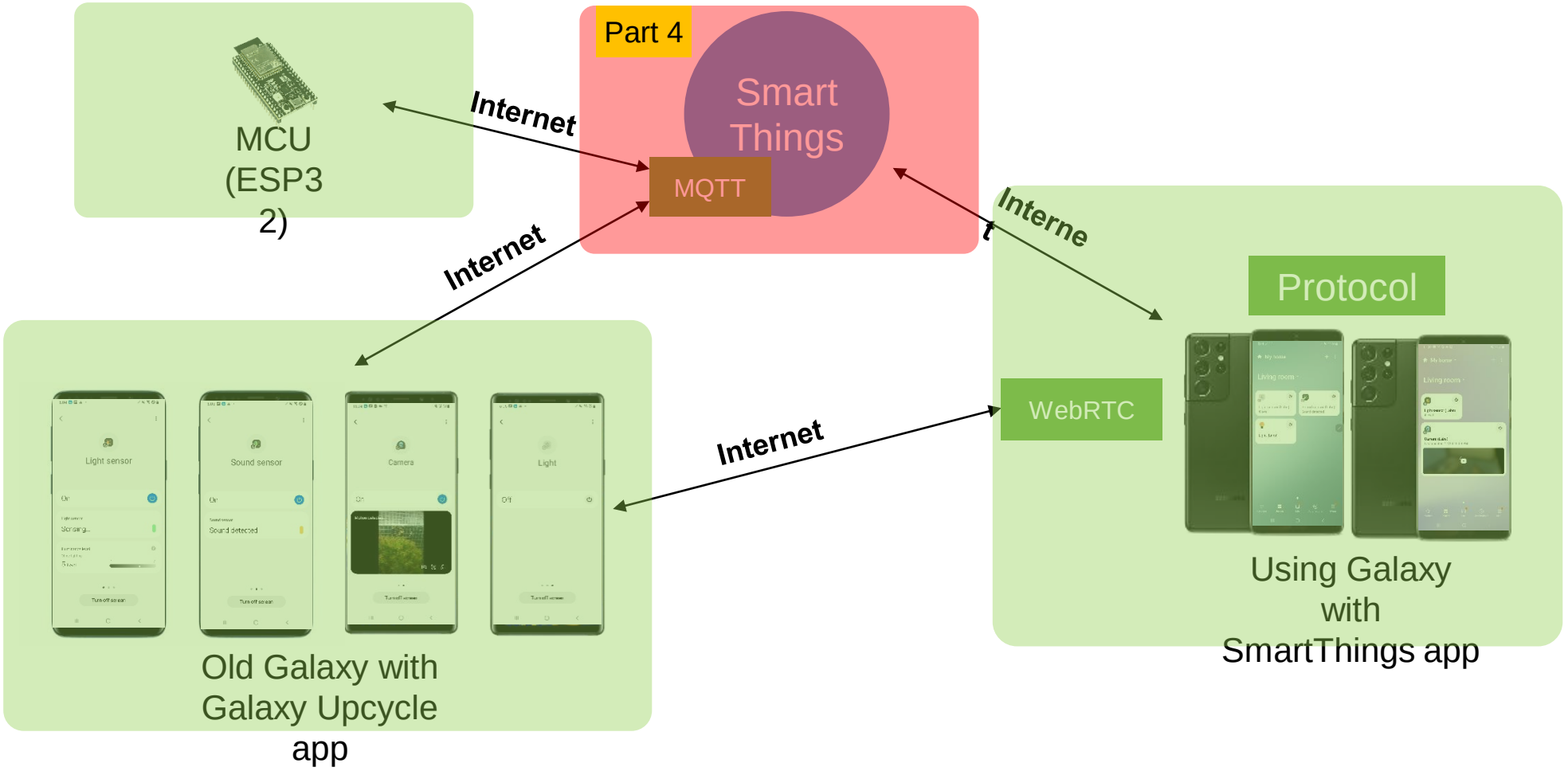
... Strings omitted ...
```

- Kiểm tra biên dịch bằng dữ liệu cấu hình thiết bị mẫu

```
... Strings omitted ...

Scanning dependencies of target __idf_main
[ 97%] Building C object esp-idf/main/CMakeFiles/__idf_main.dir/main.c.obj
[ 97%] Building C object esp-idf/main/CMakeFiles/__idf_main.dir/iot_cli_cmd.c.obj
[ 97%] Building C object esp-idf/main/CMakeFiles/__idf_main.dir/device_control.c.obj
[ 97%] Building C object esp-idf/main/CMakeFiles/__idf_main.dir/iot_uart_cli.c.obj
[ 98%] Building C object esp-idf/main/CMakeFiles/__idf_main.dir/caps_switch.c.obj
[ 98%] Building ASM object esp-idf/main/CMakeFiles/__idf_main.dir/__/__/device_info.json.S.obj
[ 98%] Building ASM object esp-idf/main/CMakeFiles/__idf_main.dir/__/__/onboarding_config.json.S.obj
[ 98%] Linking CXX static library libmain.a
[ 98%] Built target __idf_main
Scanning dependencies of target __ldgen_output_esp32.project.ld
[ 98%] Generating ld/esp32.project.ld
[ 98%] Built target __ldgen_output_esp32.project.ld
Scanning dependencies of target switch_example.elf
[ 98%] Building C object CMakeFiles/switch_example.elf.dir/project_elf_src.c.obj
[100%] Linking CXX executable switch_example.elf
[100%] Built target switch_example.elf
Scanning dependencies of target gen_project_binary
[100%] Generating binary image from built executable
esptool.py v2.8
Generated /st-device-sdk-c-ref/apps/esp32/switch_example/build/switch_example.bin
[100%] Built target gen_project_binary
Scanning dependencies of target app
[100%] Built target app

Project build complete. To flash, run this command:
/root/.espressif/python_env/idf4.0_py3.8_env/bin/python ../../../bsp/esp32/components/esptool_py/esptool/esptool.py -p (PORT) -b 460800 —before default_reset —after hard_reset write_flash —
flash_mode dio —flash_size detect —flash_freq 40m 0x1000 build/bootloader/bootloader.bin 0x8000 build/partition_table/partition-table.bin 0x11000 build/ota_data_initial.bin 0x20000
build/switch_example.bin
or run 'idf.py -p (PORT) flash'
————————————————————————
binary path : /st-device-sdk-c-ref/output/esp32/iotcore_switch_example_20211213_3df85fe_57a3f55
————————————————————————
root@ad7b99273e5e:/st-device-sdk-c-ref#
```


MCU
(ESP3
2)
Internet
Part 4
Smart
Things
MQTT
Internet
Interne
t
Protocol
WebRTC
Internet
Using Galaxy
with
SmartThings app
Old Galaxy with
Galaxy Upcycle
app

BÀI 4.

Cấu hình cuối và kiểm thử: Bóng đèn thông minh

- 4.1. Các bước thực hiện
- 4.2. Tạo tài khoản nhà phát triển SmartThings
- 4.3. Tạo dự án mới
- 4.4. Thêm thiết bị kiểm thử
- 4.5. Đăng ký thiết bị SIC Galup Direct vào ứng dụng SmartThings
- 4.6. Kiểm thử điều khiển và tự động hóa
- 4.7. Tổng kết cuối khóa

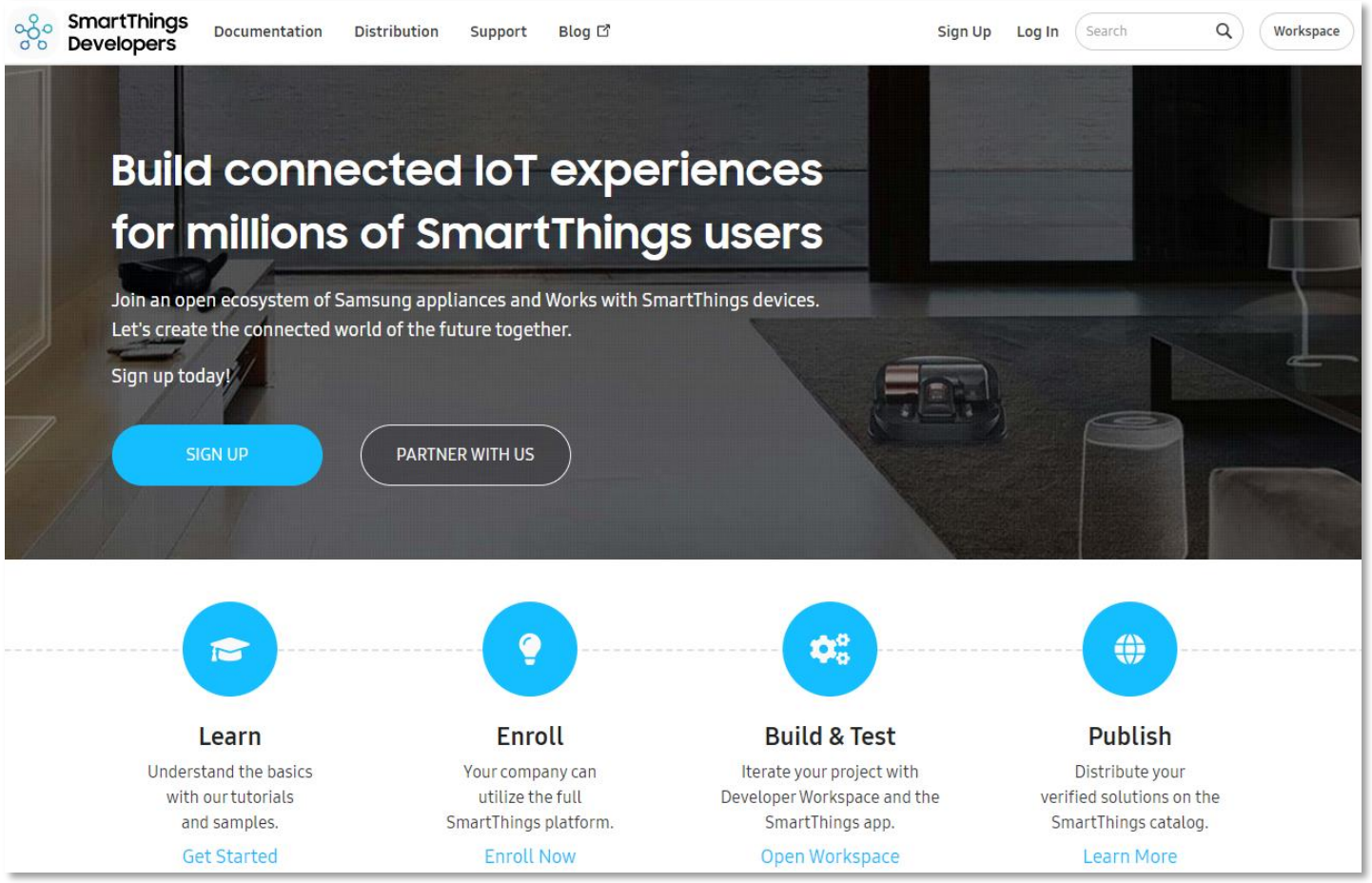
4.1. Các bước thực hiện

Thời gian dự kiến: 20 phút

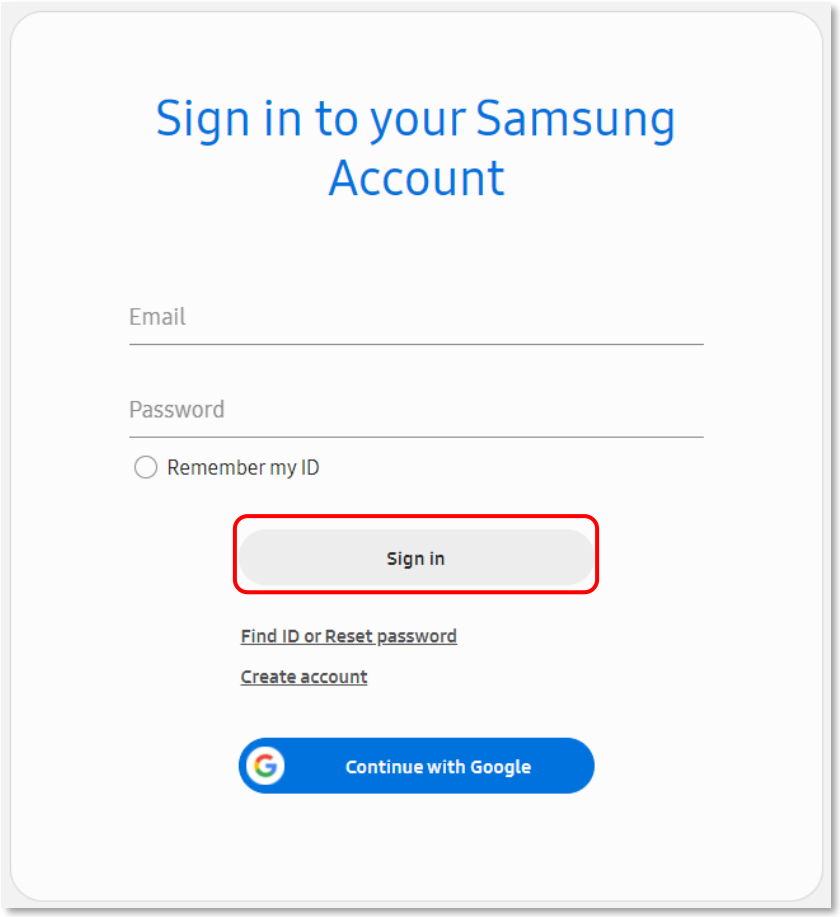
- Đăng ký tài khoản nhà phát triển trên Developer Workspace và đăng ký thiết bị thông qua nền tảng này

1. Đăng ký tài khoản nhà phát triển trên Developer Workspace
2. Tạo dự án mới – Tích hợp thiết bị – Kết nối trực tiếp
3. Tạo hồ sơ thiết bị với tên sic-galup-thing-01 và chức năng bật/tắt
4. Thiết lập quy trình đăng ký thiết bị
5. Tạo thông tin sản phẩm
6. Thêm thiết bị kiểm thử với thông tin định danh (số sê-ri và khóa công khai)

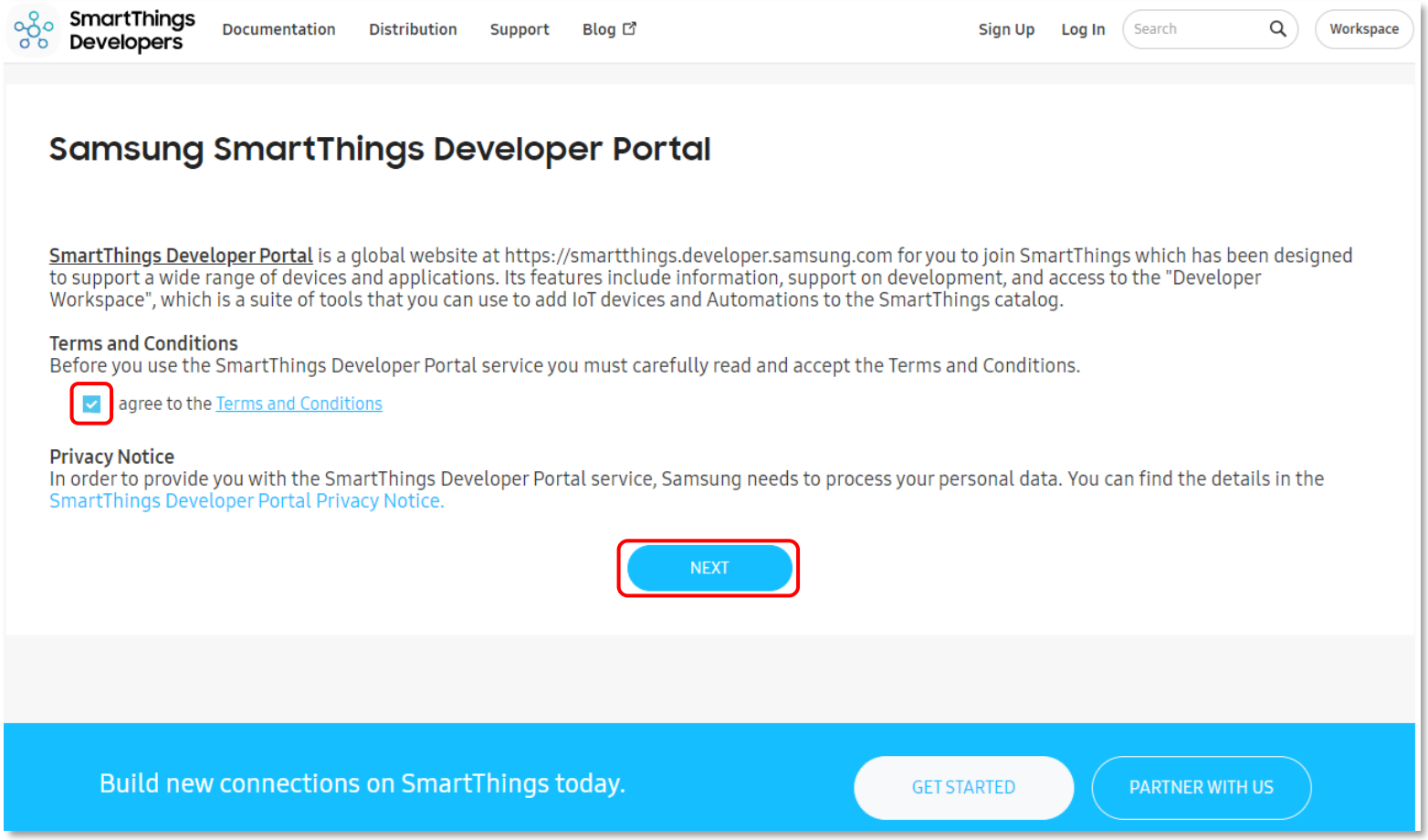
- Truy cập https://smartthings.developer.samsung.com



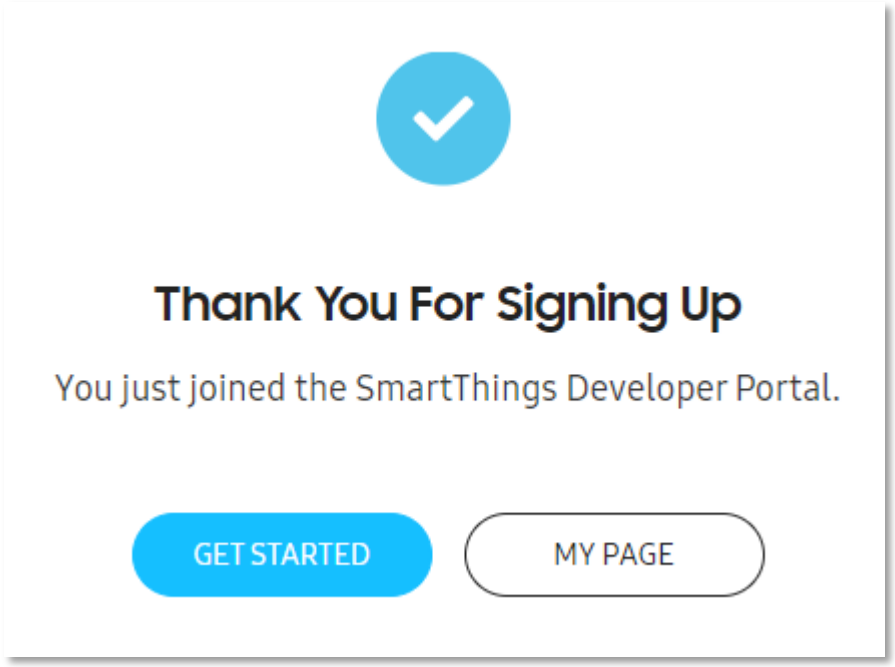
- Đăng ký bằng tài khoản SmartThings



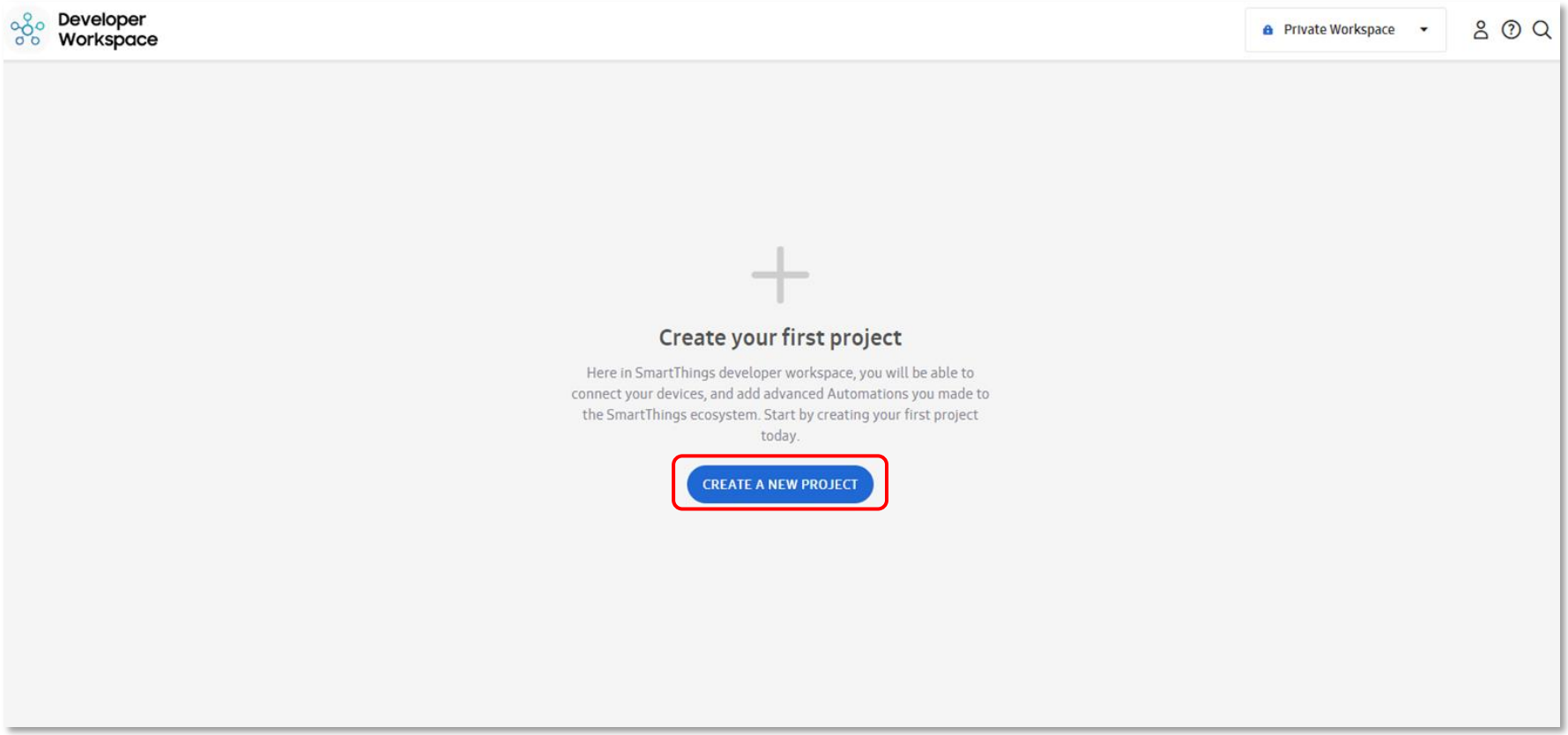
- Xác nhận điều khoản sử dụng và nhấn “Tiếp theo”



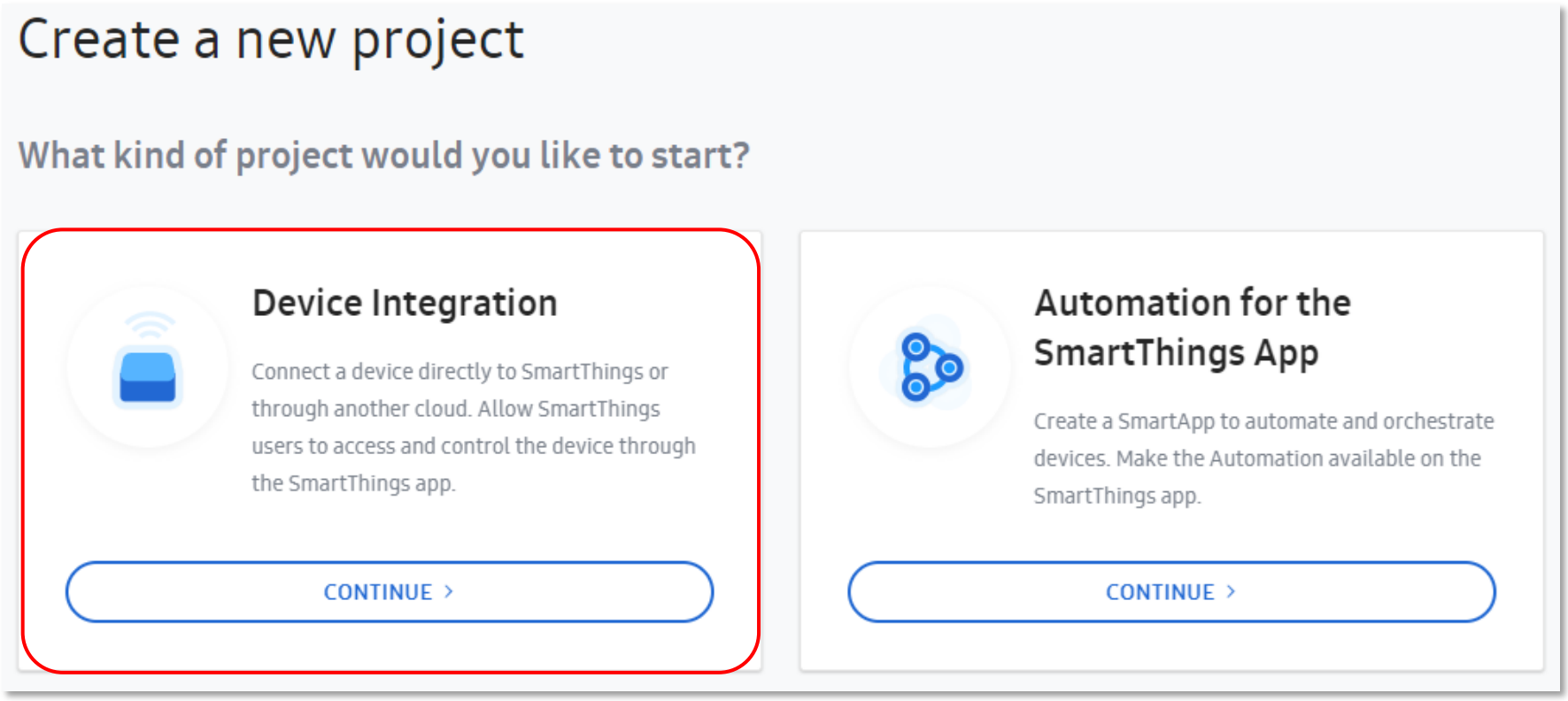
- Quá trình đăng ký đã hoàn tất



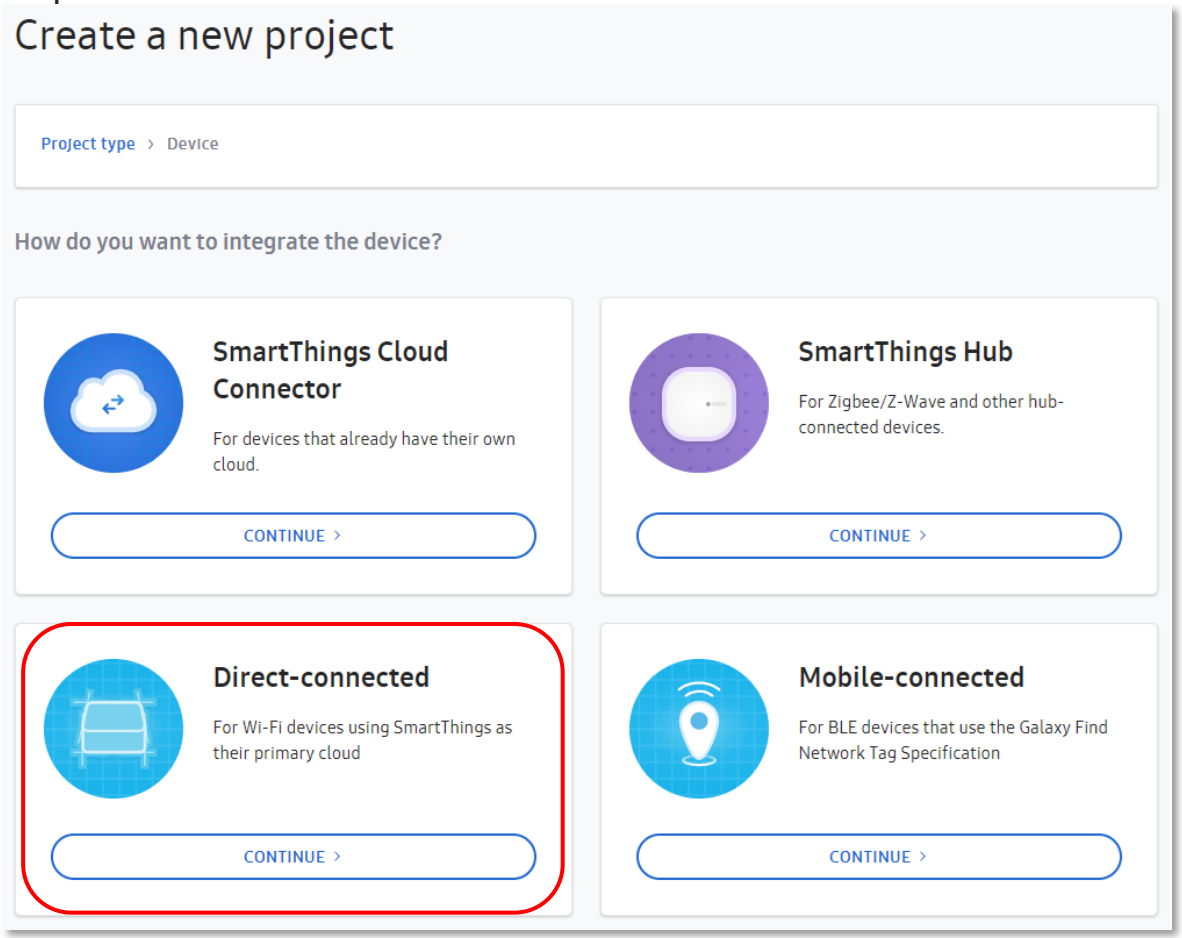
- Truy cập mục “Workspace” và tạo dự án mới



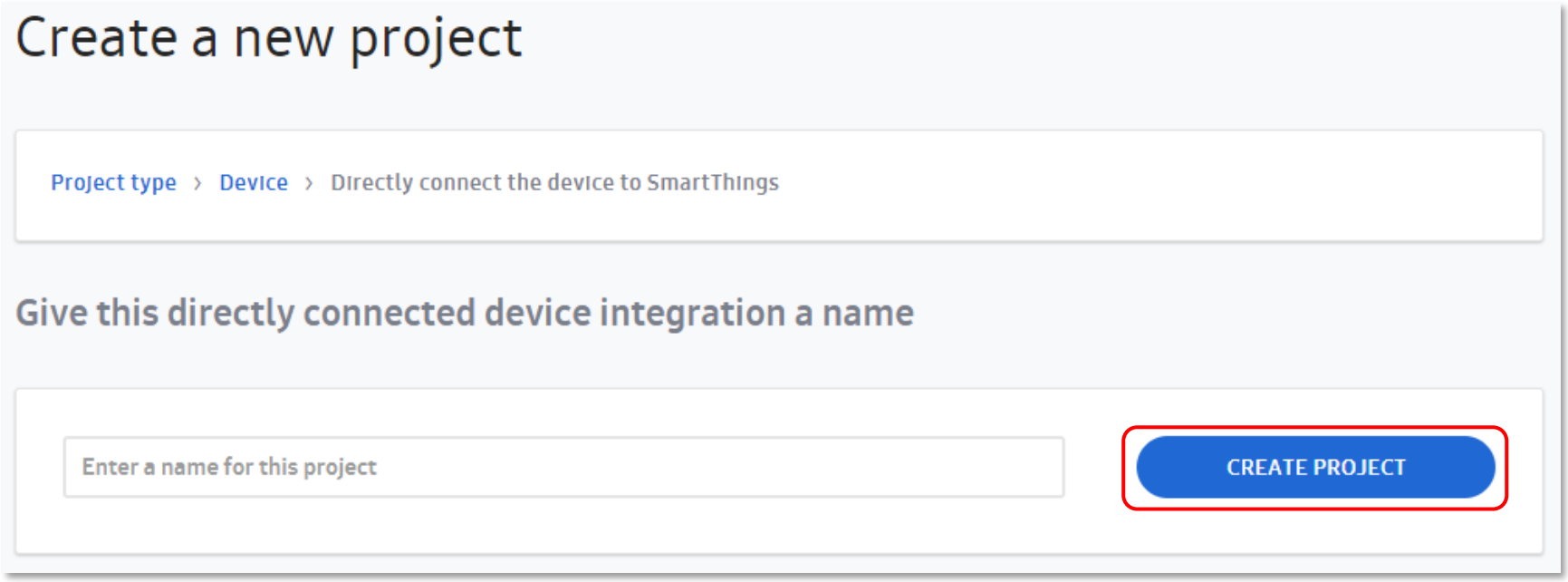
- Chọn mục “Tích hợp thiết bị”



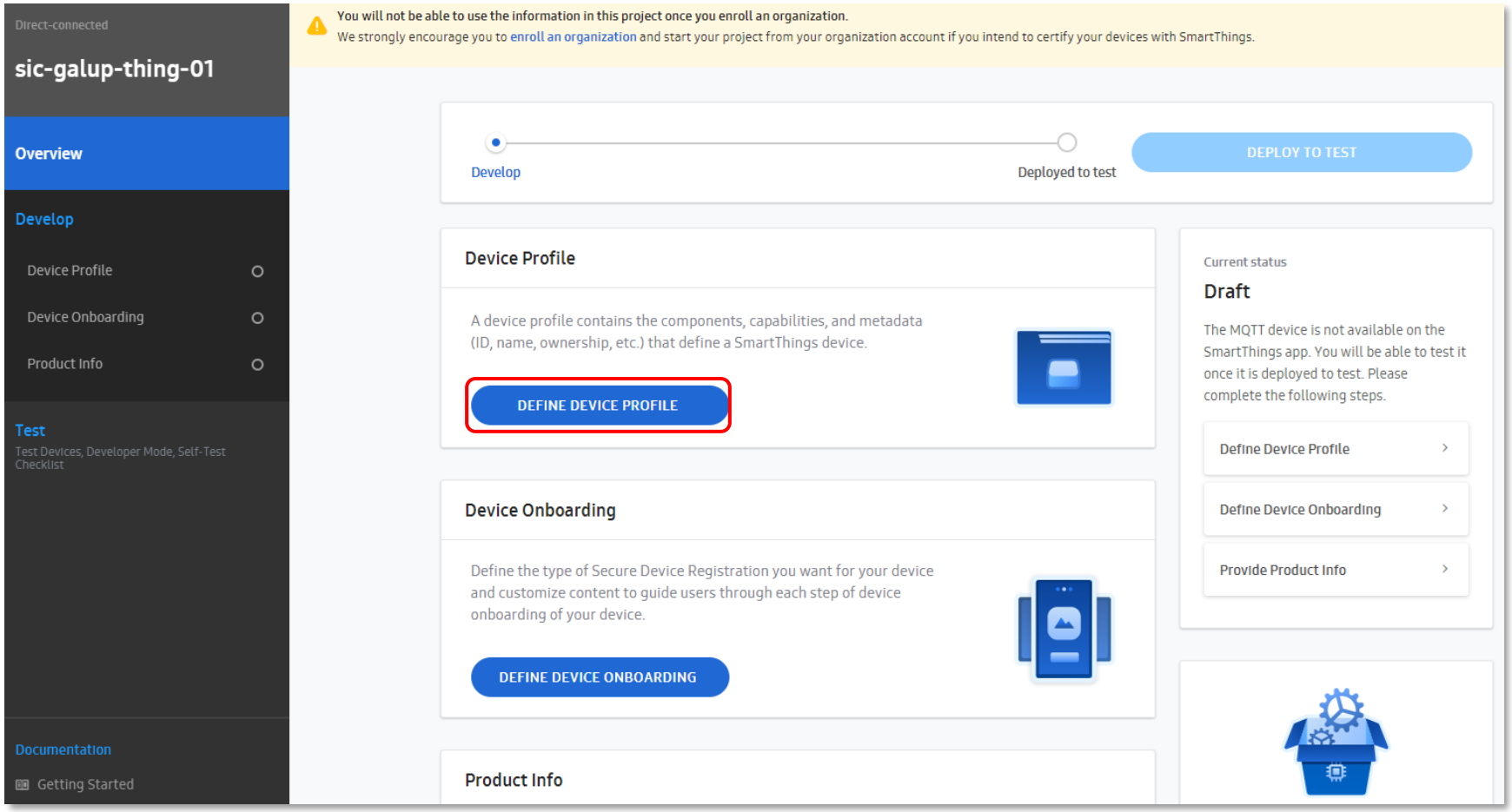
- Chọn “Kết nối trực tiếp”



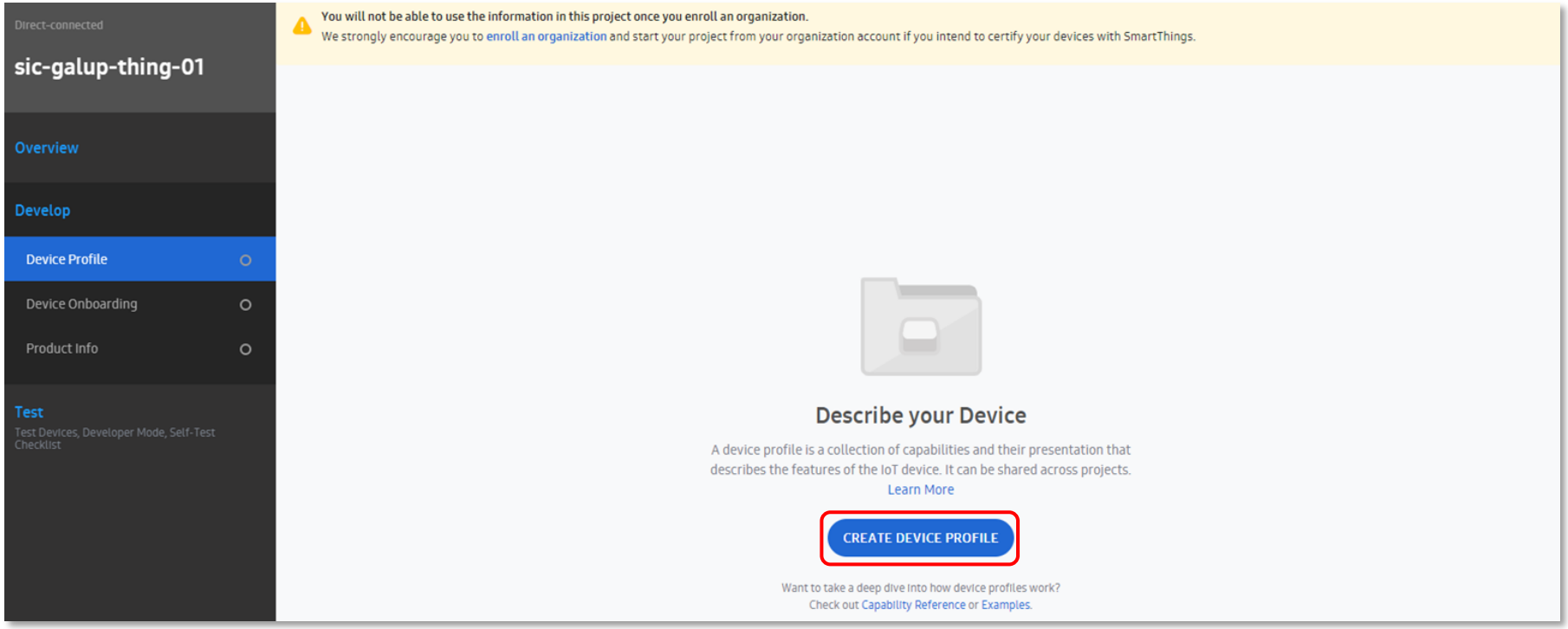
- Nhập tên thiết bị kết nối trực tiếp (ví dụ: sic-galup-thing-01) và nhấn nút “Tạo dự án”



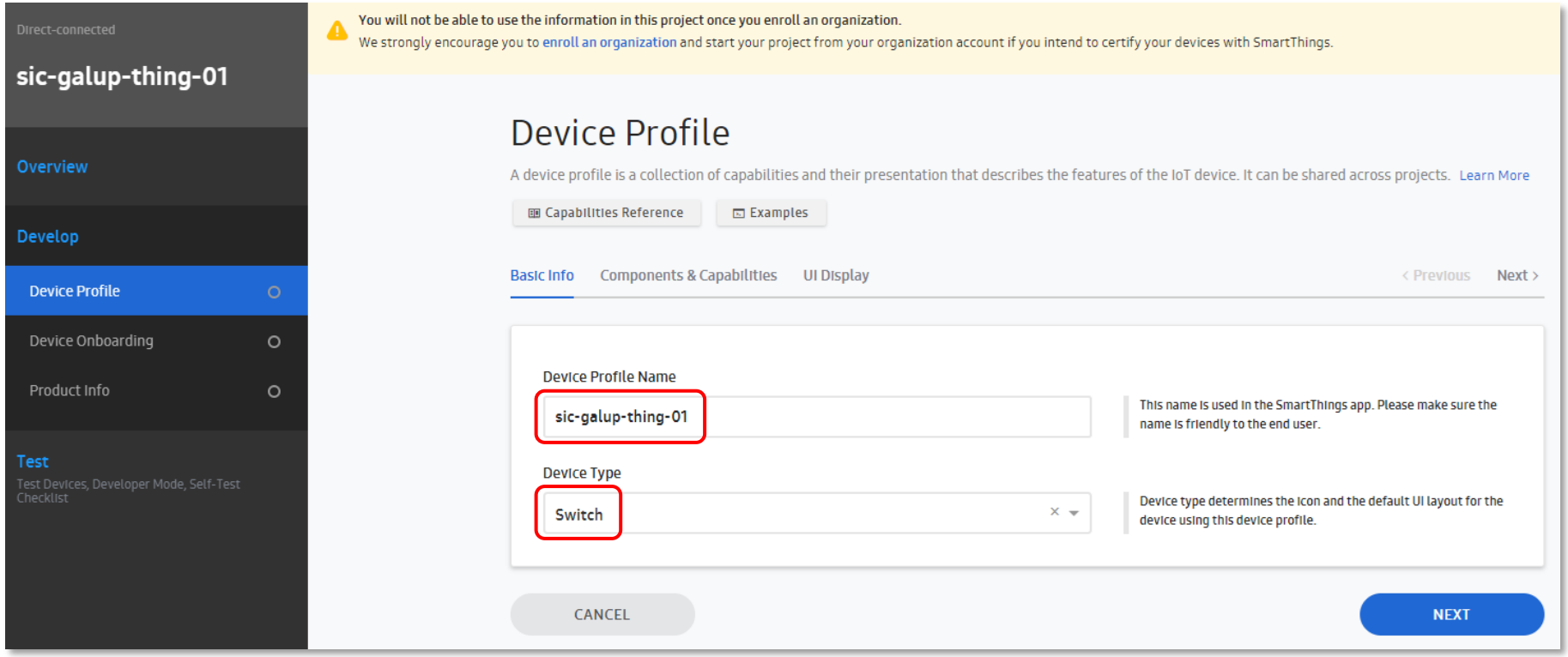
- Chọn “Định nghĩa hồ sơ thiết bị”



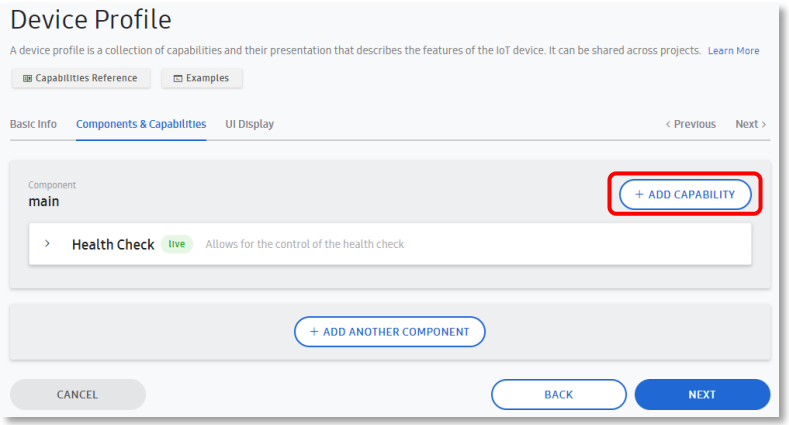
- Chọn nút “Tạo hồ sơ thiết bị”

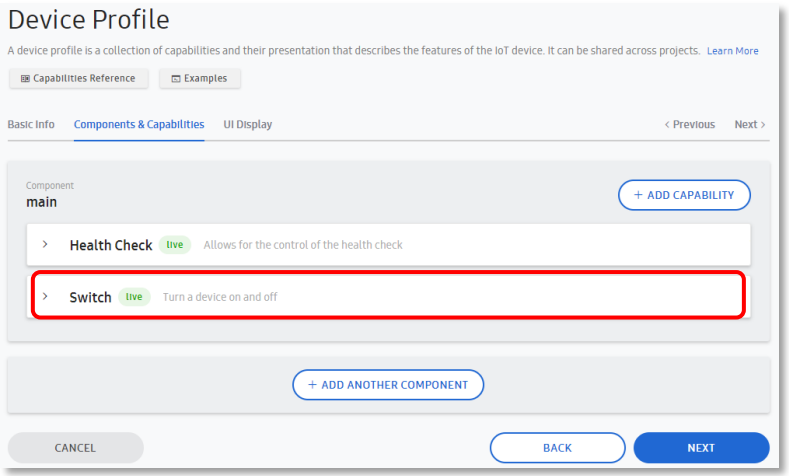


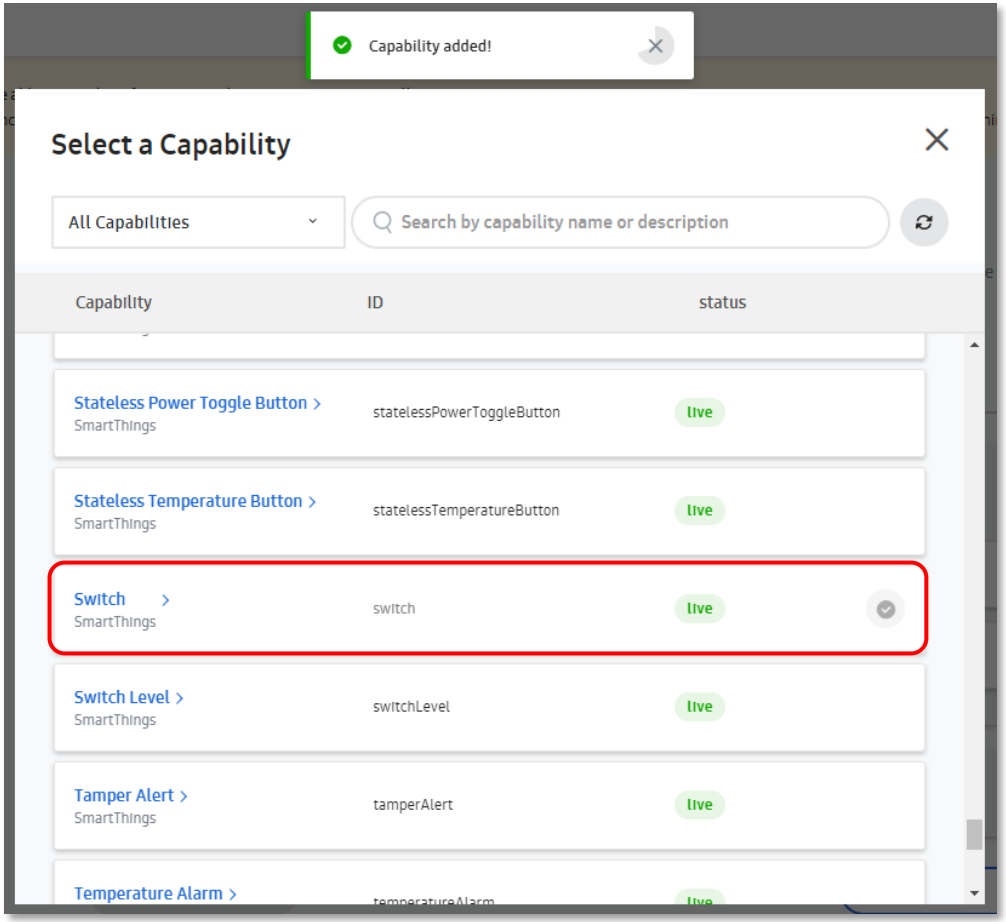
- Nhập tên vào mục “Tên hồ sơ thiết bị” (ví dụ: sic-galup-thing-01) và chọn loại thiết bị (ví dụ: công tắc)



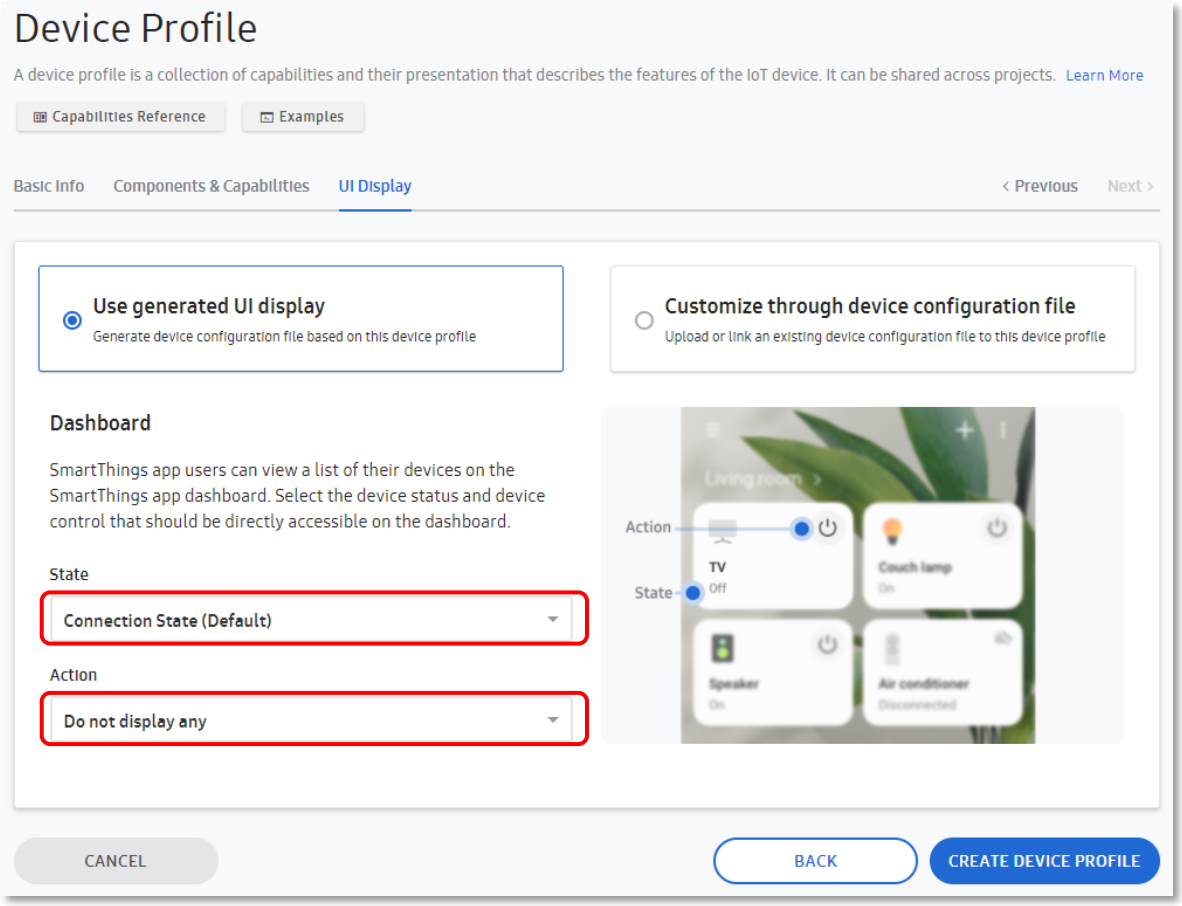
- Nhấn nút “Thêm chức năng”, thêm các chức năng (ví dụ: công tắc) và nhấn “Tiếp theo”



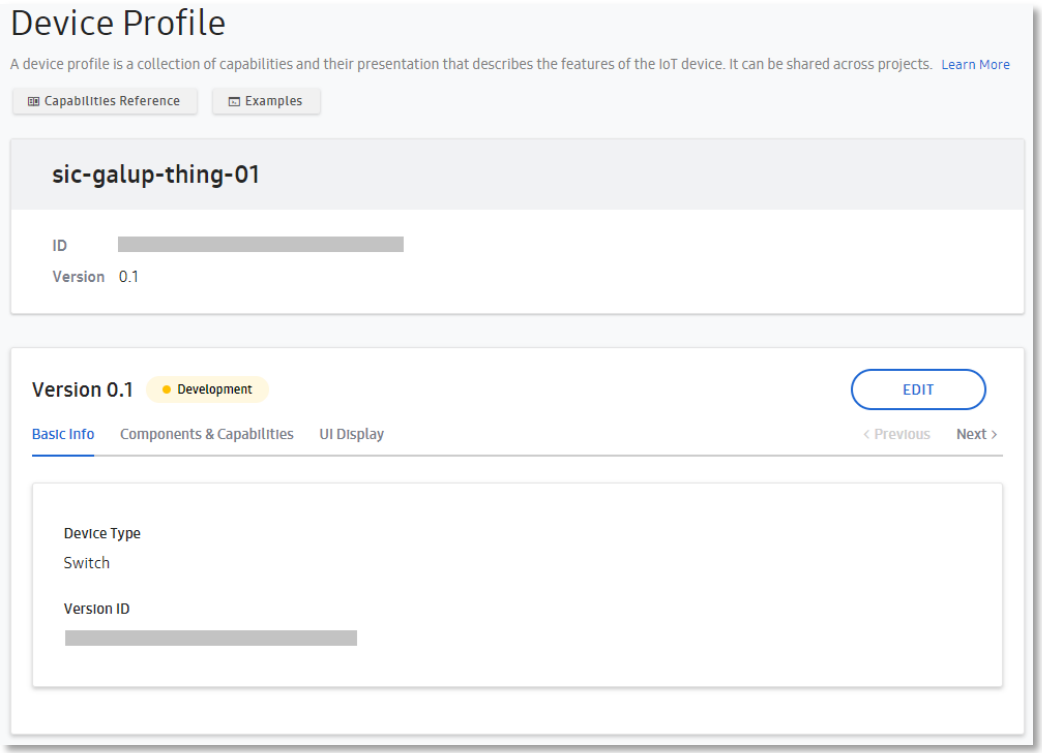


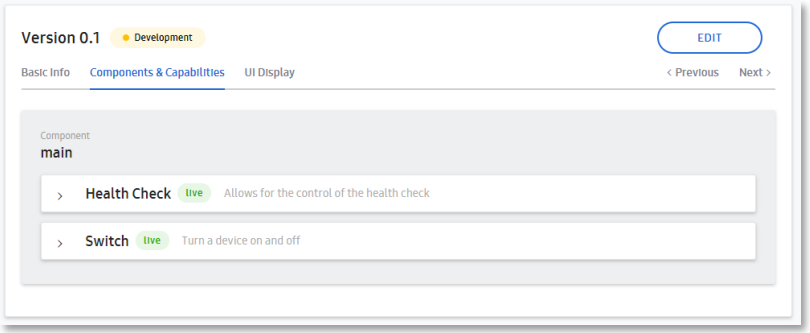


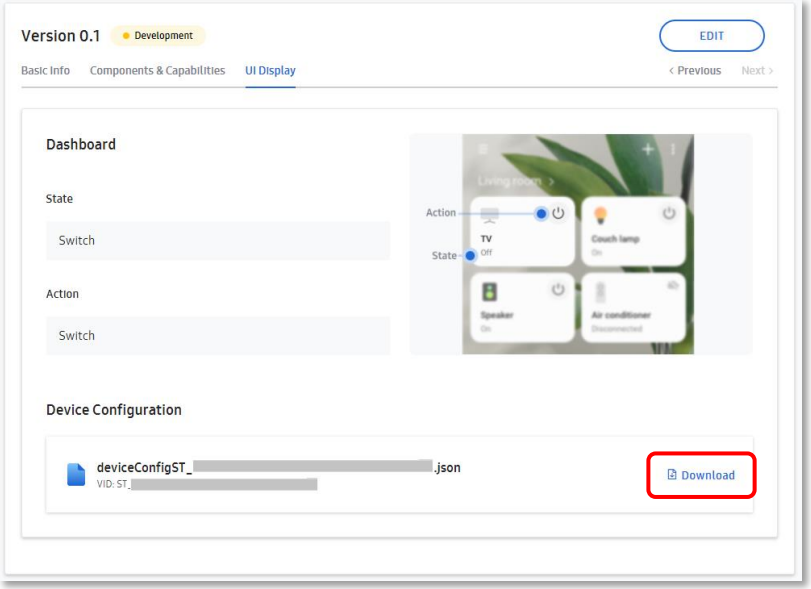
Chọn trạng thái hiển thị trên bảng điều khiển (ví dụ: công tắc) và hành động trên bảng điều khiển (ví dụ: công tắc)



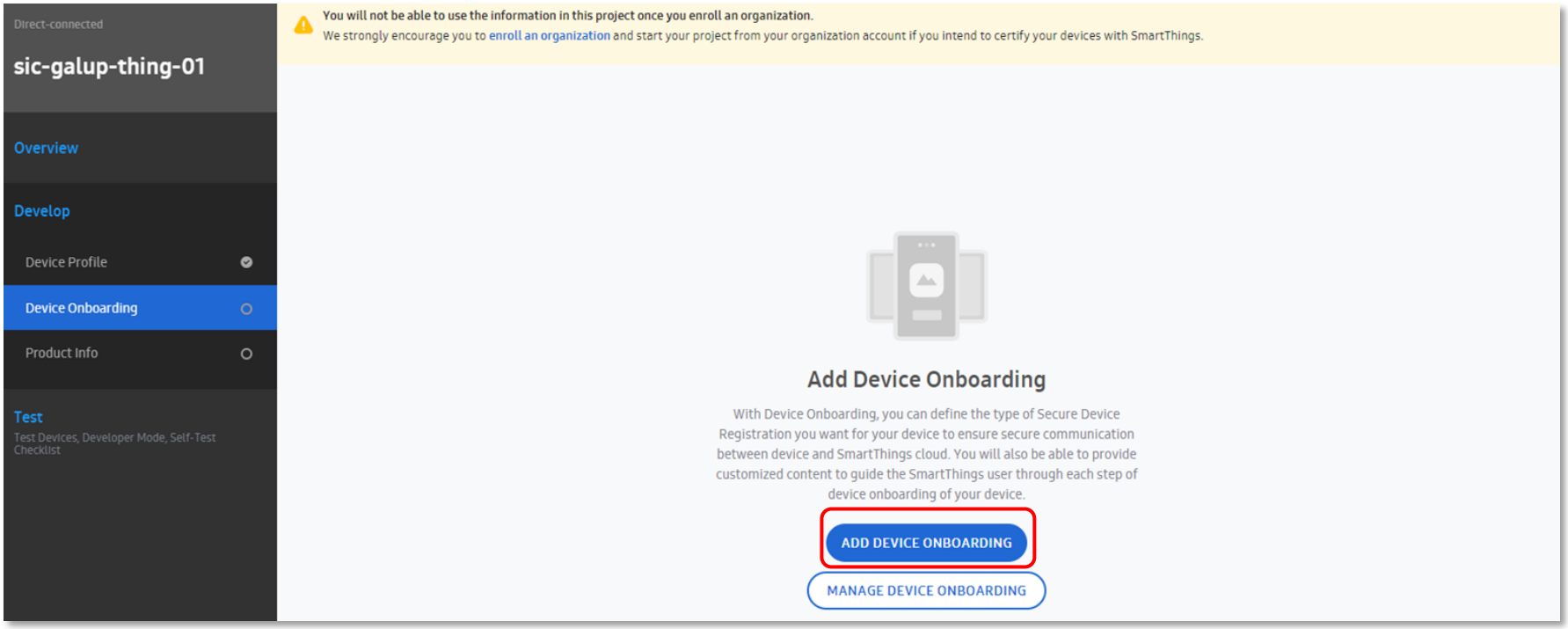
Kiểm tra hồ sơ thiết bị đã lưu và tải xuống tệp cấu hình thiết bị (định dạng JSON) để tùy chỉnh giao diện



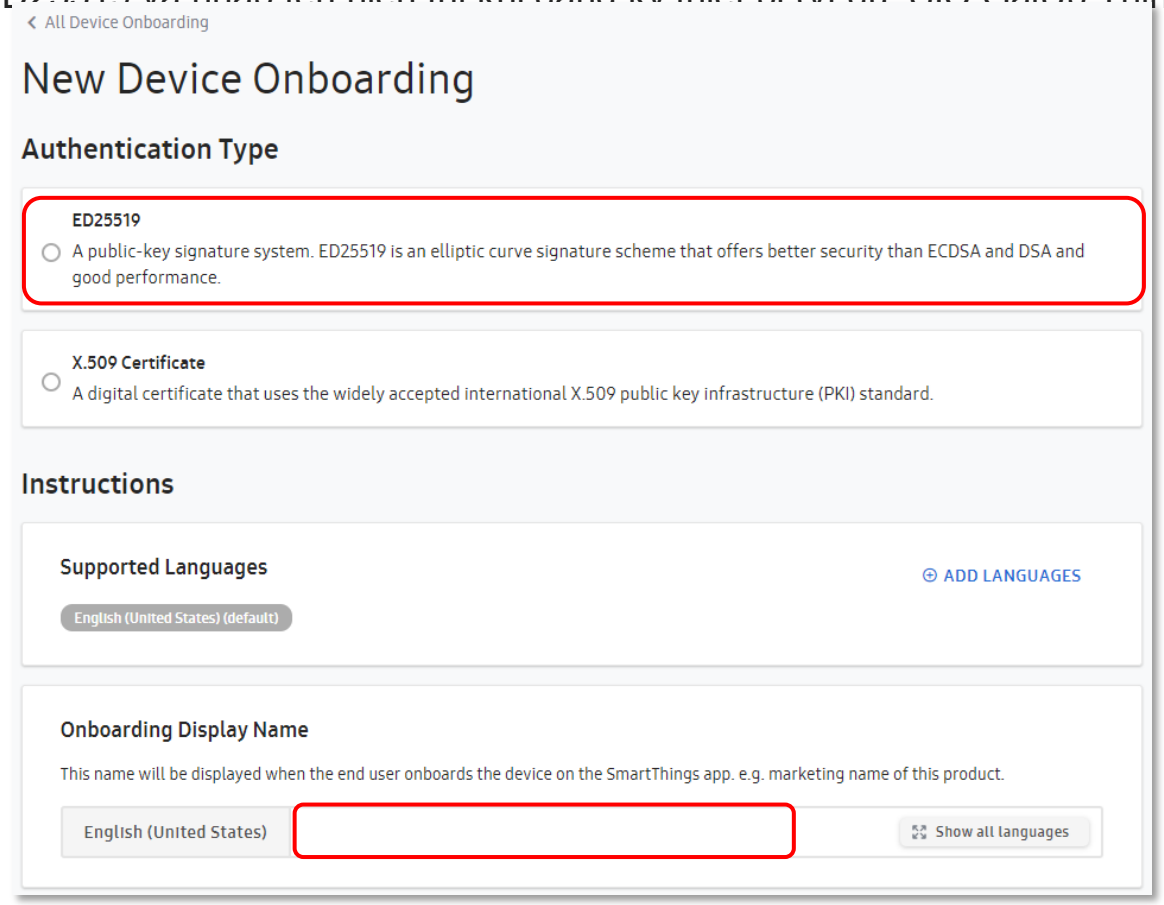




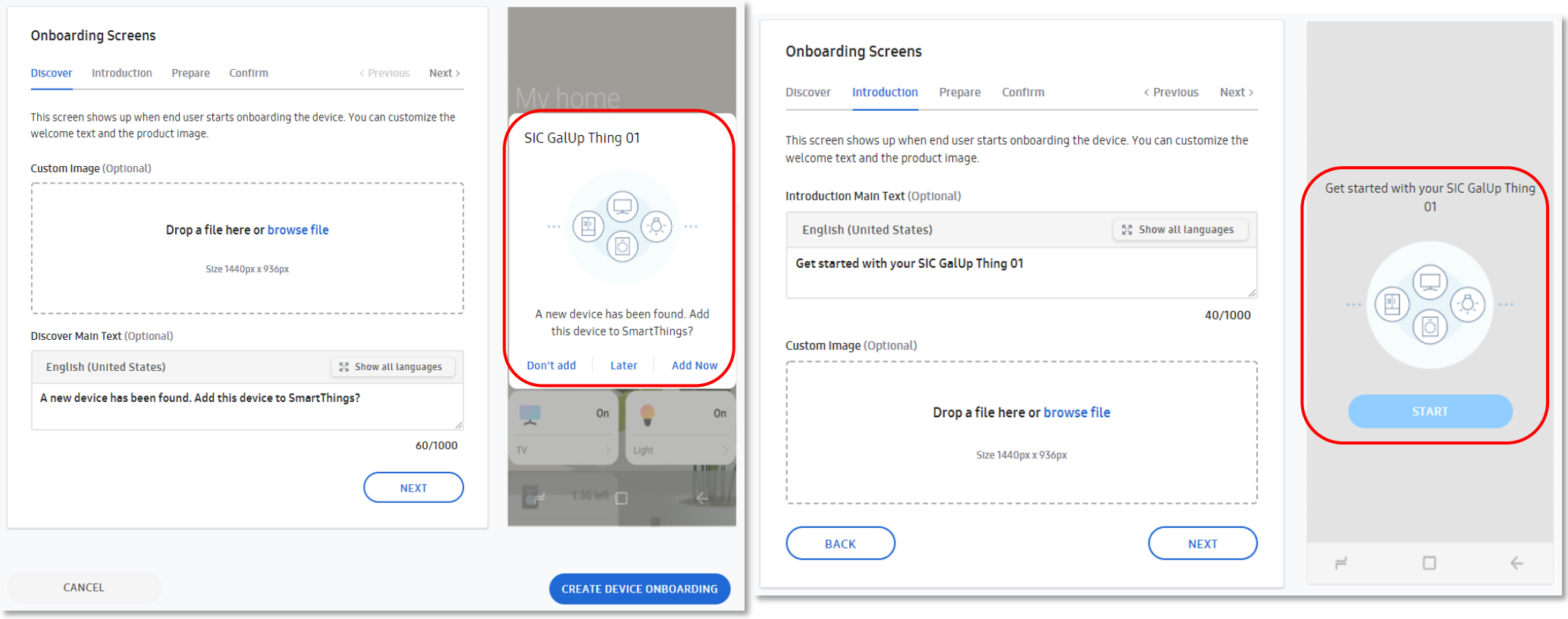
Chọn nút “Thêm quy trình đăng ký thiết bị”



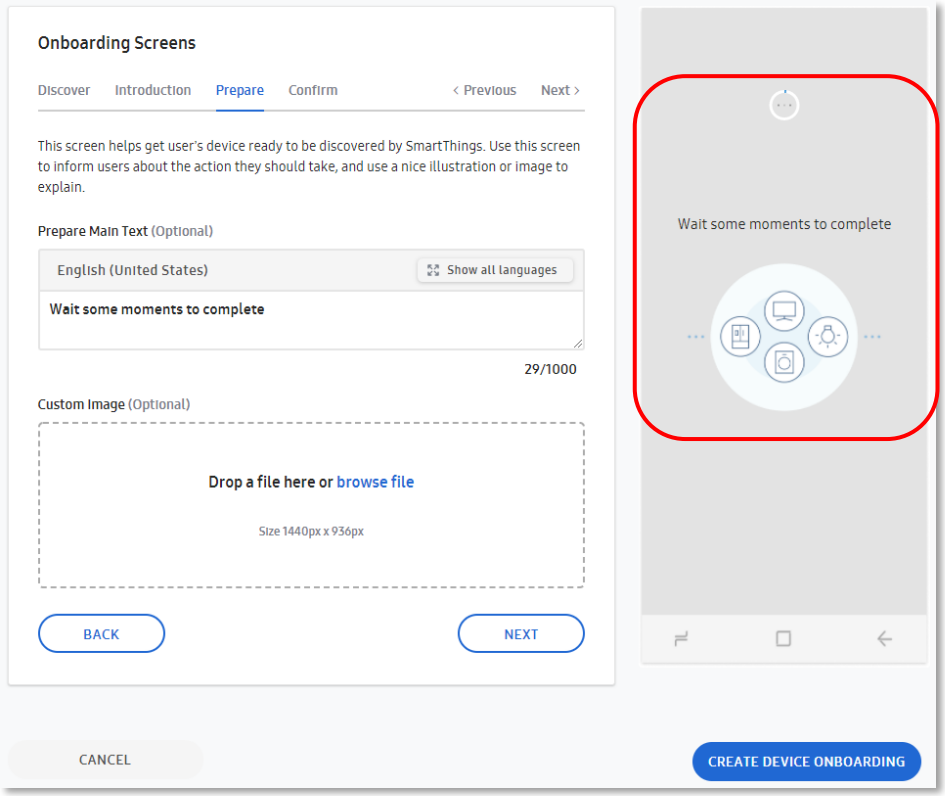
Chọn loại xác thực ED25519 và nhập tên hiển thị khi đăng ký thiết bị (ví dụ: SIC Gall In Thing 01)

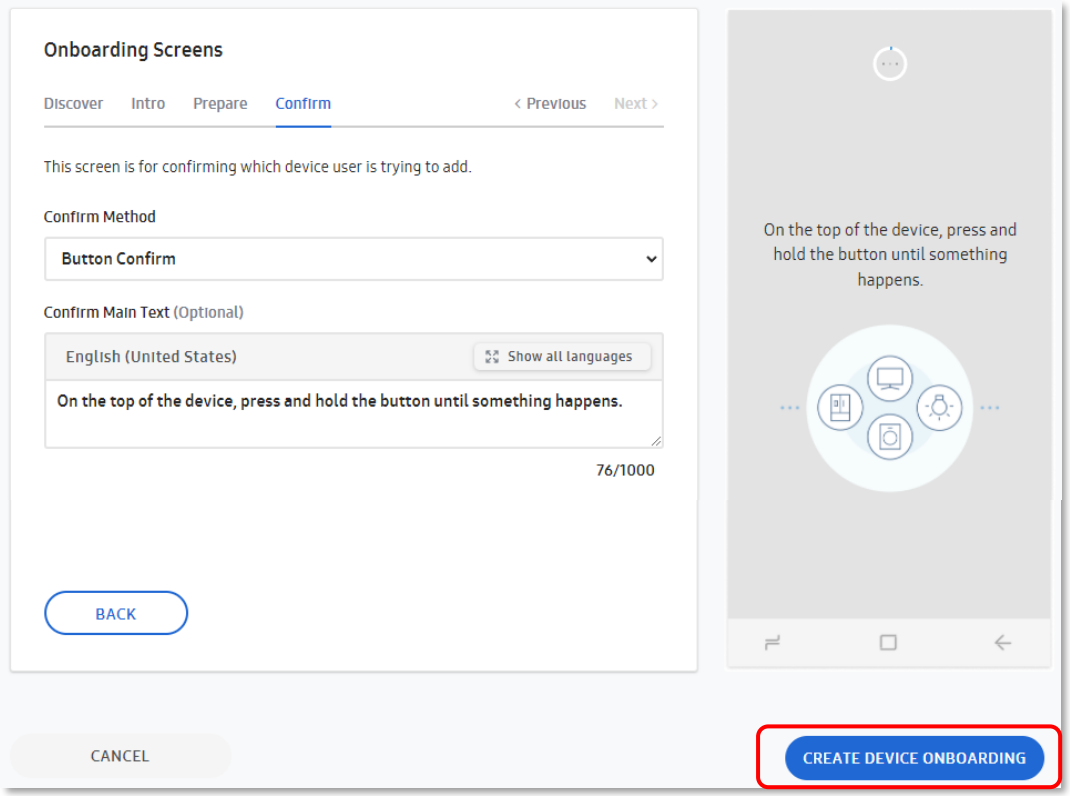


- Kiểm tra các màn hình đăng ký thiết bị (Khám phá và Giới thiệu)

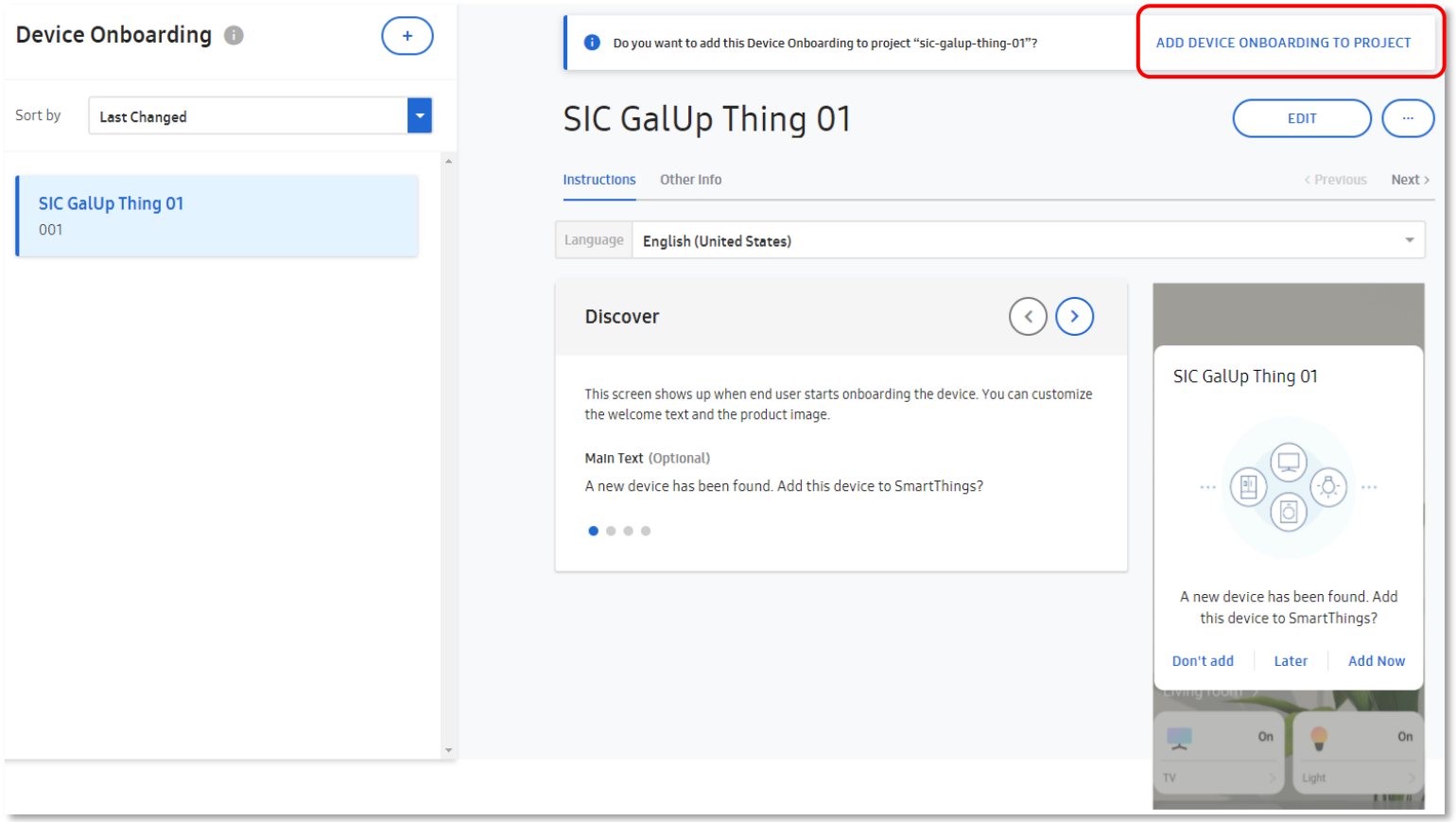


- Kiểm tra các màn hình đăng ký thiết bị (sử dụng nút xác nhận) và chọn nút “Tạo quy trình đăng ký thiết bị”

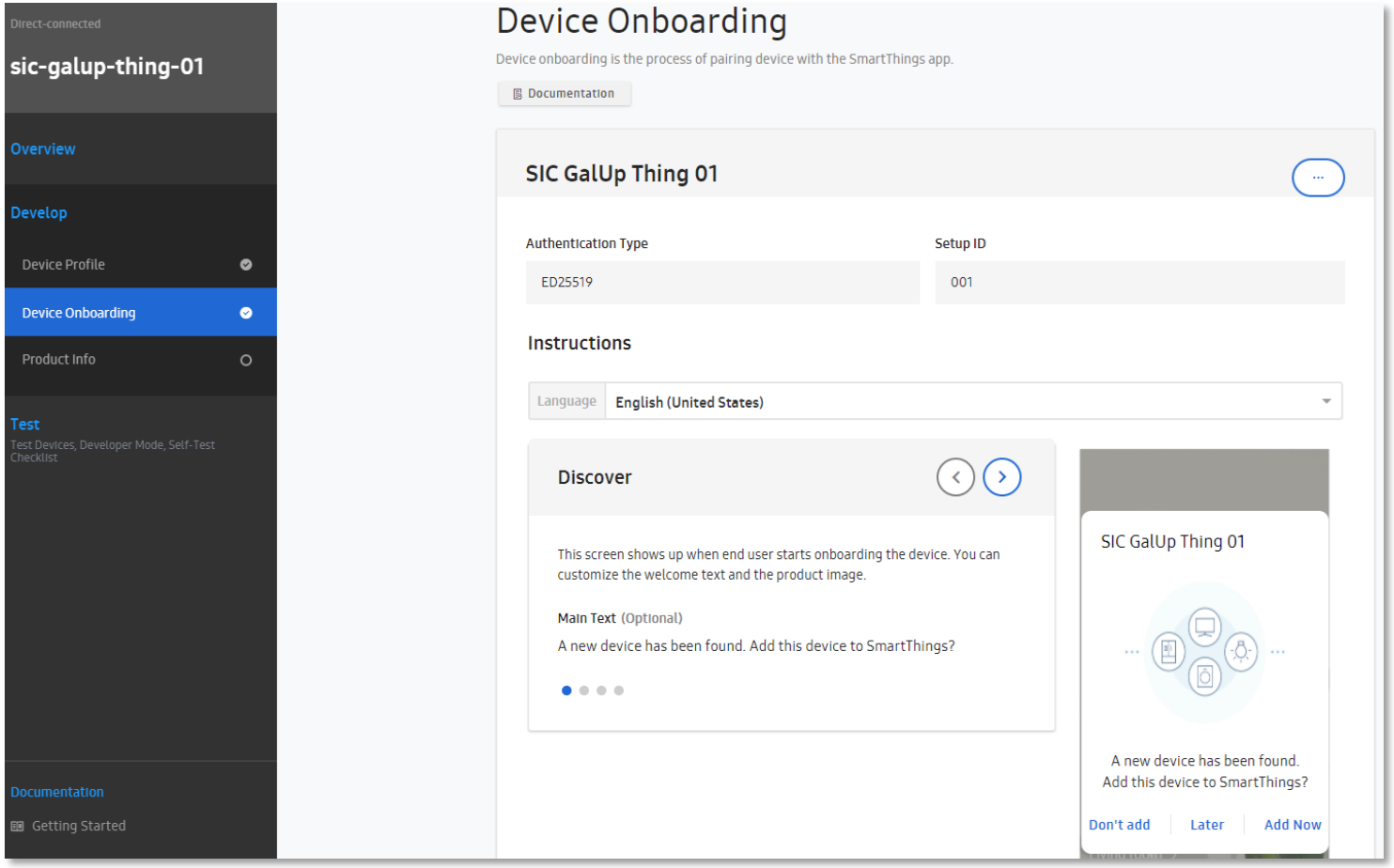




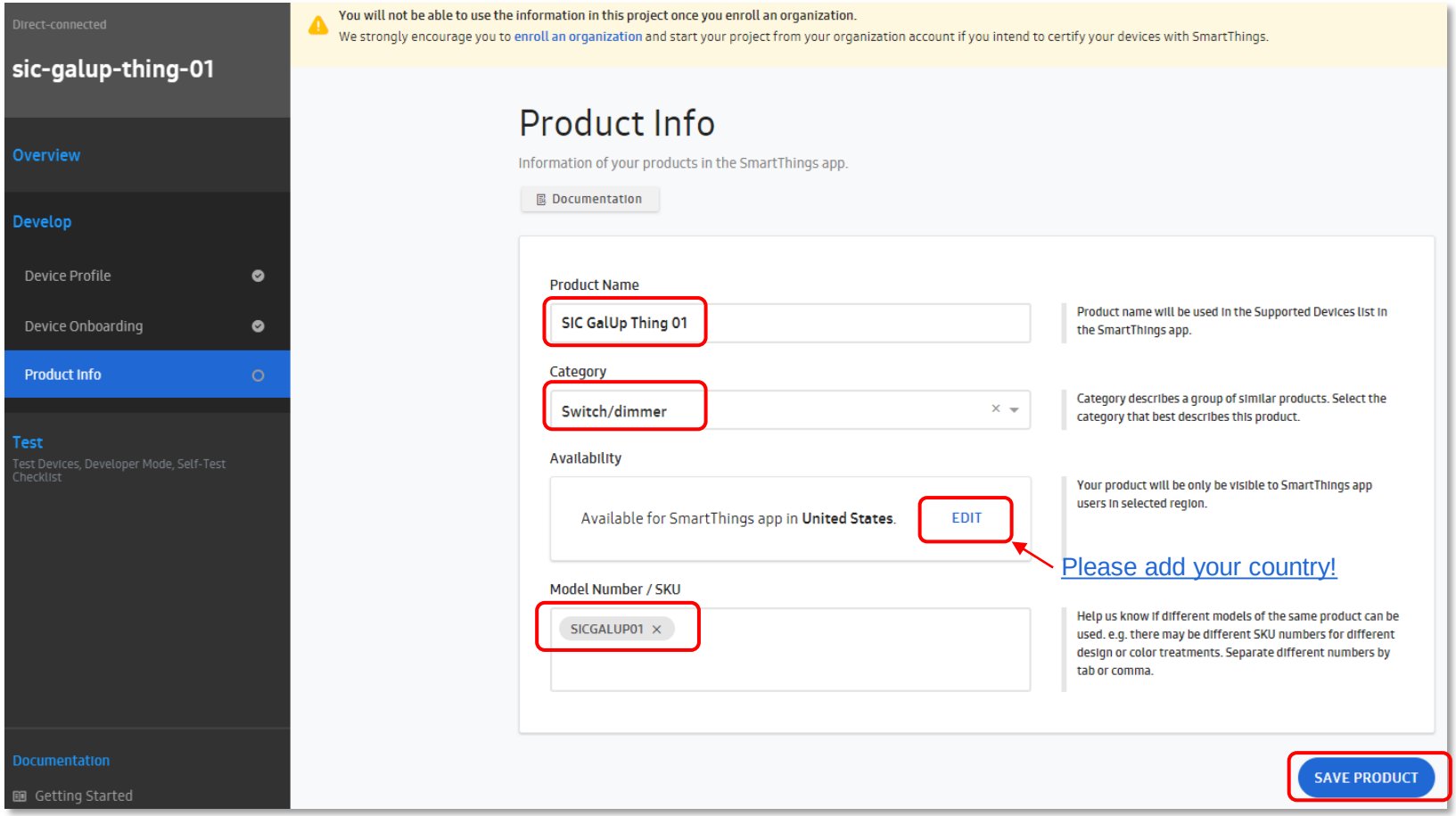
- Chọn nút “Thêm quy trình đăng ký thiết bị vào dự án”



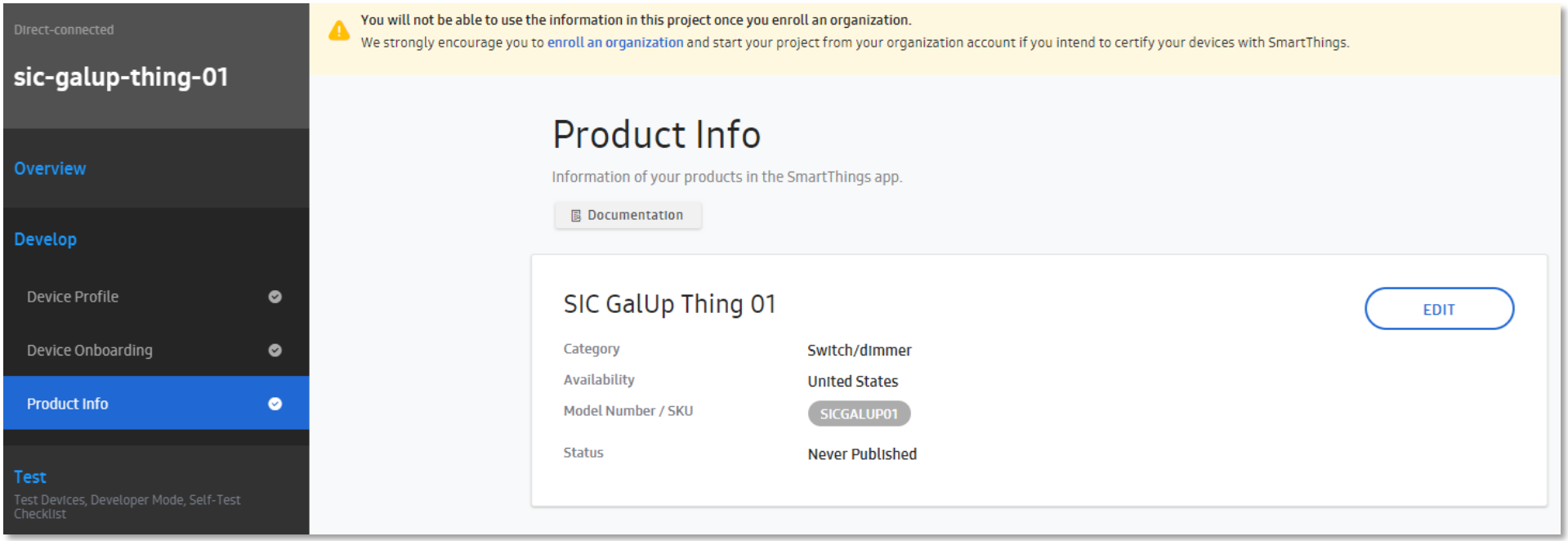
- Kiểm tra quy trình đăng ký thiết bị đã lưu



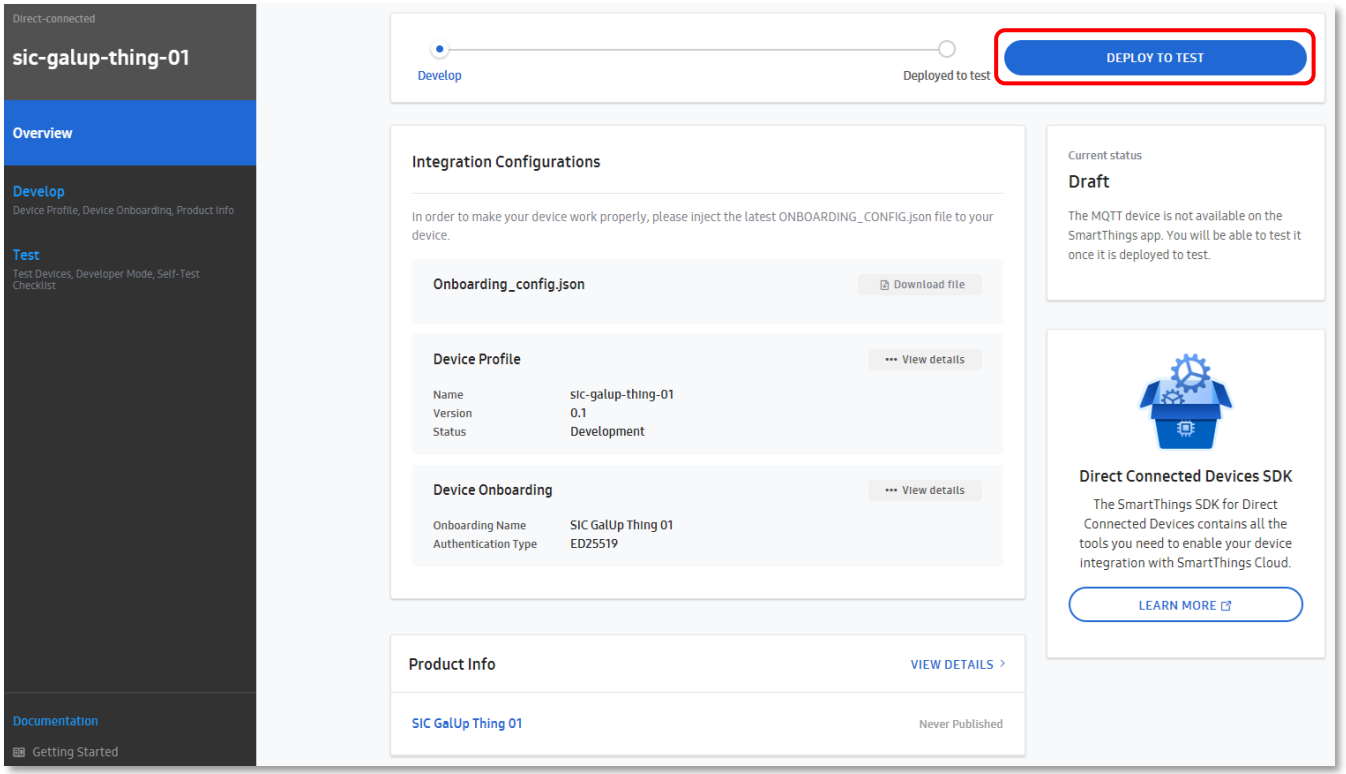
- Nhập tên sản phẩm, mã SKU và chọn danh mục, sau đó nhấn nút “Lưu sản phẩm”

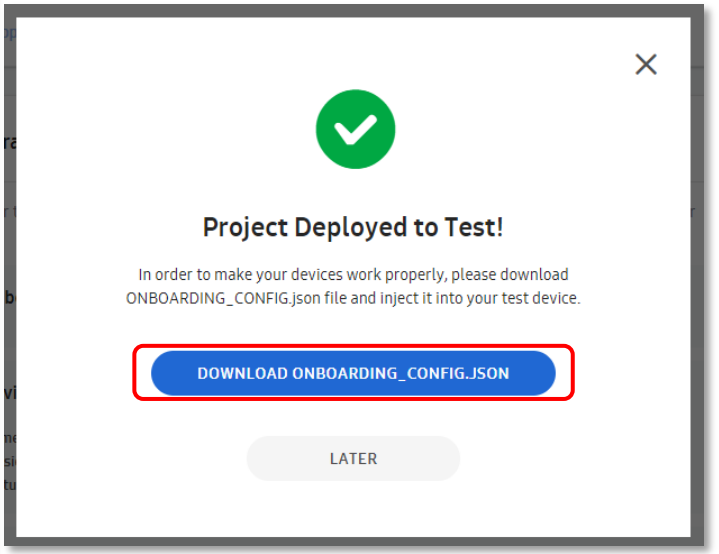


- Kiểm tra thông tin sản phẩm đã lưu

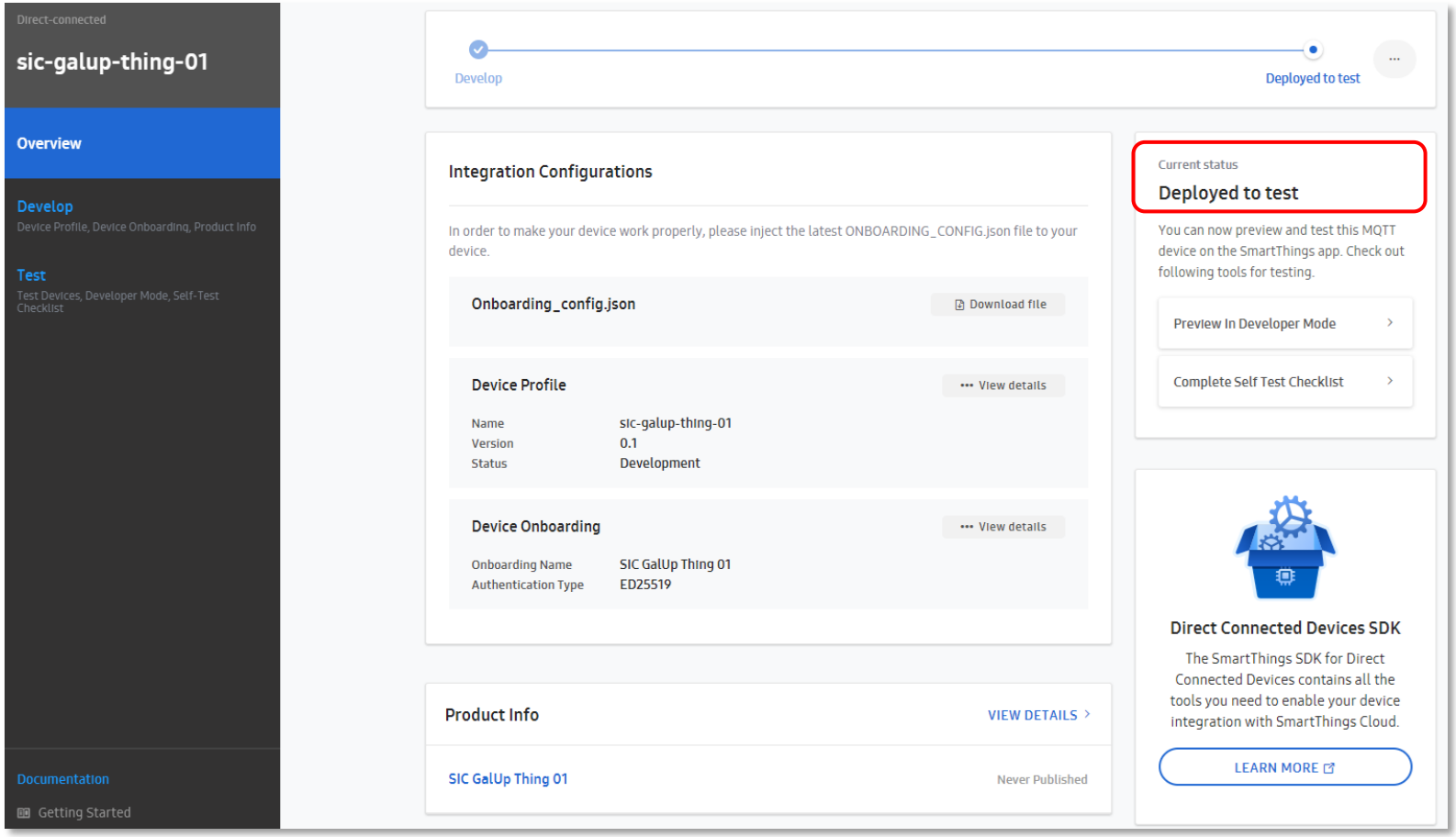


- Nhấn nút “Triển khai để kiểm thử” và chọn “Tải xuống tệp ONBOARDING_CONFIG.JSON”

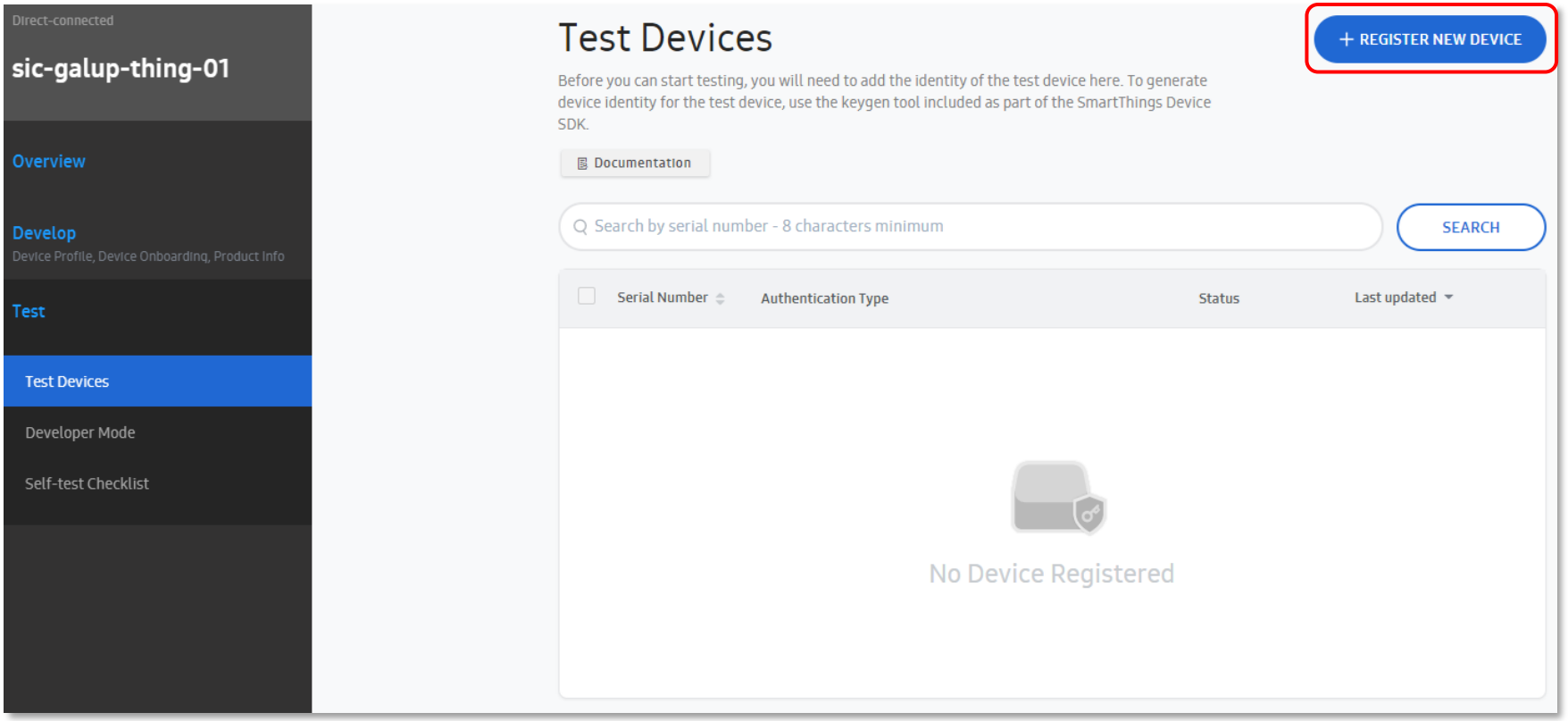




- Kiểm tra trạng thái hiện tại



- Chọn nút “Đăng ký thiết bị mới”



4.4. Thêm thiết bị kiểm thử

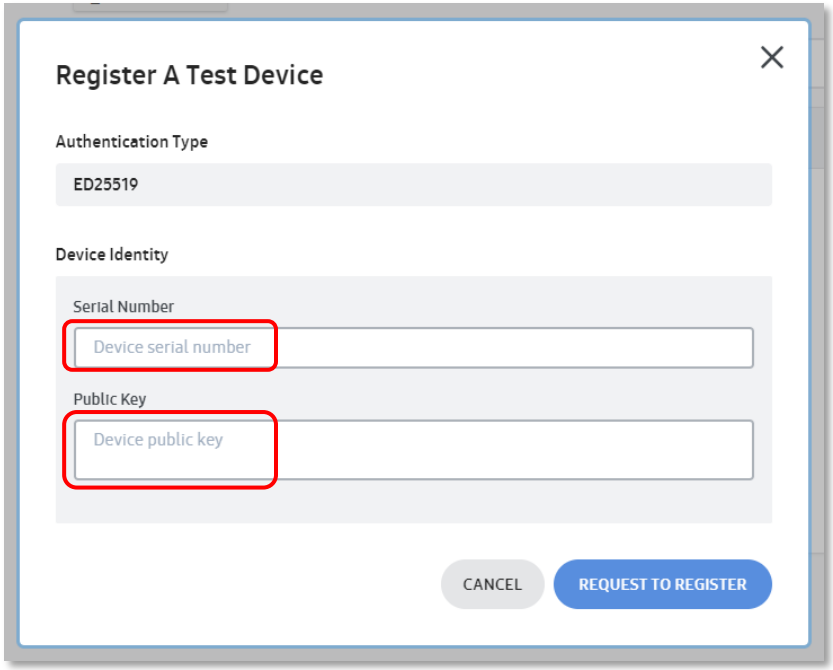
- Nhập số sê-ri và khóa công khai được tạo từ STDK

```
root@e1ae8b10fa68:/st-device-sdk-c-ref# cd /st-device-sdk-c-ref/iot-core/tools/keygen/
root@e1ae8b10fa68:/st-device-sdk-c-ref/iot-core/tools/keygen# ls -al
total 32
drwxr-xr-x 2 root root  4096 Jul  5 00:27 .
drwxr-xr-x 4 root root  4096 Jul  5 00:27 ..
-rw-r--r-- 1 root root  6369 Jul  5 00:27 README.md
-rwxr-xr-x 1 root root 13032 Jul  5 00:27 stdk-keygen.py
root@e1ae8b10fa68:/st-device-sdk-c-ref/iot-core/tools/keygen# python stdk-keygen.py --mnid YOUR_MNID --firmware YOUR_VERSION
-----------------------------NOTICE---------------------------------
qrcode module failed to import. QR code generation will not execute.
If you want to make QR code, please install qrcode module first.
--------------------------------------------------------------------
Use following serial number and public key
for the identity of your device in Developer Workspace.

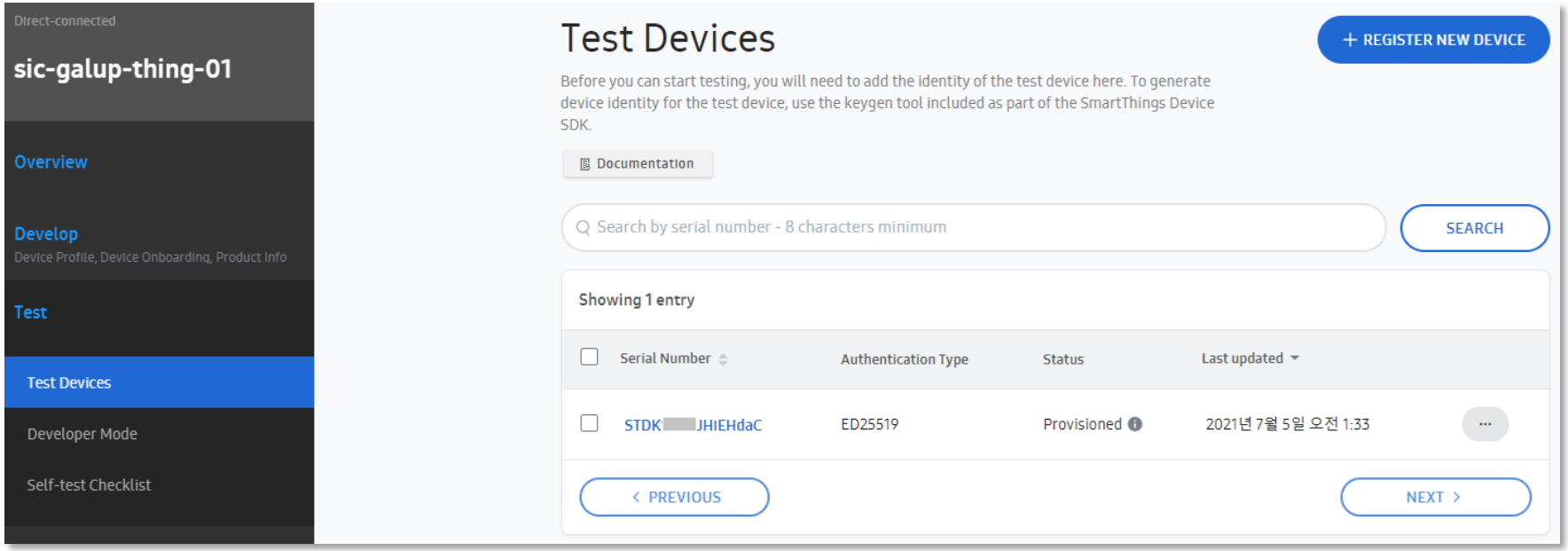
Serial Number:
STDKYOUR_MNIDJHiEHdaC

Public Key:
CX/Ir8vBqcn+OHEFYsxL0gWKmMmR16KESOtS9GEjG/E=

root@e1ae8b10fa68:/st-device-sdk-c-ref/iot-core/tools/keygen#
```



- Kiểm tra thiết bị kiểm thử đã lưu



- Cập nhật tệp cấu hình thiết bị (device_info.json)

```
root@e1ae8b10fa68:/st-device-sdk-c-ref/iot-core/tools/keygen# ls -al output_STDKYOUR_MNIDJHiEHdaC/
total 20
drwxr-xr-x 2 root root 4096 Jul  5 01:20 .
drwxr-xr-x 1 root root 4096 Jul  5 01:20 ..
-rw-r--r-- 1 root root   44 Jul  5 01:20 device.pubkey.b64
-rw-r--r-- 1 root root   44 Jul  5 01:20 device.seckey.b64
-rw-r--r-- 1 root root  222 Jul  5 01:20 device_info.json
root@e1ae8b10fa68:/st-device-sdk-c-ref/iot-core/tools/keygen# cat output_STDKYOUR_MNIDJHiEHdaC/device_info.json
{
    "deviceInfo": {
        "firmwareVersion": "V20210705",
        "privateKey": "xPSS3QFuTjhSRuqH9zqD+QXs9ZLgm3Ojet1TKSvEkfw=",
        "publicKey": "CX/Ir8vBqcn+OHEFYsxL0gWKmMmR16KESOtS9GEjG/E=",
        "serialNumber": "STDKYOUR_MNIDJHiEHdaC"
    }
}
root@e1ae8b10fa68:/st-device-sdk-c-ref/iot-core/tools/keygen# cat /st-device-sdk-c-ref/apps/esp32/switch_example/main/device_info.json
{
    "deviceInfo": {
        "firmwareVersion": "firmwareVersion_here",
        "privateKey": "privateKey_here",
        "publicKey": "publicKey_here",
        "serialNumber": "serialNumber_here"
    }
}
root@e1ae8b10fa68:/st-device-sdk-c-ref/iot-core/tools/keygen# cp output_STDKfcAMJHiEHdaC/device_info.json /st-device-sdk-c-
ref/apps/esp32/switch_example/main/device_info.json
```

- Cập nhật tệp cấu hình đăng ký thiết bị (onboarding_config.json) (1/2)

```
$ docker ps
CONTAINER ID       IMAGE            COMMAND          CREATED          STATUS           PORTS            NAMES
e1ae8b10fa68       stdk_esp32_dev     "bash"           2 hours ago       Up 3 seconds                      blissful_shtern
$ docker exec e1ae cat /st-device-sdk-c-ref/apps/esp32/switch_example/main/onboarding_config.json
{
  "onboardingConfig": {
    "deviceOnboardingId": "NAME",
    "mnId": "MNID",
    "setupId": "999",
    "vid": "VID",
    "deviceTypeId": "TYPE",
    "ownershipValidationTypes": [
      "JUSTWORKS",
      "BUTTON",
      "PIN",
      "QR"
    ],
    "identityType": "ED25519",
    "deviceIntegrationProfileKey": {
      "id": "DIP_UUID",
      "majorVersion": 0,
      "minorVersion": 1
    }
  }
}
```

- Update device configuration file (onboarding_config.json) (2/2)

```
$ docker cp onboarding_config.json e1ae:/st-device-sdk-c-ref/apps/esp32/swtch_example/main/onboarding_config.json
$ docker exec e1ae cat /st-device-sdk-c-ref/apps/esp32/switch_example/main/onboarding_config.json
{
  "onboardingConfig": {
    "deviceOnboardingId": "SIC_GalUp_Thi",
    "mnId": “YOUR_MNID",
    "setupId": "001",
    "vid": "ST_YOUR_VERSION_ID",
    "deviceTypeId": "Switch",
    "ownershipValidationTypes": [
      “BUTTON"
    ],
    "identityType": "ED25519",
    "deviceIntegrationProfileKey": {
      "id": “YOUR_DEVICE_ID",
      "majorVersion": 0,
      "minorVersion": 1
    },
    "productId": "7dce8ed8-74cc-4b02-9c04-f2e0c45de33c"
  }
}
$ docker exec e1ae ls -al /st-device-sdk-c-ref/apps/esp32/switch_example/main/onboarding_config.json
-rw-r—r— 1 501 dialout 489 Dec 13 20:23 /st-device-sdk-c-ref/apps/esp32/switch_example/main/onboarding_config.json
$ docker exec e1ae chown root:root /st-device-sdk-c-ref/apps/esp32/switch_example/main/onboarding_config.json
$ docker exec e1ae ls -al /st-device-sdk-c-ref/apps/esp32/switch_example/main/onboarding_config.json
-rw-r—r— 1 root root 489 Dec 13 20:23 /st-device-sdk-c-ref/apps/esp32/switch_example/main/onboarding_config.json
```

- Biên dịch với các tệp cấu hình thiết bị đã cập nhật (device_info.json và onboarding_config.json)

```
$ docker ps
CONTAINER ID   IMAGE           COMMAND        CREATED        STATUS          PORTS          NAMES
ad7b99273e5e   stdk_esp32_dev  "bash"         2 hours ago    Up 13 minutes                  blissful_shtern
$ docker attach ad7b
root@ad7b99273e5e:/st-device-sdk-c-ref# ./build.py esp32 switch_example
... Strings omitted ...
[ 97%] Linking CXX static library libmain.a
[ 98%] Built target __idf_main
[ 98%] Generating ld/esp32.project.ld
[ 98%] Built target __ldgen_output_esp32.project.ld
[100%] Linking CXX executable switch_example.elf
[100%] Built target switch_example.elf
[100%] Generating binary image from built executable
esptool.py v2.8
Generated /st-device-sdk-c-ref/apps/esp32/switch_example/build/switch_example.bin
[100%] Built target gen_project_binary
[100%] Built target app

Project build complete. To flash, run this command:
/root/.espressif/python_env/idf4.0_py3.8_env/bin/python ../../../bsp/esp32/components/esptool_py/esptool/esptool.py -p (PORT) -b 460800 --
before default_reset --after hard_reset write_flash --flash_mode dio --flash_size detect --flash_freq 40m 0x1000
build/bootloader/bootloader.bin 0x8000 build/partition_table/partition-table.bin 0x11000 build/ota_data_initial.bin 0x20000
build/switch_example.bin
or run 'idf.py -p (PORT) flash'
————————————————————————————
binary path : /st-device-sdk-c-ref/output/esp32/iotcore_switch_example_20211213_3df85fe_57a3f55
————————————————————————————
root@ad7b99273e5e:/st-device-sdk-c-ref#
```

- Kiểm tra các tệp firmware đầu ra và chuẩn bị công cụ nạp chương trình (esptool)

```
$ cd output/esp32/iotcore_switch_example_latest
$ ls -al
total 1952
drwxr-xr-x  7 nani  staff    224 12 13 17:17 .
drwxr-xr-x  4 nani  staff    128 12 13 17:17 ..
-rw-r—r—  1 nani  staff   24672 12 13 17:17 bootloader.bin
drwxr-xr-x  8 nani  staff    256 12 13 17:17 debug
-rw-r—r—  1 nani  staff   8192 12 13 17:17 ota_data_initial.bin
-rw-r—r—  1 nani  staff   3072 12 13 17:17 partition-table.bin
-rw-r—r—  1 nani  staff  955696 12 13 17:17 switch_example.bin
$ git clone https://github.com/espressif/esptool.git
Cloning into 'esptool'...
remote: Enumerating objects: 3225, done.
remote: Counting objects: 100% (262/262), done.
remote: Compressing objects: 100% (215/215), done.
remote: Total 3225 (delta 162), reused 144 (delta 47), pack-reused 2963
Receiving objects: 100% (3225/3225), 10.25 MiB | 7.71 MiB/s, done.
Resolving deltas: 100% (2032/2032), done.
Checking connectivity... done.
```

- Cài đặt trình điều khiển cổng nối tiếp USB cho Windows/Mac/Linux
  - https://www.silabs.com/developers/usb-to-uart-bridge-vcp-drivers

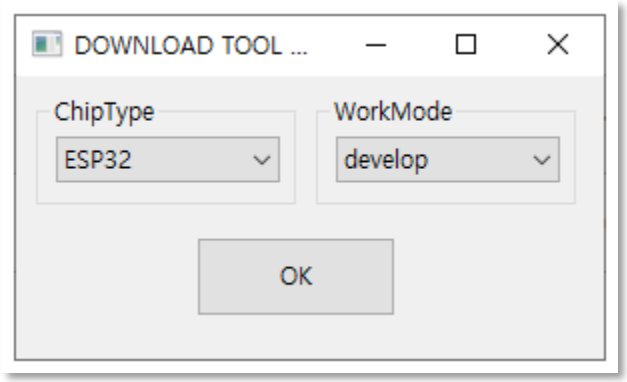


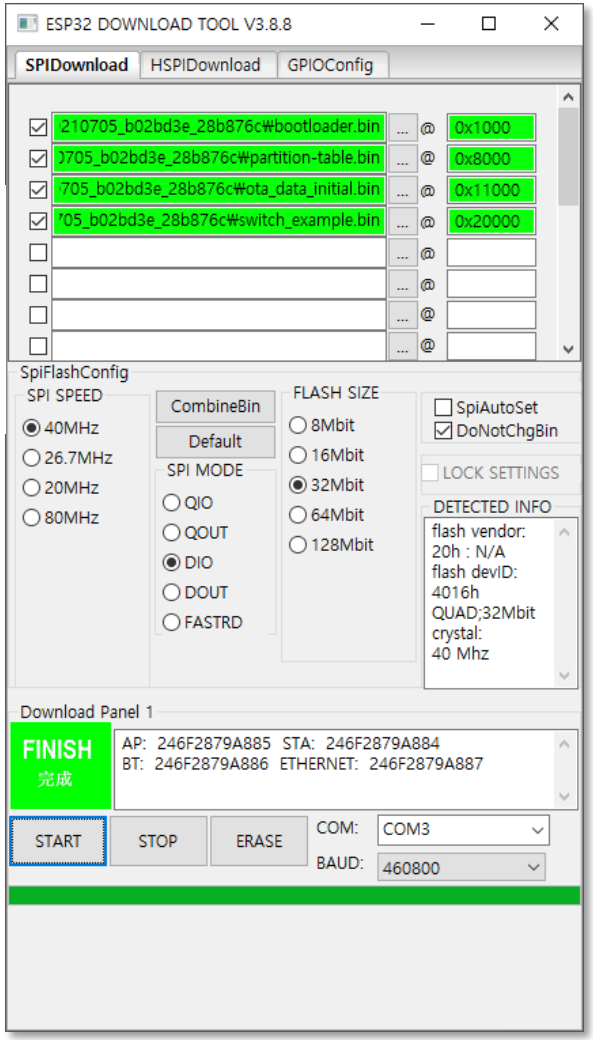
- Sử dụng công cụ nạp chương trình ESP cho Windows
  - https://www.espressif.com/en/support/download/other-tools

Flash Download Tools

Expand all + Download selected

	Title	Platform	Version	Release Date	Download
+	Flash Download Tools	Windows PC	V3.9.2	2021.11.10	





- Sử dụng công cụ dòng lệnh nạp ESP cho Mac/Linux (1/2)

```
$ pwd
/YOUR_ABSOLUTE_PATH/stdk-docker-build/output/esp32/iotcore_switch_example_latest
$ python3 ../../../esptool/esptool.py -p /dev/tty.SLAB_USBtoUART erase_flash
esptool.py v3.3-dev
Serial port /dev/tty.SLAB_USBtoUART
Connecting.....
Detecting chip type... Unsupported detection protocol, switching and trying again...
Connecting.....
Detecting chip type... ESP32
Chip is ESP32-D0WD (revision 1)
Features: WiFi, BT, Dual Core, Coding Scheme None
Crystal is 40MHz
MAC: 30:ae:a4:97:48:d8
Uploading stub...
Running stub...
Stub running...
Erasing flash (this may take a while)...
Chip erase completed successfully in 8.5s
Hard resetting via RTS pin...
$ python3 ../../../esptool/esptool.py -p /dev/tty.usbserial-0001 -b 460800 --before default_reset --after hard_reset write_flash --flash_mode dio
--flash_size detect --flash_freq 40m 0x1000 ./bootloader.bin 0x8000 ./partition-table.bin 0x11000 ./ota_data_initial.bin 0x20000
./switch_example.bin
esptool.py v3.3-dev
Serial port /dev/tty.usbserial-0001
Connecting.....
Detecting chip type... Unsupported detection protocol, switching and trying again...
Connecting....
Detecting chip type... ESP32
Chip is ESP32-D0WDQ6 (revision 1)
```

- Sử dụng công cụ dòng lệnh nạp ESP cho Mac/Linux (2/2)

```
Features: WiFi, BT, Dual Core, 240MHz, VRef calibration in efuse, Coding Scheme None
Crystal is 40MHz
MAC: YOUR_ESP32_MAC_ADDRESS
Uploading stub...
Running stub...
Stub running...
Changing baud rate to 460800
Changed.
Configuring flash size...
Auto-detected Flash size: 4MB
Flash will be erased from 0x00001000 to 0x00007fff...
Flash will be erased from 0x00008000 to 0x00008fff...
Flash will be erased from 0x00011000 to 0x00012fff...
Flash will be erased from 0x00020000 to 0x00109fff...
Compressed 24672 bytes to 14685...
Wrote 24672 bytes (14685 compressed) at 0x00001000 in 0.8 seconds (effective 250.0 kbit/s)...
Hash of data verified.
Compressed 3072 bytes to 151...
Wrote 3072 bytes (151 compressed) at 0x00008000 in 0.1 seconds (effective 247.9 kbit/s)...
Hash of data verified.
Compressed 8192 bytes to 31...
Wrote 8192 bytes (31 compressed) at 0x00011000 in 0.1 seconds (effective 451.2 kbit/s)...
Hash of data verified.
Compressed 955648 bytes to 616526...
Wrote 955648 bytes (616526 compressed) at 0x00020000 in 15.1 seconds (effective 505.3 kbit/s)...
Hash of data verified.

Leaving...
Hard resetting via RTS pin...
```

- Kiểm tra trạng thái khởi động của ESP32 có bình thường hay không (1/3)

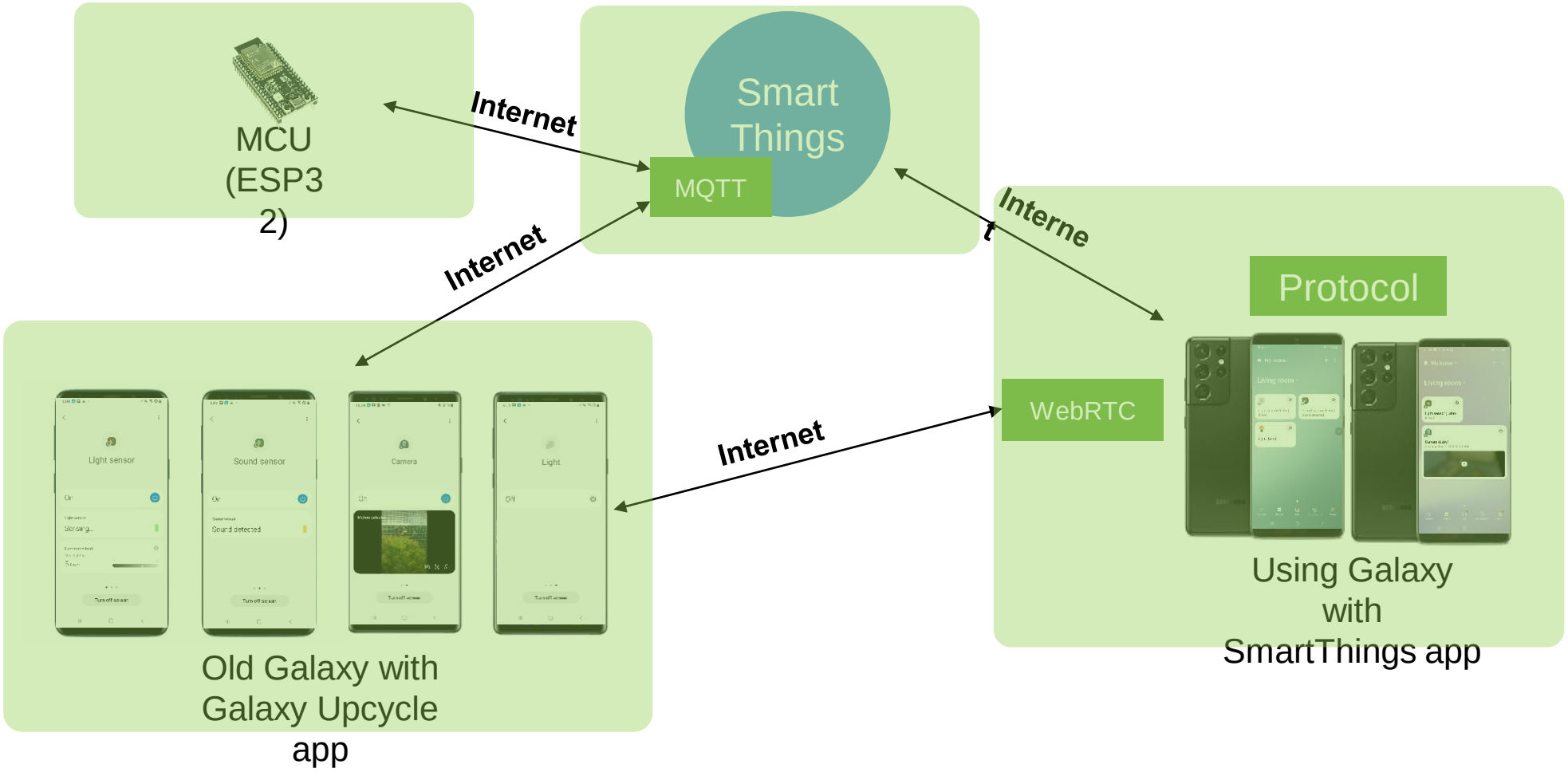
```
$ miniterm.py --raw --eol CR --encoding ascii /dev/tty.SLAB_USBtoUART 115200
--- Miniterm on /dev/tty.SLAB_USBtoUART  115200,8,N,1 ---
--- Quit: Ctrl+] | Menu: Ctrl+T | Help: Ctrl+T followed by Ctrl+H ---
MSH?HH???0x1 (POWERON_RESET),boot:0x13 (SPI_FAST_FLASH_BOOT)
configsip: 0, SPIWP:0xee
clk_drv:0x00,q_drv:0x00,d_drv:0x00,cs0_drv:0x00,hd_drv:0x00,wp_drv:0x00
mode:DIO, clock div:2
load:0x3fff0018,len:4
load:0x3fff001c,len:5676
load:0x40078000,len:14440
load:0x40080400,len:4460
entry 0x4008069c
I (339) cpu_start: Pro cpu up.
I (339) cpu_start: Application information:
I (339) cpu_start: Project name:     switch_example
I (342) cpu_start: App version:      v1.7.0-29-g3df85fe-dirty
I (349) cpu_start: Compile time:     Dec 13 2021 16:45:39
I (355) cpu_start: ELF file SHA256:  1a545297fa99736a...
I (361) cpu_start: ESP-IDF:          v4.0.1-316-g48ea44f3d1
I (367) cpu_start: Single core mode
I (371) heap_init: Initializing. RAM available for dynamic allocation:
I (378) heap_init: At 3FFAE6E0 len 00001920 (6 KiB): DRAM
I (384) heap_init: At 3FFB9268 len 00026D98 (155 KiB): DRAM
I (391) heap_init: At 3FFE0440 len 0001FBC0 (126 KiB): D/IRAM
I (397) heap_init: At 40078000 len 00008000 (32 KiB): IRAM
I (403) heap_init: At 400956BC len 0000A944 (42 KiB): IRAM
I (409) cpu_start: Pro cpu start user code
I (427) spi_flash: detected chip: generic
I (427) spi_flash: flash io: qio
```

- Kiểm tra trạng thái khởi động của ESP32 có bình thường hay không (2/3)

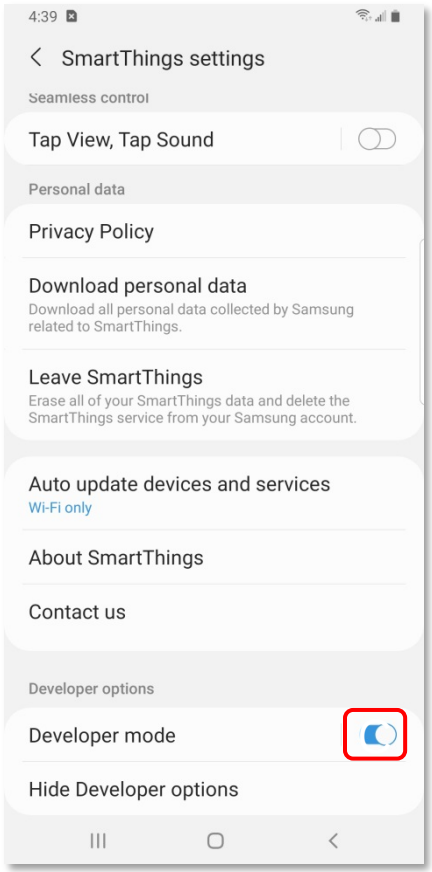
```
I (427) cpu_start: Starting scheduler on PRO CPU.
I (451) [IoT]: _dump_device_info(706) > firmware_version: V20211213
I (451) [IoT]: iot_bsp_wifi_init(178) > [esp32] iot_bsp_wifi_init
I (461) wifi:wifi driver task: 3ffc2478, prio:23, stack:6656, core=0
I (461) system_api: Base MAC address is not set, read default base MAC address from BLK0 of EFUSE
I (461) system_api: Base MAC address is not set, read default base MAC address from BLK0 of EFUSE
I (481) wifi:wifi firmware version: 9156a0d
I (481) wifi:config NVS flash: enabled
I (481) wifi:config nano formating: disabled
I (491) wifi:Init data frame dynamic rx buffer num: 32
I (491) wifi:Init management frame dynamic rx buffer num: 32
I (501) wifi:Init management short buffer num: 32
I (501) wifi:Init dynamic tx buffer num: 32
I (501) wifi:Init static rx buffer size: 1600
I (511) wifi:Init static rx buffer num: 10
I (511) wifi:Init dynamic rx buffer num: 32
I (521) [IoT]: iot_bsp_wifi_init(216) > [esp32] iot_bsp_wifi_init done
I (521) [IoT]: st_conn_init(1372) > previous error code[]
I (531) [IoT]: st_conn_init(1380) > stdk_version : 1.7.4
I (541) gpio: GPIO[12]| InputEn: 0| OutputEn: 1| OpenDrain: 0| Pullup: 0| Pulldown: 1| Intr:0
I (541) gpio: GPIO[26]| InputEn: 0| OutputEn: 1| OpenDrain: 0| Pullup: 0| Pulldown: 1| Intr:0
I (551) gpio: GPIO[14]| InputEn: 0| OutputEn: 1| OpenDrain: 0| Pullup: 0| Pulldown: 1| Intr:0
I (561) gpio: GPIO[27]| InputEn: 0| OutputEn: 1| OpenDrain: 0| Pullup: 0| Pulldown: 1| Intr:0
I (571) gpio: GPIO[13]| InputEn: 1| OutputEn: 0| OpenDrain: 0| Pullup: 1| Pulldown: 0| Intr:3
I (2581) [IoT]: st_conn_start(1850) > st_conn_start start (no-pin)
I (2581) [IoT]: _do_iot_main_command(499) > curr_main_cmd:10, curr_main_state:-1/-1
I (2581) [IoT]: _do_update_timeout(200) > Current timeout : 100000 for -1/0
I (2591) [IoT]: _do_status_report(304) > Call usr status_cb with 1/0
status: 1, stat: 0
```

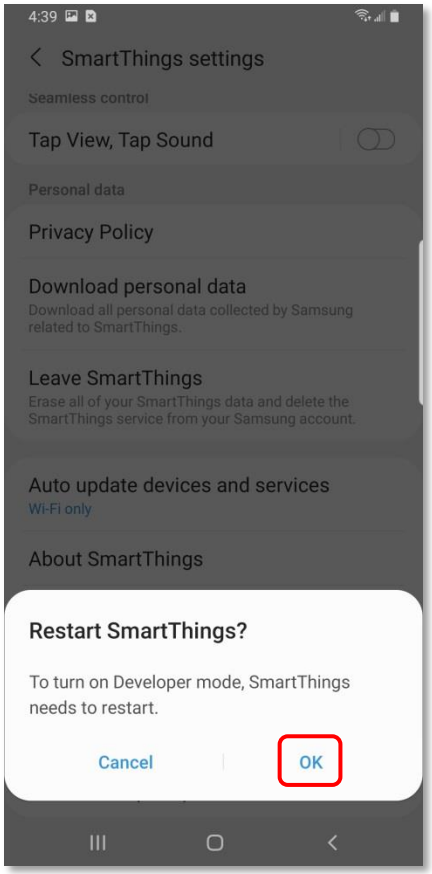
- Kiểm tra trạng thái khởi động của ESP32 có bình thường hay không (3/3)

```
I (2601) [IoT]: _do_iot_main_command(499) > curr_main_cmd:2, curr_main_state:-1/0
I (2611) [IoT]: _do_cmd_tout_check(1041) > New state updated for 0
I (2611) [IoT]: _do_iot_main_command(499) > curr_main_cmd:10, curr_main_state:0/0
I (2621) [IoT]: iot_bsp_wifi_set_mode(234) > iot_bsp_wifi_set_mode = 1
I (2711) phy: phy_version: 4180, cb3948e, Sep 12 2019, 16:39:13, 0, 0
I (2711) wifi:mode : sta (30:ae:a4:97:48:d8)
I (2711) [IoT]: iot_bsp_wifi_set_mode(265) > WiFi Station Started
I (4761) [IoT]: event_handler(167) > event_handler = 1
I (4811) [IoT]: iot_easysetup_create_ssid(91) > >> SIC GalUp Thi[eips] <<
I (4811) [IoT]: _do_update_timeout(200) > Current timeout : 300000 for 0/1
I (4811) [IoT]: _do_iot_main_command(499) > curr_main_cmd:1, curr_main_state:0/1
I (4821) [IoT]: iot_bsp_wifi_set_mode(234) > iot_bsp_wifi_set_mode = 3
I (4831) wifi:mode : sta (YOUR_ESP32_WIFI_MAC) + softAP (YOUR_ESP32_SOFTAP_MAC)
I (4831) wifi:Total power save buffer number: 16
I (4841) wifi:Init max length of beacon: 752/752
I (4841) wifi:Init max length of beacon: 752/752
I (5821) wifi:Total power save buffer number: 16
I (5831) [IoT]: event_handler(135) > SYSTEM_EVENT_AP_START
I (5831) [IoT]: iot_bsp_wifi_set_mode(382) > AP Mode Started
I (5831) [IoT]: iot_easysetup_init(494) > IOT_STATE_PROV_ES_START
I (5841) [IoT]: es_http_init(159) > http tcp init!!
I (5841) [IoT]: iot_easysetup_init(531) > IOT_STATE_PROV_ES_INIT_DONE
I (5851) [IoT]: event_handler(140) > SYSTEM_EVENT_AP_STOP
I (5861) [IoT]: event_handler(135) > SYSTEM_EVENT_AP_START
I (5861) [IoT]: _do_cmd_tout_check(1041) > New state updated for 1
I (5871) [IoT]: _do_status_report(304) > Call usr status_cb with 2/1
status: 2, stat: 1
```


MCU
(ESP3
2)
Internet
Smart
Things
MQTT
Internet
Interne
t
Protocol
WebRTC
Internet
Light sensor
Sound sensor
Sound detected
Camera
Light
Old Galaxy with
Galaxy Upcycle
app
Using Galaxy
with
SmartThings app

- Bật chế độ dành cho nhà phát triển

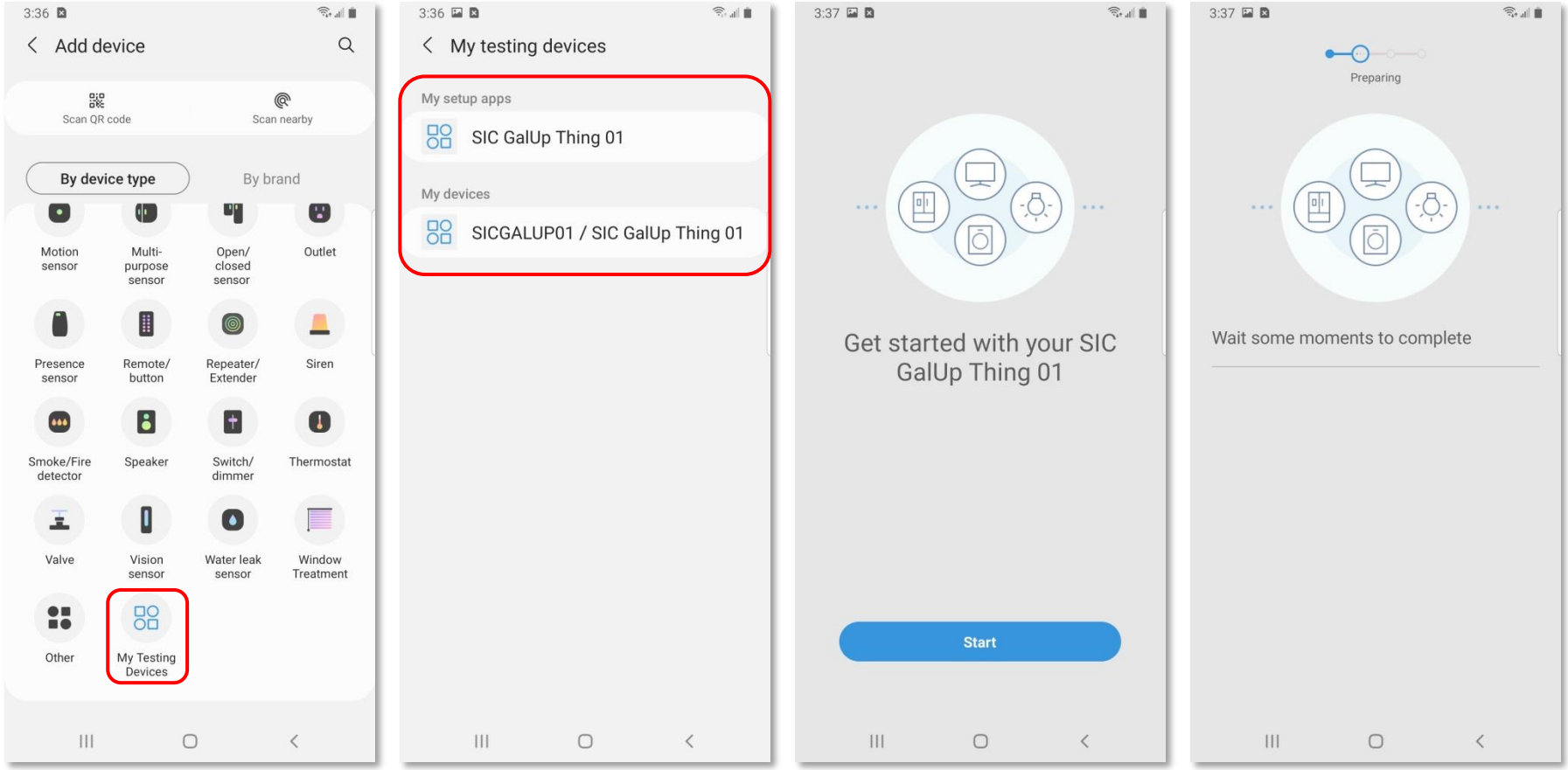




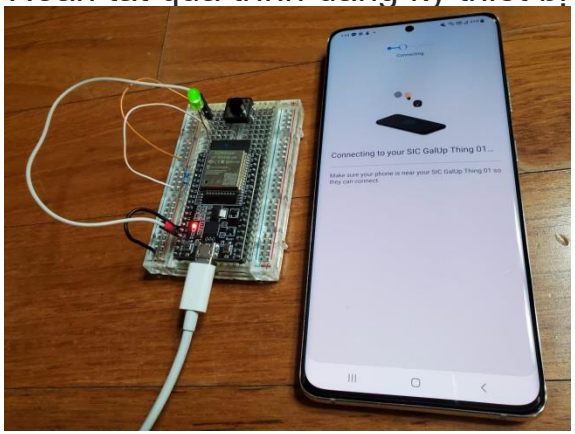
Dưới đây là 5 bước đơn giản để bật chế độ dành cho nhà phát triển trên ứng dụng SmartThings:

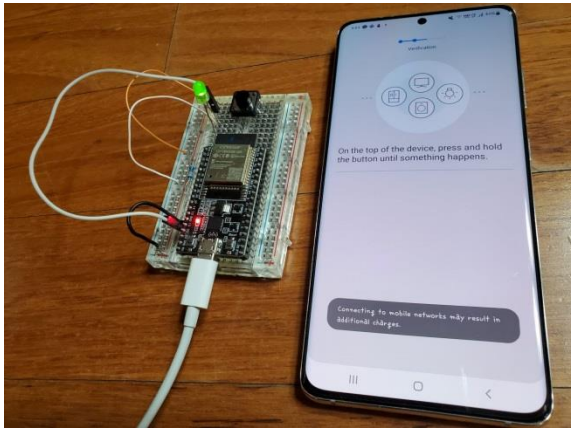
1. Mở ứng dụng SmartThings
2. Vào Bảng điều khiển > Cài đặt > Giới thiệu SmartThings
3. Nhấn giữ mục “Giới thiệu SmartThings” trong 5 giây
4. Chọn “Chế độ nhà phát triển”
5. Mở lại ứng dụng SmartThings và đảm bảo bạn đã đăng nhập

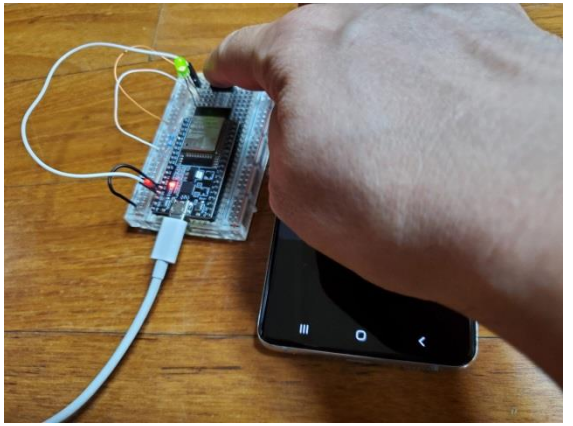
- Bắt đầu đăng ký thiết bị từ menu danh mục

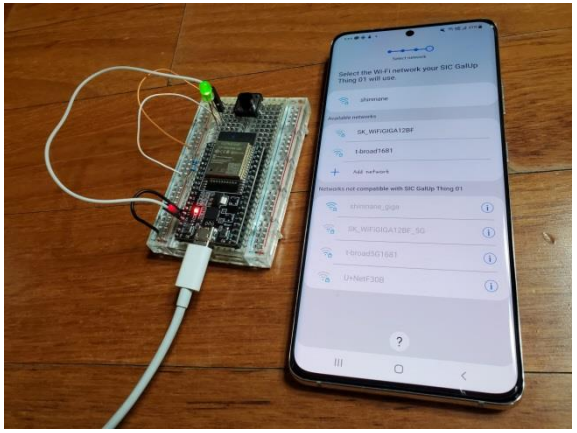


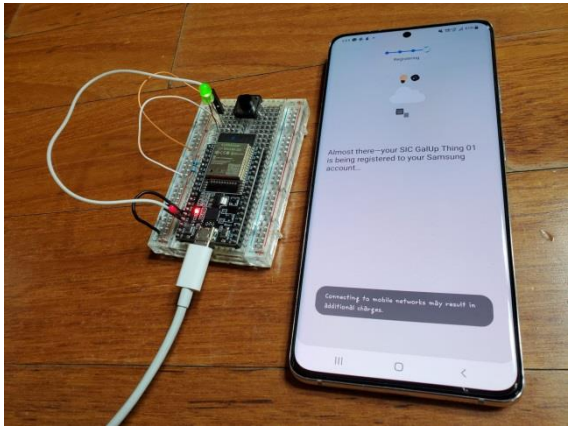
- Hoàn tất quá trình đăng ký thiết bị

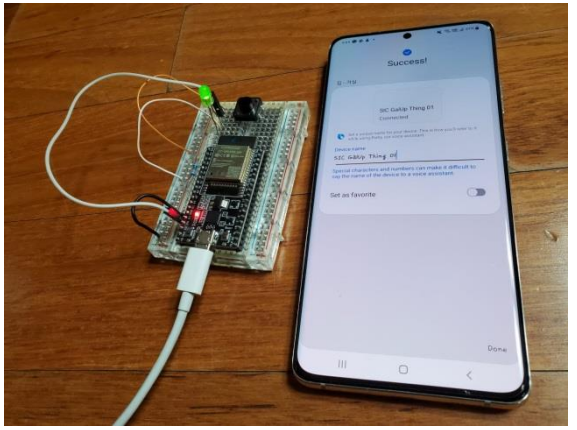






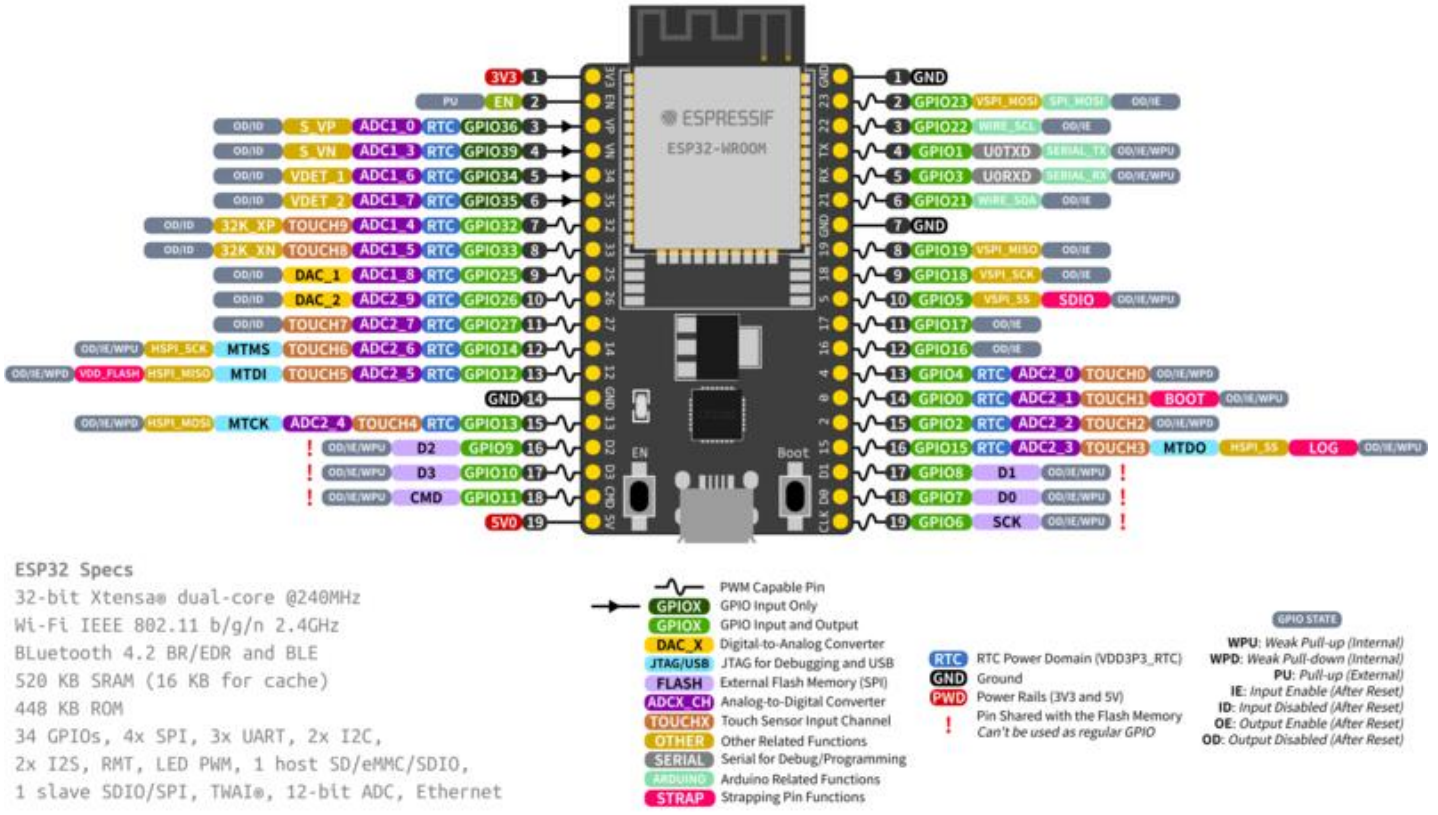






- Kiểm tra sơ đồ chân của ESP32 DevKitC V4

ESP32-DevKitC



ESP32 Specs
32-bit Xtensa® dual-core @240MHz
Wi-Fi IEEE 802.11 b/g/n 2.4GHz
BLuetooth 4.2 BR/EDR and BLE
520 KB SRAM (16 KB for cache)
448 KB ROM
34 GPIOs, 4x SPI, 3x UART, 2x I2C,
2x I2S, RMT, LED PWM, 1 host SD/eMMC/SDIO,
1 slave SDIO/SPI, TWAI®, 12-bit ADC, Ethernet

- Kiểm tra sơ đồ chân GPIO để điều khiển đèn LED

```
root@073d1c2b5fb9:/st-device-sdk-c-ref/apps/esp32/switch_example/main# cat device_control.h
...

//#define CONFIG_TARGET_WEMOS_D1_R32
#ifdef CONFIG_TARGET_WEMOS_D1_R32

#define GPIO_INPUT_BUTTON                18

#define GPIO_OUTPUT_MAINLED              16
#define GPIO_OUTPUT_MAINLED_0            26 /* use as ground */

#define GPIO_OUTPUT_NOUSE1               17
#define GPIO_OUTPUT_NOUSE2               25
#else // ESP32_DEVKITC_V4

#define GPIO_INPUT_BUTTON                13

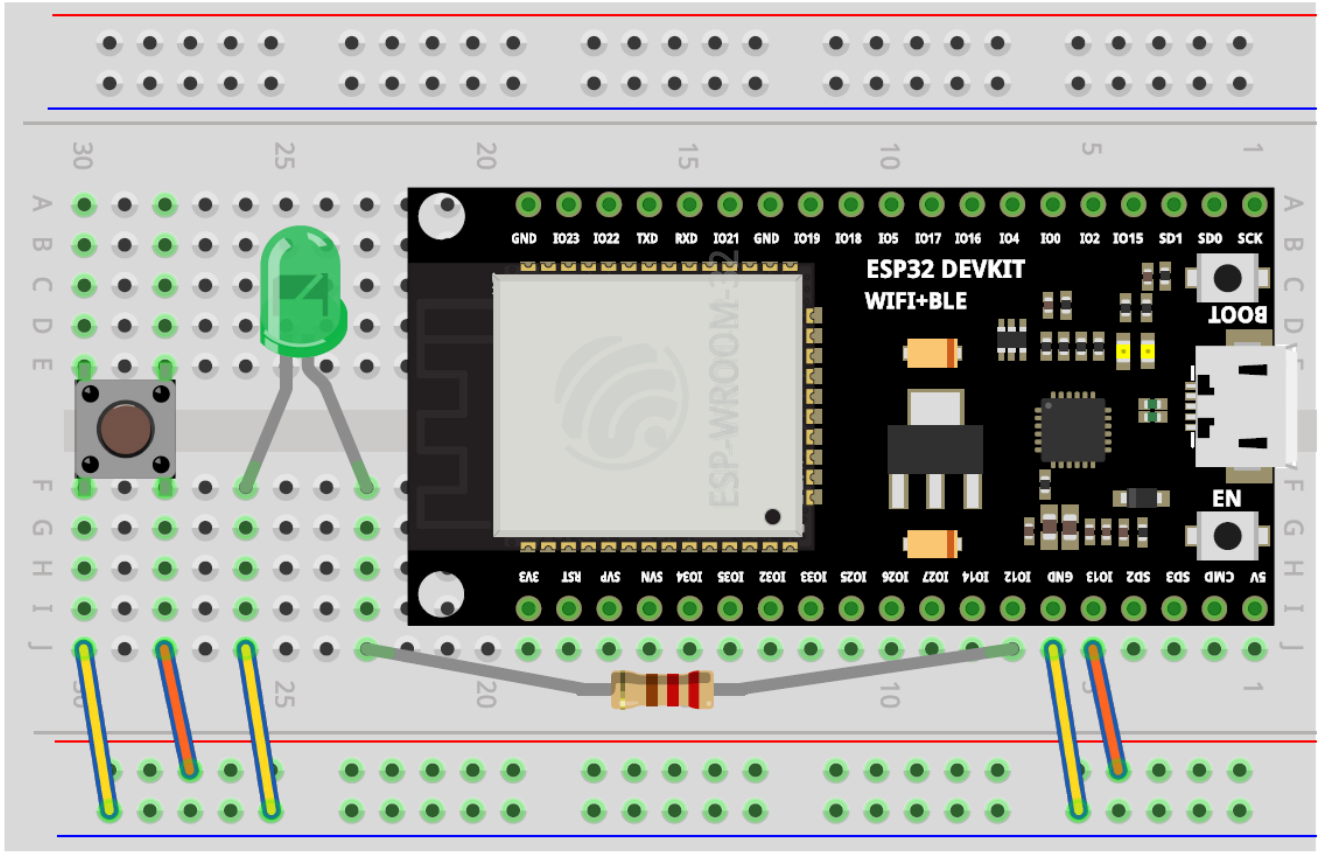
#define GPIO_OUTPUT_MAINLED              12
#define GPIO_OUTPUT_MAINLED_0            26 /* use as ground */

#define GPIO_OUTPUT_NOUSE1               14
#define GPIO_OUTPUT_NOUSE2               27

#endif

...
```

- Kiểm tra sơ đồ đấu nối mạch trên breadboard



- Kiểm tra đèn LED ngoài đã đấu nối và thử các kịch bản tự động với thiết bị IoT trong ứng dụng Galaxy Upcycle



- Giờ đây bạn đã hiểu cách dữ liệu điều khiển và sự kiện IoT được truyền trong hệ thống SmartThings thông qua ví dụ thực hành.

- Bạn có thể tự kiểm tra các thao tác điều khiển và sự kiện của mình, đồng thời kết hợp nhiều kịch bản tự động khác nhau.Để nâng cao hơn, bạn có thể sử dụng MCU để phát triển thiết bị IoT cho các ứng dụng thực tế với STDK. Bạn hoàn toàn có thể tự xây dựng bất kỳ thiết bị IoT nào theo nhu cầu của mình.


End of
Document



ⓒ2023 SAMSUNG. All rights reserved.

Samsung Electronics Corporate Citizenship Office holds the copyright of book.

This book is a literary property protected by copyright law so reprint and reproduction without permission are prohibited.
To use this book other than the curriculum of Samsung Innovation Campus or to use the entire or part of this book, you must receive written consent from copyright holder.